

分类号

学校代码 10487

学号 M202176632

密级

华中科技大学

硕士学位论文

(学术型 ☐ 专业型 ☒)

基于 BERT 的医学智能问答关键算法
的研究

学位申请人：张磊

学科专业：软件工程

指导教师：苏曙光 副教授

答辩日期：2024 年 7 月 4 日

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Master Degree in Engineering**

**Research on key algorithms for medical intelligent
question answering based on BERT**

Candidate : Zhang Lei

Major : Software Engineering

Supervisor : Su ShuGuang

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

May, 2024

摘要

近年来,人们的健康问题日益突显,关于医学领域的问答系统备受关注;与此同时,传统问答系统依赖模板和关键词匹配,对于复杂和模糊问题表现不佳,难以满足多变的查询需求和非结构化数据的挑战。因此,急需新技术和策略来突破目前的局限。

在数据预训练阶段,针对句子级处理的准确性不足的问题,提出一种结合 BERT、LSTM 和 CRF 的模型框架。该框架结合了三种技术的优点,充分利用 BERT 预训练模型的优势,通过 BERT 的强大语义表示能力来深入理解句子中的语义信息。在此基础上,使用 BERT 计算问句与属性之间的相似度,从而准确地进行属性选择。

针对实体识别任务,提出通过结合数据增强策略和 TF-IDF 实体对齐技术,提升模型在该任务上的性能,并确保模型输出与知识图谱的高度一致性。针对实体匹配精度不足的问题,提出了采用相似度匹配优化的方法,通过计算输入语句的词组与实体特征词之间的字重叠率、词向量余弦相似度以及 DP 编辑距离,将这三个指标的平均值作为相似度评分,并选择相似度最高且超过设定阈值的实体特征词作为匹配结果。

为了进一步提升意图识别的准确性,提出采用 34B 大模型进行意图识别的技术方法。通过对比了三种意图识别方案,即规则匹配、训练模型与提示工程,在准确性、多意图辨识能力、人力成本、推理效率及资源占用等多个维度上的差异,选定提示工程为首选策略。为了进一步精细化处理,结合模板匹配的方法,通过定义一系列与医学领域相关的模板,可以对识别出的意图进行进一步的校验和调整。

在某市卫健委健康教育科普馆项目中,集成和应用了医疗智能问答功能,设计了一个基于 RAG 与大模型技术的医疗问答系统,利用 DiseaseKG 数据集与 Neo4j 构建知识图谱,结合 BERT 的命名实体识别和 34b 大模型的意图识别,通过精确的知识检索和问答生成,提升系统在医疗咨询中的性能,解决大模型在医疗领域应用的可靠性问题。

关键词: BERT; 实体识别; 相似度匹配; 意图识别; RAG; 知识图谱

Abstract

In recent years, people's health issues have become increasingly prominent, and question answering systems in the medical field have received much attention; At the same time, traditional question answering systems rely on templates and keyword matching, which perform poorly on complex and ambiguous questions, making it difficult to meet the changing query requirements and challenges of unstructured data. Therefore, there is an urgent need for new technologies and strategies to overcome current limitations.

In the data pre training stage, a model framework combining BERT, LSTM, and CRF is proposed to address the issue of insufficient accuracy in sentence level processing. This framework combines the advantages of three technologies and fully utilizes the strengths of BERT pre trained models to deeply understand semantic information in sentences through BERT's powerful semantic representation capabilities. On this basis, BERT is used to calculate the similarity between the question and the attribute, in order to accurately select the attribute.

For entity recognition tasks, it is proposed to improve the performance of the model on this task by combining data augmentation strategies and TF-IDF entity alignment techniques, while ensuring high consistency between the model output and the knowledge graph. Aiming at the problem of insufficient accuracy in entity matching, a similarity matching optimization method is proposed. By calculating the word overlap rate, cosine similarity of word vectors, and DP editing distance between the input sentence phrase and entity feature words, the average of these three indicators is used as the similarity score, and the entity feature words with the highest similarity and exceeding the set threshold are selected as the matching results.

In order to further improve the accuracy of intent recognition, a technical method of using 34B large model for intent recognition is proposed. By comparing three intent recognition schemes, namely rule matching, model training, and prompt engineering, and comparing their differences in accuracy, multi intent recognition ability, labor cost, inference efficiency, and resource utilization, prompt engineering was selected as the preferred strategy. Then, in order to further refine the processing, a template matching method was combined to define a series of templates related to the medical field, which can

further verify and adjust the identified intentions.

In the project of the Health Education Science Popularization Museum of a certain city's Health Inspection Commission, the medical intelligent question answering function was integrated and applied. A medical question answering system based on RAG and large model technology was designed. A knowledge graph was constructed using the DiseaseKG dataset and Neo4j, combined with BERT's named entity recognition and the intention recognition of the 34b large model. Through accurate knowledge retrieval and question answering generation, the performance of the system in medical consultation was improved, and the reliability problem of large model application in the medical field was solved.

Keywords: BERT; Entity recognition; Similarity matching; Intention recognition; RAG; knowledge graph

目 录

1 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状概述.....	2
1.3 存在的问题.....	7
1.4 本文主要内容.....	8
2 智能问答算法的研究基础	11
2.1 RAG	11
2.2 网络爬虫技术.....	12
2.3 知识图谱.....	13
2.4 词向量技术.....	19
2.5 Transformer 神经网络.....	21
2.6 本章小结.....	22
3 基于深度学习的智能问答模型	23
3.1 命名实体识别技术.....	23
3.2 基于 BERT 的命名实体识别.....	31
3.3 基于 BIOES 标注的 NER 模型训练.....	38
3.4 基于数据增强策略和 TF-IDF 算法的模型改进.....	42
3.5 基于 ADAM 优化器的 BERT 算法优化	44
3.6 实验结果及分析.....	52
3.7 本章小结.....	57
4 基于 BERT 预训练模型的中文文本分类	58
4.1 基于 padding 与 mask 的数据预处理	58
4.2 加载 BERT 预训练模型.....	67
4.3 基于前向传播的文本分类	73
4.4 本章小结.....	78
5 基于 RAG 和大模型技术的医疗智能问答系统.....	79
5.1 系统架构与工作流程.....	79
5.2 知识数据获取.....	80
5.3 基于 NEO4J 的知识图谱可视化.....	82
5.4 基于 34B 大模型的意图识别.....	83
5.5 医学领域的智能问答.....	87
5.6 本章小结.....	90
6 总结及展望.....	92
6.1 总结.....	92

6.2 展望.....	93
参考文献.....	95

1 绪论

1.1 研究背景及意义

在当今信息时代的背景下，智能问答技术的重要性日益凸显。随着互联网技术的迅猛发展和大数据时代的全面到来，个体及组织普遍面临着信息过载的严峻挑战。尽管海量的数据被持续创建与广泛收集，但如何高效、精准地从这些数据中提取有效的信息，仍是一个亟待深入探索与实践解决的学术与实践难题。智能问答系统作为应对这一挑战的有效手段，其研究与应用具有重要的现实意义与广阔的前景。

智能问答算法作为一种组织和处理大量信息的有效手段，为解决复杂信息处理问题提供了可能。其主要通过将数据组织成相互连接的图形网络实现。在此网络中，节点代表实体（如个人、地点、组织），而边则代表实体间的关系^[1]。这种结构使得信息的存储既直观又高效，极大提升了对海量数据的查询与分析能力。知识图谱的出现，尤其是在语义网络和本体论的基础上，为机器提供了理解和处理复杂数据的新方式，这对于人工智能领域具有重要意义，推动了自然语言处理、机器学习及数据挖掘等技术领域的发展，并使其成为构建和优化此类系统的关键。例如，自然语言处理技术使系统能够理解以自然语言形式提出的问题；而机器学习技术则使系统能够从用户交互中学习并优化其性能。不仅如此，智能问答系统应用十分广泛。在商业领域，它们可用于提供客户服务，通过快速准确地回答客户的问题来提高用户满意度和效率；在医疗领域，它们有助于医生和研究人员快速获取与疾病、药物及治疗方法相关的信息^[2]。在教育领域，这些系统可作为教学辅助工具，帮助学生和教师访问并理解丰富的教育资源。尤其是基于知识图谱的系统，利用这些图谱来响应用户的查询。与传统基于关键词搜索的引擎不同，这些系统能够理解用户的问题，并通过在知识图谱中寻找相关信息来提供更准确、更有深度的答案。这种方式使得回答不仅仅是关于检索信息，而更多地涉及理解和解释信息。

1.2 国内外研究现状概述

智能问答算法的发展历程可追溯至上个世纪 60 年代，其初期探索主要聚焦于基于规则的系统上，例如 ELIZA 程序，其通过模拟心理治疗师的交互模式来响应用户输入；进入 70 年代和 80 年代，随着专家系统的兴起，基于规则的问答系统得以深化，然而这些系统往往依赖于详尽的手工编码及专家知识；至 21 世纪初，随着机器学习技术，尤其是深度学习的兴起，智能问答系统经历了革命性的转变，自然语言处理技术的进步使得系统能够更有效地理解和处理自然语言，进而提供更加准确且具备上下文关联性的答案^[4]；近年来，预训练的大型语言模型如 BERT 和 GPT 系列在智能问答领域展示了卓越的性能，这些模型能够理解复杂的查询并生成更自然、更准确的回答。

在国内，智能问答算法领域的发展同样取得了显著进展，已成为该领域的核心参与者。伴随着人工智能技术的迅猛发展，中国的研究机构和企业积极参与智能问答系统的研究与应用，推动了该领域的创新与发展^[4]。以百度、阿里巴巴、腾讯、华为等为代表的科技巨头，在自然语言处理与深度学习领域投入巨大资源，成功开发了多款应用于搜索引擎、电子商务、客户服务、智能助手等多个场景的智能问答系统，如百度的 DuerOS、阿里巴巴的天猫精灵以及腾讯的微信智能助手，这些系统不仅在自然语言理解方面表现出色，还能够提供个性化的用户体验。在学术研究方面，中国的高等教育机构和研究单位亦在智能问答算法方面取得了重要突破^[5]。研究重点不仅涵盖基础算法的优化，如提高语义理解、深度挖掘上下文内容及多轮对话管理能力，并积极探索该技术在特定领域的应用，如医学诊断、法律咨询及教育智能化等。作为人工智能领域的重要组成部分，智能问答系统正处于快速发展和变革之中。该领域汇聚了全球科研人员的智慧与资源，不断推动技术前沿的发展。

深度学习作为智能算法的一个重要分支，在图像识别、语音处理及自然语言处理等多个领域已取得显著成果^[6]。特别是卷积神经网络(CNN)和循环神经网络(RNN)等模型架构，在各自领域表现出了卓越的性能^[7]。例如，BERT 和 GPT-3 等预训练模型在自然语言处理领域的应用，极大地推进了机器对人类语言的理解和生成能力，为智能问答技术的发展注入了新的活力。

1.2.1 大模型研究现状

随着人工智能技术的飞速发展，特别是深度学习领域的突破，大模型（Large Models）逐渐成为自然语言处理、计算机视觉等多个领域的研究热点^[8]。大模型通常指的是具有海量参数和复杂结构的深度学习模型，它们能够在大规模数据集上进行训练，从而学习到丰富的知识和表示能力^[9]。

1.2.1.1 大模型的兴起

大型模型的兴起主要归因于计算能力的增强与大规模数据集的可用性。近年来，GPU、TPU 等高性能计算硬件的广泛普及，加之互联网所催生的海量数据积累，研究者现能够训练参数规模达数十亿乃至上万亿的模型^[10]。这些大型模型在多项任务中实现了显著的性能突破，尤其在自然语言处理领域表现突出，推动了相关技术的进步与发展。

1.2.1.2 代表性大模型

在自然语言处理领域，诸如 BERT、GPT 系列（包括 GPT-3、GPT-4 等）和 T5 等代表性大型模型已经展示了卓越的语言理解和生成能力。这些模型通过在大规模文本数据上的预训练，成功学习到了丰富的语言表示，进而能够灵活应用于多样化的下游任务之中，包括但不限于文本分类、情感分析、问答系统等，极大地拓宽了自然语言处理技术的应用边界与深度^[11]。

BERT（Bidirectional Encoder Representations from Transformers），是由 Google 于 2018 年创新提出的一种基于 Transformer 结构的预训练语言模型。该模型巧妙地设计了遮蔽语言模型（Masked Language Model, MLM）与下一句预测（Next Sentence Prediction, NSP）两项预训练任务^[12]，从而在多项自然语言处理任务中展现出显著的性能优势。

GPT 系列是由 OpenAI 开发的一系列生成型预训练 Transformer 模型，其中包括 GPT-3、GPT-4 等版本。这些模型采用自回归语言建模（Autoregressive Language Modeling）的方法进行预训练，能够生成连贯且自然的文本，并被广泛应用于文本生成与对话系统等多个领域^[13]。

T5（Text-to-Text Transfer Transformer）由 Google 研发，该模型将各类自然语言

处理（NLP）任务框架统一为文本到文本的形式，并通过统一的预训练目标进行模型训练^[14]，T5 在多种 NLP 任务上展现了其通用性和高效性。

1.2.1.3 大模型的挑战与优势

大模型在自然语言处理领域展现出显著优势，其核心优势可归结为三大方面：一是强大的表示能力，通过在大规模数据集上的训练，大型模型能够学习到丰富的知识和表示能力，从而在多个 NLP 任务上实现卓越性能；二是卓越的泛化能力，大模型展现出良好的泛化特性，能够在未见数据上维持稳定的性能输出，体现其广泛的适用性；三是多任务处理能力，借助统一的预训练框架与灵活的微调策略，大模型能够高效应对并处理多样化的 NLP 任务需求^[15]。

然而，大模型的广泛应用亦伴随着不容忽视的挑战。首先，计算资源要求高，其训练与推理过程均需依赖高性能的 GPU 或 TPU 等硬件设备，导致资源消耗巨大；其次，数据需求庞大，为训练出高性能的大模型，需收集并处理大规模、高质量的数据集，对数据质量与数量均提出严格要求；最后，模型可解释性不足，大模型的复杂性与黑盒特性使得其决策过程难以被直观理解，这在某些需要高度透明度的应用场景中构成障碍^[16]。

1.2.1.4 未来发展趋势

随着技术的持续进步，大模型的研究与应用将呈现三大发展趋势：一是模型压缩与轻量化，研究者将致力于通过剪枝、量化、知识蒸馏等技术手段降低大模型的计算成本与资源消耗；二是多模态融合，未来的大型模型将更加注重多模态信息的融合，如文本、图像与语音等，以实现更加全面和智能化的交互；三是可解释性增强，研究者将探索新方法工具，以解析并可视化大模型的决策过程，提升模型的可解释性与透明度，从而拓展其应用范围与深度^[17]。

综上所述，大模型作为人工智能领域的重要研究方向，正在不断发展和完善。随着技术的不断进步和应用场景的不断拓展，大模型将在更多领域发挥重要作用。

1.2.2 传统问答方法

传统的问答匹配方法，主要在于将用户的问题与一组预定义的问题或信息库进行匹配，以找到最合适的答案^[18]。此类方法往往高度依赖于文本比对与关键词检索

策略，而非对查询意图的深层次解析。早期的问答系统多基于规则匹配，依赖于人工编写的规则来解析问题并匹配答案^[19]。此类系统通常有一个固定的问题和答案库，当用户提出的问题符合某一规则时，系统便会返回预先设定的答案。这种方法的优势在于其确定性和可控性，然而，其局限性亦显而易见，表现为灵活性不足，难以应对规则之外的复杂或变化性问题^[20]。

随着技术的进步，基于关键词的搜索引擎技术已被应用于问答系统中。这种方法依赖于从用户问题中提取关键词，并在数据库或互联网中搜索这些关键词以定位相关信息^[21]。相较于单一的规则匹配，此方法更为灵活，能够处理更为广泛的问题类型，但仍不足以理解问题的深层意图。为提升匹配的精确度，部分系统转而采用基于模板的匹配策略，该策略通过精心构建一系列问题模板，每个模板均针对特定类型的问题设计相应的答案^[22]。当用户的提问与某个模板相匹配时，系统即能输出预设答案。这一方法较之于纯规则匹配有所进步，因为它能够适应更为多样化的问题表述方式。这种方法相较于单一的规则匹配有所改进，能够适应更多样化的问题表述。此外，还有一些方法尝试运用简单的自然语言处理技术，如语法分析和同义词替换，以增强问题理解的准确性^[23]。这些技术虽能在一定程度上应对问题的变化和复杂性，但在理解和解析上下文及隐含意义方面依然存在局限性^[24]。

1.2.3 智能问答算法研究现状

近年来，预训练语言模型如 GPT-3.5 等在智能问答领域取得了显著成果。这些模型通过在大规模文本数据上进行预训练，学习了丰富的语言表示，随后可应用于多种下游任务，包括智能问答系统^[25]。这些模型的出现极大地提升了智能问答系统处理复杂自然语言查询的能力。智能问答算法文献的主要主题分布如图 1-1 所示。

基于 BERT 的医学智能问答系统通常包含若干关键步骤：首先，利用自然语言处理技术对用户提出的问题进行解析与理解；然后，模型在其训练过的知识库中检索相关信息；最后，系统依据检索到的数据生成相应的回答。此类系统的优点在于能够处理更为自然且多样化的语言表达形式，并提供精确而详尽的答案^[26]。同时，它们能够在一定程度上理解上下文，结合先前的对话内容来提供更合适的回答。此外，借助深度学习技术，这些问答系统具备持续学习与适应的能力，通过接触更多数据不断提

升自身性能，从而更好地理解 and 回应各类问题^[27]。然而，基于 BERT 的智能问答方法亦面临若干挑战，例如对大规模训练数据的需求、较高的计算资源消耗以及模型解释性的不足。此外，在处理特定领域的专业问题时，这些系统可能还需要额外的领域专业知识^[28]。

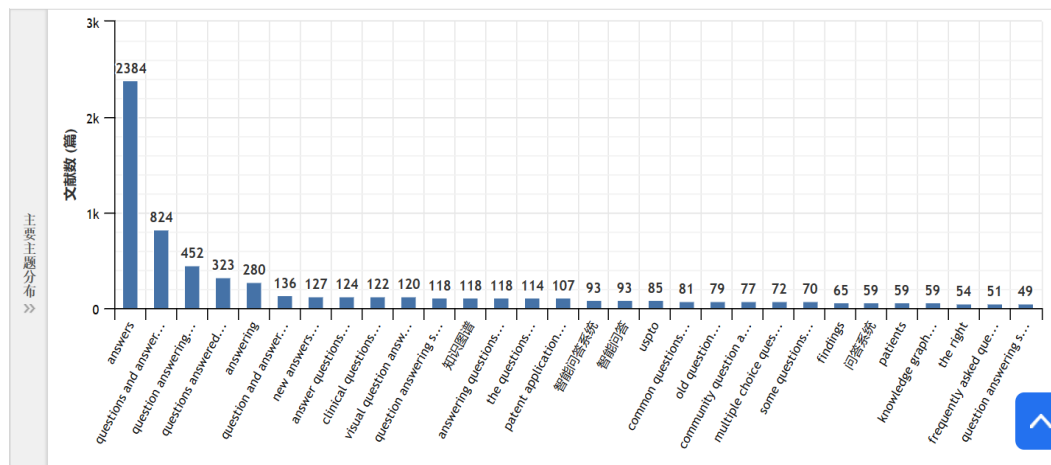


图 1-1 智能问答算法文献的主要主题分布

基于 Bert 的智能问答方法标志着智能问答技术的一大进步，其在理解和处理自然语言方面的能力远超过传统方法。随着技术的不断发展，这些系统有望在更多领域提供高效和准确的问答服务^[29]。

1.2.4 知识图谱研究现状

在计算机科学的发展历程中，知识图谱的概念与计算机的发明密切相关。据 Price 所述，知识图谱的概念在计算机理论形成的初期便已被提出^[30]。时至今日，知识图谱已经发展成为一个包含 5 亿实体和 35 亿实体属性的庞大网络^[31]。

在国内，对知识图谱的研究呈现出双向发展的态势。一方面，产业应用成为研究的重要方向，众多科技企业如海智星图等，以知识图谱为核心技术，推动了知识图谱在实际应用中的发展和创新^[32]。另一方面，理论研究也备受关注，不少大学教授致力于此，深入探究知识图谱的内在机制与应用潜力，并发表了多篇相关学术论文。这种理论与实践并重的研究模式，不仅促进了知识图谱技术的不断完善，也为其在更广泛领域的应用提供了坚实支撑。有数据显示，我国有关知识图谱的文献数量呈指数爆炸式增长，如图 1-2 所示：

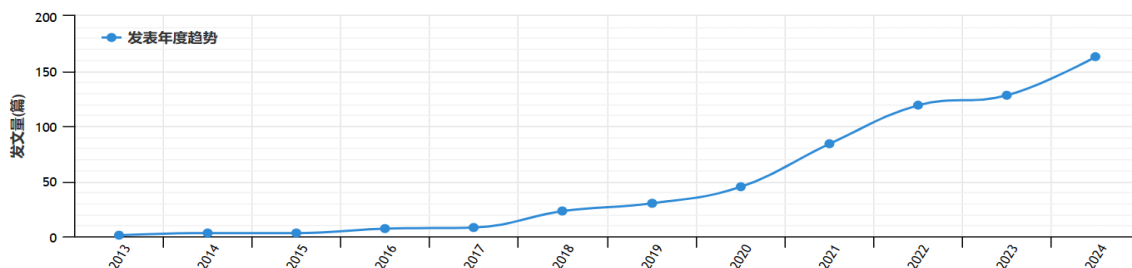


图 1-2 知识图谱的文献数量趋势

1.3 存在的问题

智能问答系统在智能化时代的应用日益广泛，并因其高效性和准确性而获得广泛认可。然而，传统的智能问答系统在实际应用中仍面临若干问题，这些问题限制了其进一步的发展与普及。

(1) 句子级的处理准确性不足。句子级处理的准确性不足是数据预训练的关键阶段面临的显著挑战。该问题的主要源于多个方面，包括但不限于对复杂句法结构理解的不足、对上下文语境把握的不够精准，以及潜在的数据噪声和偏差。句子级处理的准确性对于后续的自然语言处理任务至关重要，直接关系到模型对文本信息的提取与理解能力。因此，解决这一问题不仅需要持续优化算法模型，还要求在数据预处理阶段进行更为精细的工作，以确保输入数据的质量与准确性，从而为后续的训练过程奠定坚实基础^[33]。

(2) 实体匹配精度亦有欠缺。实体匹配精度的不足是数据处理与信息提取领域内一个值得深入关注的技术挑战。该问题通常发生在自动识别与关联来自不同来源或格式数据中的实体时，如人名、地名或机构名等。实体匹配精度的不足可能导致数据关联错误、信息冗余或遗漏，从而严重影响后续的数据分析与业务决策^[34]。具体而言，实体匹配精度不足的原因多样，包括算法设计的局限性、数据质量的不一致性、文本的复杂性及多样性等。此外，不同的实体匹配任务可能面临各自的挑战，例如，跨语言或跨领域的实体匹配由于语言差异和领域知识的不同，其匹配难度会进一步加大。

(3) 知识表示和推理方面也存在不足。当前的众多系统在处理知识时, 普遍呈现出浅层次的特点, 主要依赖于简单的关键词匹配机制或预设的模板化方法来试图解析与回答问题。然而, 这种方法忽略了知识体系的复杂性与多样性本质, 缺乏对知识深层结构与内在逻辑的深刻理解与推理能力。尤为突出的是, 在面对涉及复杂逻辑推理或需跨领域知识融合的问题时, 这些系统往往难以洞悉问题背后的深层意涵, 无法高效地整合与运用跨越不同学科领域的知识以产出令人满意的解答。此种局限性不仅限制了知识表示与推理技术在实际应用场景中的效能发挥, 还成为制约人工智能领域在这一关键方向上实现进一步飞跃与突破的主要障碍。

综上所述, 传统的智能问答系统在句子级处理准确性、实体匹配精度以及知识表示与推理等方面存在若干问题。为克服这些问题, 需要持续探索新的技术和方法以改进和优化智能问答系统, 从而提供更加准确、高效及个性化的服务。

1.4 本文主要内容

1.4.1 主要研究内容

本文主要对智能问答算法的进行研究, 旨在实现实时、精准的健康咨询解答。本文的主要研究内容如下:

(1) 构建一种结合 BERT、LSTM 和 CRF 的模型框架。该框架巧妙地融合这三种技术的核心优势, 不仅可以充分发挥 BERT 预训练模型在深度语义理解方面的卓越能力, 还通过 BERT 的强大语义表示功能, 使模型能够深入剖析句子中的语义内涵。在该框架中, Bert 模型被用作基础, 负责捕捉文本中的丰富语义信息, 并将其转化为高维度的向量表示。随后, 这些经 BERT 精细编码的向量被传递给 LSTM 网络, 利用其特有的记忆机制来捕捉文本中的时序依赖性和上下文关系。通过 LSTM 的处理, 模型能够进一步理解和分析文本中的深层结构。尤为关键的是, 该框架在 BERT 基础上引入了属性相似度计算。通过 BERT 计算问句与特定属性之间的相似度, 模型能够精确地进行属性选择, 确保在问答系统中准确匹配用户的问题与相关属性信息。这一创新方法不仅显著提升了属性选择的准确性, 还大幅增强了整个问答系统的性能与效率。

(2) 结合数据增强策略和 TF-IDF 实体对齐技术改进实体识别。为了提升 BERT 模型在实体识别任务中的性能,设计三种数据增强策略,即实体替换、实体掩码以及实体拼接;同时,为了确保模型的输出与现有知识图谱中的实体名一致,采用 TF-IDF (Term Frequency-Inverse Document Frequency, 词频-逆文档频率) 算法来进行实体对齐。除此之外,为了应对实体匹配精度不足的挑战,提出使用相似度匹配的优化方法。该方法通过深入计算输入语句中的词组与实体特征词之间的字重叠率、词向量余弦相似度以及 DP 编辑距离,得出一个综合的相似度评分。这个评分是上述三个指标的平均值,能够全面反映词组与实体特征词之间的相似程度。在此基础上,选择相似度最高且超过预设阈值的实体特征词作为最终的匹配结果。

(3) 运用 34B 大型语言模型进行意图识别。在对当前主流的意图识别方案进行深度剖析后,细致地对比规则匹配、训练模型以及提示工程三种方法。这一对比不仅涉及识别的准确性,还考量多意图辨识能力、所需的人力成本、推理效率以及资源占用等多个关键维度。经过综合评估,发现提示工程在这些方面展现出了显著的优势,特别是在确保高准确性的同时,还能灵活应对包含多个意图的复杂查询。因此,决定采用提示工程作为进行意图识别的首选策略,以期为用户提供更为精准、高效的服务。

(4) 设计一个基于 RAG 与大模型技术的医疗问答系统。借助爬虫技术和 API 接口,能够高效地从多元化的数据源中捕获医学领域的数据,并对其进行精细的结构化处理。随后,将这些数据存储于 Neo4j 图数据库中,并构建出一个全面且详尽的医学领域知识图谱。在此基础上,结合 BERT 的命名实体识别和 34b 大模型的意图识别,设计并实现一个针对该领域的智能问答系统,为用户提供便捷的查询服务。当用户发起查询时,该系统会利用问答模型精准地识别出查询中的关键实体。紧接着,结合这些实体,系统运用 Cypher 查询语言在图数据库中迅速检索相关的医学信息,确保用户能够在短时间内获得所需的专业知识。

1.4.2 论文组织架构

具体地,本文内容分为以六个部分:

第 1 章绪论,介绍研究背景、意义、国内外研究现状以及本研究的内容和结构。

第 2 章智能问答算法的研究基础，涵盖 RAG、爬虫技术、知识图谱的基础理论结构、信息提取和存储技术，以及词向量和 Transformer 神经网络。

第 3 章基于深度学习的问答模型，首先介绍命名实体识别技术（NER），然后是基于 BERT 的命名实体识别和 NER 模型的训练和改进，以及对 BERT 算法的优化，最后对实验结果进行分析和总结。

第 4 章基于 BERT 预训练模型的中文文本分类，主要包括数据预处理、加载预训练模型、文本分类模型训练等。

第 5 章基于 RAG 和大模型的医学领域智能问答，详述医学领域的知识图谱的建立过程、知识获取、处理、抽取、存储，结合 BERT 的命名实体识别和 34b 大模型的意图识别，构建该领域的智能问答系统，以及实验结果的分析展示。

第 6 章总结与展望，概括研究工作，指出不足之处和未来的发展方向。

2 智能问答算法的研究基础

本章节主要介绍智能问答算法的研究基础。包括但不限于 RAG 技术、用于获取数据的爬虫技术、用于数据存储和处理的知识图谱、用于 NLP 的词向量技术、用于构建模型的 Transformer 神经网络等。

2.1 RAG

RAG (Retrieval-Augmented Generation)，即检索增强生成技术，将外部知识库中的信息与文本生成相融合，从而大幅提升了文本生成的质量^[35]。这一技术可谓是人工智能领域的一大创新，为自然语言处理注入了新的活力。

RAG 技术的核心在于其独特的技术路线，该路线将信息检索与自然语言生成技术紧密结合，形成了一种强大的文本生成能力^[36]。首先，它运用先进的信息检索技术，从浩如烟海的知识库中精准捕捉与用户查询最为契合的信息。随后，这些信息被巧妙地作为上下文呈现给生成模型，为其创作提供灵感与指导，从而催生出更为精确、深入的回答^[37]。RAG 架构如图 2-1 所示。

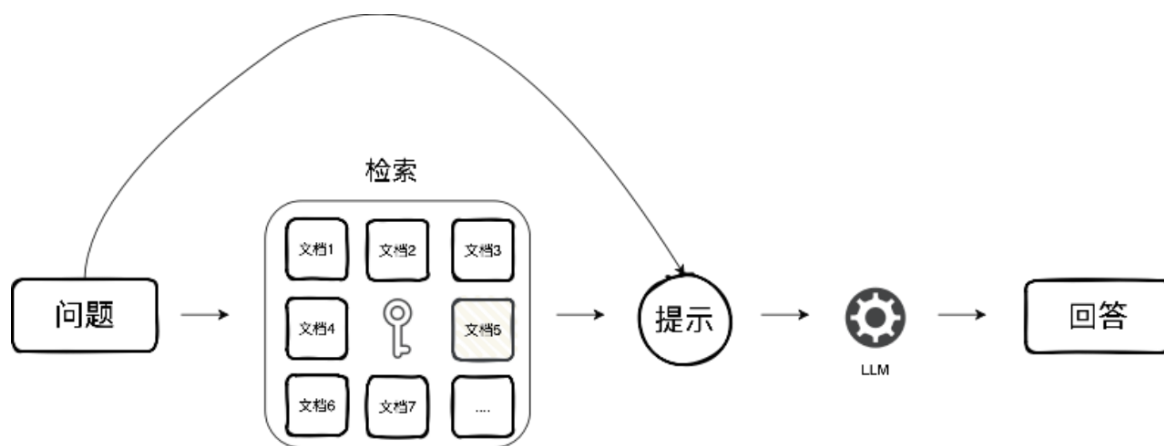


图 2-1 RAG 架构

RAG 首先进行文件预处理，这一环节涉及将多样化的专业知识文件，如 PDF、Word、TXT 等格式，转化为大语言模型能够轻松理解的纯文本数据^[37]。为确保信息的准确无误，专门的文档加载器或多模态模型如 OCR 技术被用于处理这些丰富的知识源^[39]。

接下来是文档切片环节，即将庞大的文档分解成若干个文本块。这一步骤不仅满足了 embedding 模型对输入 token 的限制，同时有效降低了嵌入内容中的噪声干扰，使得与用户查询最相关的文档部分能够更为高效地被检索到^[40]。

紧接着，这些切割后的文本块将被向量化，也就是转换成便于相似度计算和检索的数值向量^[41]。这些向量被妥善存储在特定的向量知识库中，以备快速检索之需。当用户抛出问题时，系统会迅速根据问题构建查询，并从向量知识库中寻觅最匹配的向量，即相关的文本块。检索算法在此刻发挥关键作用，通过精准评估查询与知识库中向量的相似度，迅速锁定最契合的信息^[42]。

最终，这些搜寻到的相关信息将与用户问题一同呈现给生成模型，如大型语言模型。在丰富的上下文信息助力下，生成模型得以孕育出更为准确、深刻的回答。

RAG 技术的优势不言而喻。首先，它凭借检索外部知识库的能力，极大地丰富了生成文本的信息量和准确性，使得回答更加全面、权威^[43]。其次，RAG 技术能够动态地从持续更新的知识库中汲取信息，确保生成的内容始终与时俱进，充满新鲜活力。最后，通过检索和整合长文本信息，RAG 技术显著增强了模型对长上下文的理解和生成能力，有效打破了输入长度的束缚^[44]。

综上所述，RAG 技术以其独特的整合外部知识库信息的能力，显著提升了文本生成的质量。在问答系统、内容创作、智能助手等多个领域，它都展现出了广阔的应用前景，成为推动人工智能发展的重要力量。

2.2 网络爬虫技术

为构建知识图谱，此次选择了可靠的医学数据源，即寻医问药网（<http://www.xywy.com/>）。通过利用爬虫技术，成功地从该网站上抓取了大量医学数据，这些数据被视为构建知识图谱的基础。在处理这些数据时，运用了 XPATH 和正则表达式等文本处理技术用于定位和提取实体、属性和关系信息，并清理数据，包括去除 HTML 标签、特殊字符等^[45]。XPATH 被用于分析网页结构，确定实体信息的位置，确保准确提取目标数据。而正则表达式则用于清理数据，去除不必要的标签和特殊字符，以确保最终数据以 JSON 格式存储在本地，易于理解和处理。据网页排版规律逐一提取

实体、属性和关系信息^[46]。

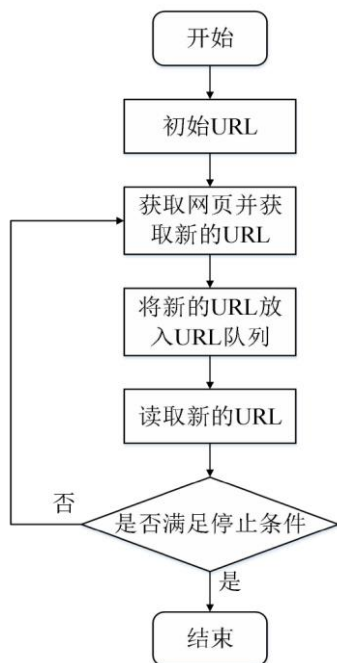


图 2-2 爬虫工作原理

在将原始数据转化为结构化数据之前，进行了一系列清理步骤，包括去除文本中的特殊字符，如“\n”、“\t”，以确保最终数据以 JSON 格式存储在本地，并能够轻松理解和处理^[47]。整个过程的目标是实现从非结构化数据到结构化数据的有效转换。通过这种方式，通用网络爬虫能够在互联网上自动、高效地收集大量数据，其工作原理如图 2-2 所示。

2.3 知识图谱

知识图谱和机器学习是两个在人工智能领域中起着重要作用的概念，它们之间存在着密切的关系。知识图谱是一种结构化的知识表示方法，它将实体、概念和它们之间的关系以图的形式呈现出来^[48]。而机器学习则是一种通过数据训练模型，使计算机能够从数据中学习并做出预测或决策的方法。

知识图谱为机器学习提供了丰富的背景知识。机器学习模型可以从知识图谱中学习实体之间的关系和属性，从而提高其在特定任务上的表现。通过将知识图谱中的信息结合到机器学习模型中，可以帮助模型更好地理解数据和做出准确的推理^[49]。

知识图谱可以用于机器学习模型的知识蒸馏。通过将知识图谱中的丰富信息转化为机器学习模型可以理解的形式，可以有效地将知识传递给模型，从而提高模型的泛化能力和性能。

知识图谱还可以用于辅助机器学习模型的数据集成和清洗。在构建机器学习模型时，数据的质量和一致性对模型的性能至关重要^[50]。知识图谱可以帮助识别和解决数据集成过程中的冲突和不一致性，从而改善数据质量并提高模型的准确性。

总结来说，知识图谱和机器学习相辅相成，相互促进。知识图谱提供了丰富的背景知识和数据，为机器学习模型提供了学习和推理的基础，而机器学习模型则可以通过学习知识图谱中的信息来提高其性能和泛化能力^[51]。使用知识图谱可以帮助机器学习模型更好地理解 and 利用数据，从而实现更加准确和智能的决策和预测。

它通常表现为一个巨大的网络，其中节点代表实体（如人物、地点、组织等），边代表实体间的各种关系和属性^[52]。这种图形结构使得信息互联互通，易于检索和分析。其核心是将现实世界的事物及其相互关系转化为机器可理解的格式。它广泛应用于搜索引擎优化、推荐系统、自然语言处理等领域，帮助改进信息检索的准确性和相关性。通过知识图谱，计算机不仅能处理数据，还能理解数据背后的语义联系，从而实现更智能的数据分析和决策支持^[53]。

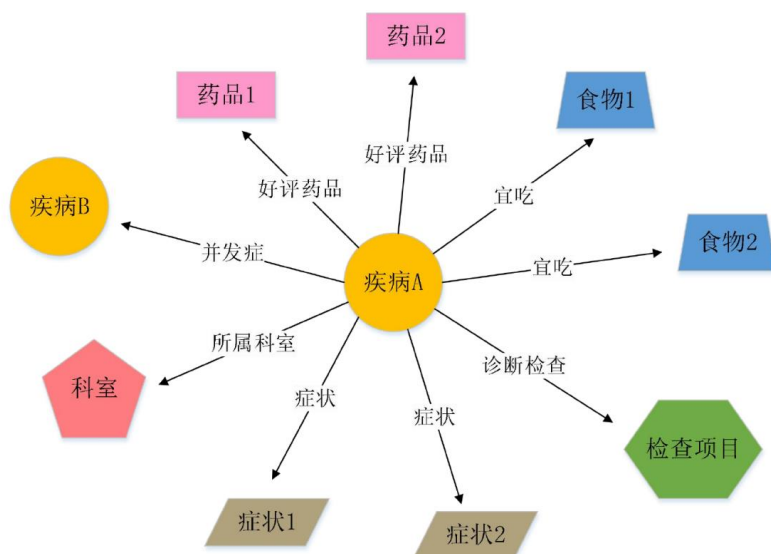


图 2-3 图谱示例

如图 2-3 所示，可以看到的是一个针对医学健康领域的知识图谱示例。这个图谱

采用图形的方式来展示信息，使得复杂的数据关系变得更加直观和易于理解。在这个图谱中，各种元素通过节点和有向边的形式来表示。节点代表不同的实体，例如疾病、药品等，而有向边则表示节点间的特定关系^[54]。例如，疾病 A 和药品 1 就是通过一个标记为“好评药品”的有向边相连的，这表明了两者之间存在的正面关联。在这种可视化表示中，每个实体和它们之间的关系都被清晰地标识出来，从而为用户提供了一个关于疾病 A 及其相关信息的全面视图。

2.3.1 知识图谱的架构

知识图谱的架构通常分为三个主要部分：数据层、存储层和应用层。

数据层是知识图谱的基础，涉及数据的收集、整理和预处理^[55]。数据可以来源于各种渠道，如公开的数据库、文本文档、互联网等。在这一层，重要的步骤包括数据清洗、实体识别、关系抽取和数据融合，以确保数据的准确性和一致性。存储层，这一层负责高效地存储和管理处理过的数据。知识图谱通常需要一种特殊的数据库来存储其复杂的关系结构，如图数据库^[56]。这些数据库优化了对图形结构数据的查询和处理，能够快速响应复杂的查询请求。应用层是知识图谱的最终输出，包括各种基于知识图谱的应用和服务。在这一层，知识图谱被用于支持搜索优化、推荐系统、智能问答、数据分析等多种应用^[57]。通过语义搜索、关联分析等技术，知识图谱在这一层将复杂的数据转化为有价值的洞察和知识。

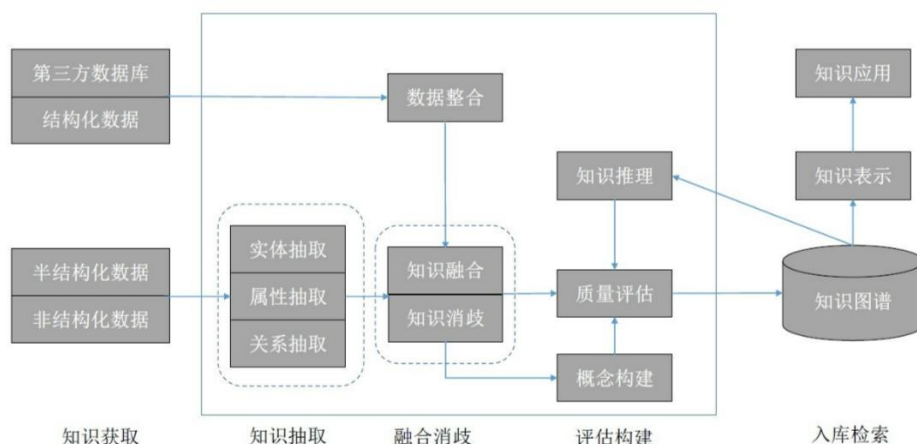


图 2-4 知识图谱的建立流程

在图 2-4 中，可以看到知识图谱的建立涉及到多种类型的原始数据：结构化数

据、半结构化数据和非结构化数据^[58]。结构化数据如关系型数据库，是高度组织并格式化的；半结构化数据则拥有一定的结构，例如 HTML、XML 和 JSON 文件；非结构化数据如视频和图片，则没有固定的格式。

为了构建知识图谱，不同类型的数据需要通过特定的方法进行处理。这包括从这些原始数据和外部知识库中提取出有结构的三元组(实体-关系-实体)形式的知识^[59]。在此基础上，通过指代消解和其他技术对这些知识进行融合，并进行质量评估，以确保知识图谱的准确性和可靠性。

最终构建的知识图谱能够用于多种应用，如智能问答系统，它可以利用图谱中的丰富信息来回答用户的查询^[60]。这一过程显示了从多样化的数据源到实用应用的转化，展示了知识图谱在信息组织和应用上的重要价值。

2.3.2 知识图谱的信息抽取

知识图谱的信息抽取是从各种数据源中提取有用信息并转化为知识图谱可理解和存储的格式的过程。这是构建知识图谱的关键步骤之一，涉及从结构化、半结构化和非结构化数据中提取实体、关系和属性。

实体抽取、关系抽取和属性抽取是构建知识图谱的关键环节，它们涉及不同的技术和方法^[61]。实体抽取是从文本中识别具有特定意义的实体，如人名、地点、机构等。实体抽取的方法有基于字典或规则的方法、基于统计学习的方法和基于神经网络的方法等。

(1) 基于字典或规则的方法：这种方法使用预定义的词典和语法规则来识别文本中的实体。词典包含了一系列已知实体的列表，而规则则基于文本模式或关键词来匹配实体^[62]。这种方法的优势在于准确性高，特别是在专业领域内，但它的缺点是灵活性差，难以适应新实体或变化的语言用法。

(2) 基于统计学习的方法：统计学习方法，如隐马尔可夫模型(HMM)、条件随机场(CRF)等，通过学习大量标注数据来识别实体^[63]。这些方法能够考虑到文本中的上下文信息，比纯粹的规则方法更加灵活。

(3) 基于神经网络的方法：深度学习技术，特别是基于神经网络的方法，如循环神经网络(RNN)和其变种 LSTM、BERT 等，已成为实体识别的主流技术^[64]。这

些方法通过学习大量的文本数据，能够捕获复杂的语言模式，提高实体识别的准确性和鲁棒性。

而关系抽取旨在识别文本中实体间的语义关系。传统方法依赖于规则和模式匹配，但现代方法，如基于神经网络的模型，能自动学习和识别文本中的关系模式。属性抽取则专注于从文本中提取实体的特定属性，如人的年龄、公司的成立日期等^[65]。这通常涉及识别实体周围的描述性短语和上下文。在实体、关系和属性抽取中，深度学习方法特别是预训练的语言模型如 BERT，因其强大的语言理解能力，已成为当前研究和应用的热点。这些方法能够有效地处理自然语言的复杂性和多样性，提高抽取任务的性能^[66]。

2.3.3 知识图谱的储存

在本文中，对于知识图谱的存储方式，采用了 Neo4j 这一开源图形数据库作为存储医学知识图谱的工具。Neo4j 以其卓越的性能和灵活性而闻名，提供了强大的 Cypher 语言，使得能够在这次设计中知识图谱中进行复杂的关系查询，从而更好地理解和分析医学领域的复杂关联^[67]。Neo4j 是一种基于图数据库的管理系统，其主要功能包括图形数据模型、Cypher 查询语言、高性能和可扩展性、事务支持以及灵活的数据模型。

首先，Neo4j 采用图形数据模型来表示和存储数据，其中节点表示实体，关系表示实体之间的连接，属性为节点和关系附加的数据信息。这种数据模型使得 Neo4j 能够有效地存储和查询复杂的关系数据。Neo4j 使用 Cypher 查询语言来进行数据查询和操作，用户可以方便地编写和执行各种复杂的图形查询^[68]。

此外，Neo4j 具有出色的性能和可扩展性，能够处理大规模的图形数据，并且支持水平扩展，可以通过集群方式实现横向扩展。Neo4j 还提供了强大的事务支持，能够确保数据的一致性和完整性，用户可以通过事务来执行一系列的数据操作，并在需要进行回滚或提交。

最后，Neo4j 的数据模型非常灵活，能够轻松地适应不同类型的数据结构和应用场景，用户可以根据实际需求定义节点和关系的结构，并为它们添加任意数量的属性，以满足各种复杂的数据建模需求。综上所述，Neo4j 作为一种功能强大的图数据

库管理系统，具有丰富的功能和灵活的数据模型，适用于各种复杂的关系数据存储和查询场景。

此次 Neo4j 是根据医学知识图谱中的实体构建节点，每个实体都被视为一个节点，这些节点在图数据库中形成了一个具有层次结构和关联性的网络。通过 Neo4j 的灵活性，能够为每个节点定义属性信息，这些属性信息对于更全面地描述实体的特征和属性至关重要。这为医学知识图谱提供了精确而丰富的数据基础。知识图谱将实体之间的关系信息映射为图数据库中的边，而 Neo4j 能轻松创建这些边，连接不同的实体，形成图谱中的关系网络。这种关系的建模使得能够捕捉到医学领域中各种实体之间错综复杂的联系，为进一步的分析和挖掘奠定了基础。

通过这一存储方式，成功地构建了医学知识图谱，并将其存储于 Neo4j 图形数据库中。这个知识图谱不仅包含了丰富的实体信息，还展现了实体之间错综复杂的关系。例如，图中可能包括医学领域的疾病、药物、症状等实体，它们之间的关联通过图数据库中的节点和边清晰地呈现出来。

在图 2-5 中，展示了关于肺气肿部分知识在图数据库中的存储。这一展示呈现了肺气肿实体及其与其他相关实体之间的关系。这不仅有助于医学从业者深入了解阿洪病的特性，还为研究人员提供了一个可视化且结构清晰的数据平台，以支持更深入的医学研究。

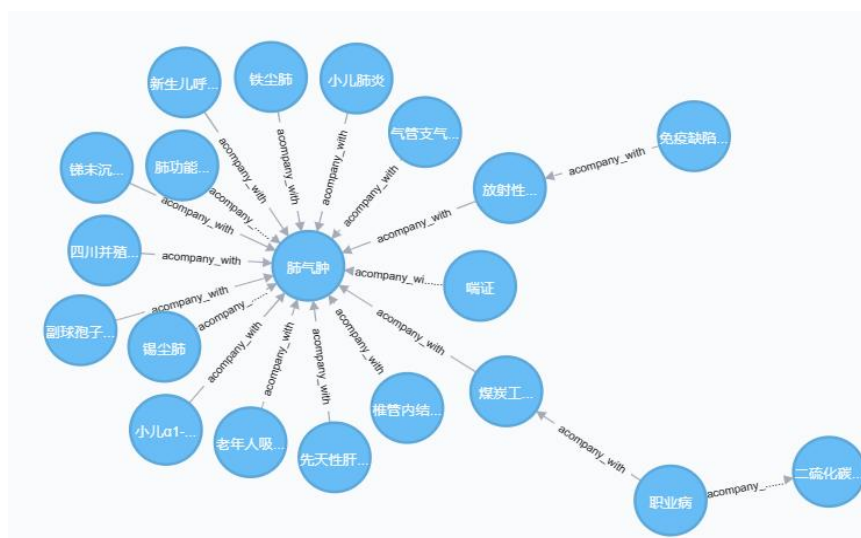


图 2-5 肺气肿部分知识在图数据库中的存储

2.4 词向量技术

词向量表示是一种将词汇表达为数值向量的技术，它是自然语言处理（NLP）中的一个基本概念。在这种表示法中，每个单词或短语被映射到一个多维空间中的一个点，这个点由一组数字（向量）表示。这些向量捕捉了单词的多种语义属性和语境关系。词向量的维度可以从几十到几千不等。维度越高，向量能够捕捉的语义信息越丰富。在词向量空间中，语义相似或功能相似的单词通常在空间中彼此接近。这种现象基于“分布假设”，即在相似上下文中出现的词汇具有相似的意义。生成词向量的方法有 One-hot Encoding、基于计数的方法和预测模型等。

(1) One-hot Encoding: 最简单的词表示方法，每个词被表示为一个很长的向量，其中只有一个元素是 1，其余都是 0。这种表示法简单，但不包含任何关于单词之间关系的信息。

(2) 基于计数的方法：如 TF-IDF 或共现矩阵。这些方法统计单词在特定上下文中出现的频率，从而生成每个词的向量。这种方法能够捕捉到一些语义信息，但向量维度通常很高，且不能很好地处理多义词。

(3) 预测模型：如 Word2Vec、GloVe。这些模型使用神经网络来预测一个单词的上下文（或相反），在这个过程中学习到的权重被用作词向量。这些向量能够有效地捕捉词与词之间复杂的语义和语法关系。与传统的稀疏表示相比，词向量是高效的密集表示，减少了所需的存储空间，能够捕获词汇之间的语义关系，如同义词和反义词的关系。

Word2Vec 是生成此类词向量的相关模型，它具有两种设计思想，Cbow (通过外围词分析预测中间词) 和 Skip-Gram (通过中间词分析预测外围词)，其特征是相对较紧的词位置。在此系统中，使用跳跃语法模型分析单词向量。

Skip-Gram 模型由两部分组成：建立适当的神经网络模型，并将模型的参数设置为词向量。理解它的方法是，单词向量的本质首先是一个模型的训练过程的参数，然后使用这些参数来构造另一个模型的参数，这些参数是神经网络隐藏层的权重矩阵。

由图 2-6 可知，Skip-Gram 的设计架构的输入为一个 $W(t)$ ，经过一个类似于黑

盒操作的 **projection**，得到四个输出，即 $W(t-2)$ 、 $W(t-1)$ 、 $W(t+1)$ 、 $W(t+2)$ 。要训练向量，需要提前准备一个包含训练文本中包含的不同单词的词汇。目前假设集合表的维数是 10000 的，一个词的构造是一个热点，即离散量体满足并输入为第二章，神经网络也输出实用的一维 10000，作为向量，这个向量表示表格中所有单词的含义以及它们周围的概率。如果您不想获得如此大的特征矩阵，则可以使用隐藏层。

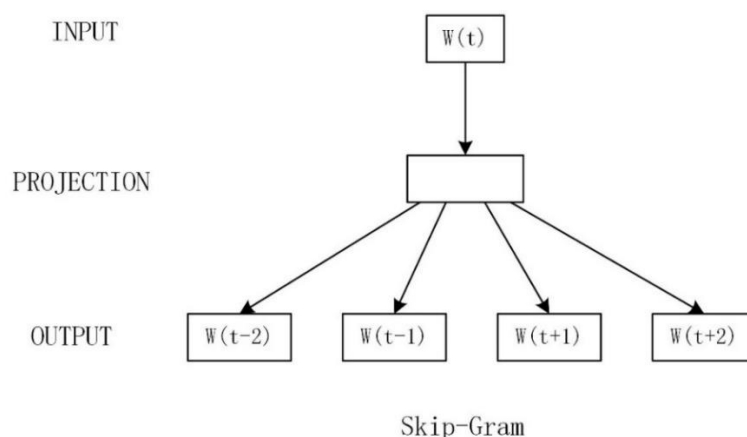


图 2-6 Skip-Gram 的设计架构

在模型设计中，为简化矩阵表示，可在输入与输出间添加隐藏层来承载词向量。通常，隐藏层大小设为 100 至 300 的整数，以优化计算效率和表达能力。Skip-gram 隐层如图 2-7 所示。

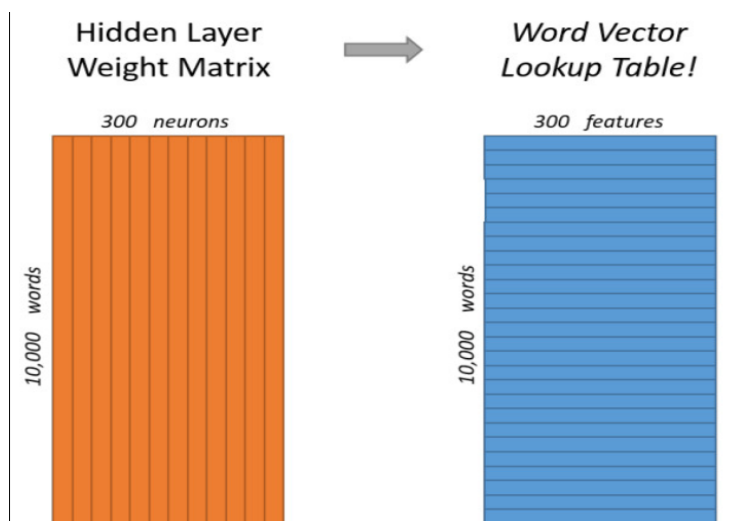


图 2-7 Skip-gram 隐层

与 Google 宣布的最新“google 新闻”数据集相对应的词向量是 300 维的，在示

例中是 200 维的。通过隐藏层获取 300 维词向量，然后通过 softmax 层输出每个词出现的概率。隐藏层的含义可以纵向和横向理解。从左侧的垂直方向来看，每列都是神经元的隐藏层，连接到 10000 维词向量。从右边看，每行是一个词的词向量。

2.5 Transformer 神经网络

"Transformer"是一种在自然语言处理（NLP）领域广泛使用的深度学习模型。它最初由 Vaswani 等人在 2017 年的论文《Attention Is All You Need》中提出，引发了 NLP 领域的一场革命^[69]。与之前的模型相比，Transformer 的主要创新在于完全依赖于"注意力机制"（Attention Mechanism），摒弃了传统的循环神经网络（RNN）和卷积神经网络（CNN）结构。Transformer 的核心思想是将输入数据（如文本）分解成更小的部分，然后并行处理这些部分^[70]。这种方法相比传统的 RNN，在处理长距离依赖关系时更加高效，因为它能够在序列的任意两点之间建立直接的联系。Transformer 的架构如图 2-8 所示。

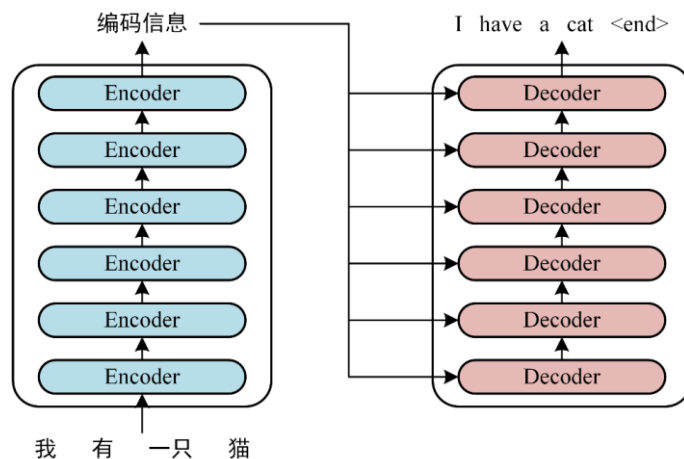


图 2-8 Transformer 架构图

Transformer 模型的主要组成部分包括编码器（Encoder）和解码器（Decoder）。编码器由多个相同的层组成，每层包括两个主要子层：多头注意力（Multi-Head Attention）机制和简单的前馈神经网络^[71]。多头注意力机制是 Transformer 的核心，它能够同时关注输入序列的不同位置，从而捕捉更丰富的上下文信息。解码器的结构与编码器类似，但在每个解码器层中增加了一个额外的多头注意力子层，用于关注编

码器的输出。

2.6 本章小结

这一章节首先介绍了 RAG，然后对网络爬虫技术做了简要的说明，接着深入讨论了知识图谱，包括其定义、结构以及如何从数据中抽取信息以及如何存储这些信息。此外，本章还详细介绍了图数据库的相关概念。紧接着，文章对词向量的两种表现形式以及 Tranformer 技术进行了详尽的解释。

3 基于深度学习的智能问答模型

本章节主要介绍基于深度学习的智能问答模型。首先介绍命名实体识别技术（NER），然后是基于 BERT 的命名实体识别和 NER 模型的训练和改进，以及对 BERT 算法的优化，最后对实验结果进行分析和总结。

3.1 命名实体识别技术

命名实体识别（NER）是自然语言处理（NLP）的一个关键任务，旨在从文本中识别出具有特定意义的实体，如人名、地点、组织名等。这项技术使用算法来找出文本中的实体，并将其分类为预定义的类别^[72]。NER 在信息提取、问答系统、机器翻译等领域中发挥着重要作用。

传统的 NER 方法依赖于复杂的规则和词典，而现代方法则更多地使用基于机器学习的技术，尤其是深度学习模型，如循环神经网络（RNN）和变压器（Transformer）架构。这些高级模型能够更有效地处理语言的复杂性和多样性，提高实体识别的准确性和效率。此次设计采用 BERT（Bidirectional Encoder Representations from Transformers）、LSTM（Long Short-Term Memory）、和 CRF（Conditional Random Field）结合的方式，旨在提高对序列结构的建模和标注准确性。其中 BERT 的主体结构如图 3-1 所示。

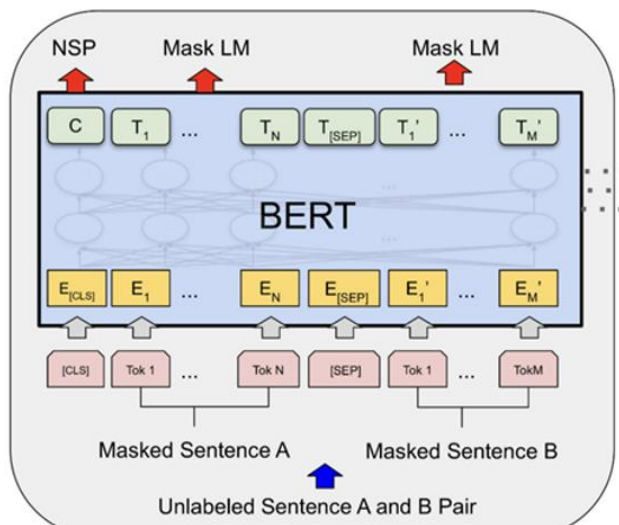


图 3-1 BERT 的主体结构

首先,在这个综合模型中,BERT 作为预训练模型,负责处理输入文本并捕捉其中的上下文信息。通过利用 BERT 的双向编码机制,模型能够深入理解每个词汇的语境,为后续的序列标注任务提供了更为丰富的语义表示。这是关键的第一步,因为实体识别任务往往需要对文本的整体语境进行综合考虑。

其次,采用 LSTM 网络专门设计用于捕捉序列中长距离依赖关系。在 BERT 的基础上,LSTM 通过双向结构继续建模序列,进一步强化了对上下文的全面理解。这对实体识别尤为关键,因为实体通常在句子中具有复杂的上下文关系^[73]。LSTM 的加入有助于模型更好地捕捉实体之间的语义关联,提高了模型对序列结构的建模能力。

然后,为了更好地解决序列标注问题,引入了 CRF 层。CRF 是一种条件随机场,专门设计用于建模标签之间的依赖关系。通过 CRF 层,模型可以更全面地考虑相邻标签之间的关联性,从而提高整体标注的一致性。这是一个关键的步骤,因为实体识别任务要求预测的标签序列在语法和语义上都是合理的。CRF 过程如图 3-2 所示。

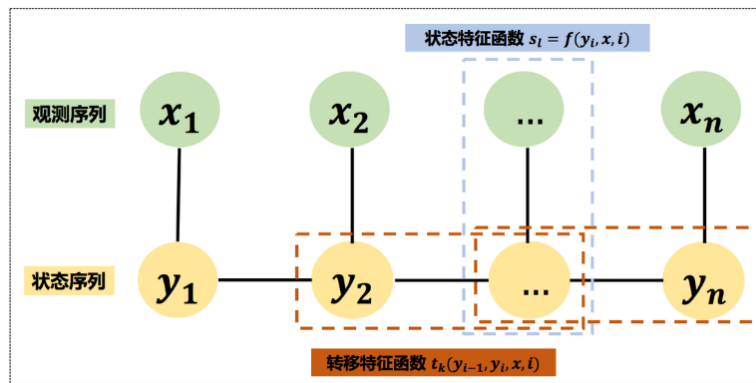


图 3-2 CRF 过程

采用 BERT+LSTM+CRF 的综合方式的原因在于,这三者相互补充,形成了一个强大的框架。BERT 提供了全局的上下文理解,LSTM 通过建模序列结构捕捉了长距离依赖,而 CRF 通过考虑标签依赖关系提高了整体的标注准确性。这个结合方式在实践中取得了显著的性能提升。

综上所述,采用这种方式进行实体识别,不仅能够适应不同类型实体的复杂语境,还提高了模型对实体关系的捕捉能力,增强了泛化性能和鲁棒性。这个综合模型在处理现实世界中多样性的语境下表现出色,为实体识别任务的成功执行提供了有

力支持。

3.1.1 命名实体识别过程

命名实体识别的目的是识别出问题中提及的特定命名实体。如图所示的过程首先涉及使用一个别名词典来确定问题中所有可能的命名实体，作为初始候选。在图 3-3 展示的候选名单中，每个候选项的浅色区域标出了候选实体的名称及其在问题文本中的位置。接下来，系统利用一个模型为每一个候选的命名实体计算一个得分（NER_SCORE）^[74]。最后，系统会选出得分最高的实体，认定其为问题中正确的命名实体。

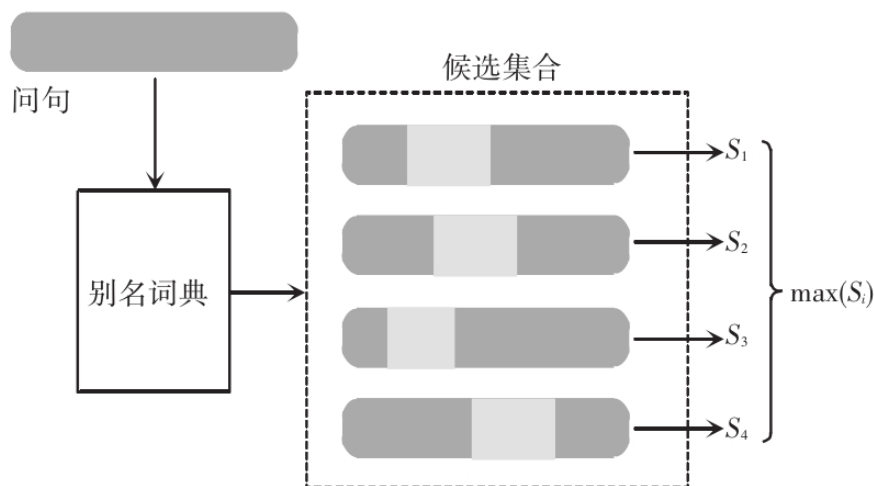


图 3-3 命名识别

可以通过一个简单的例子来说明，假设一句话“Elon Musk 在 2021 年将 SpaceX 的 Starship 发送到了火星。”首先，对这句话进行预处理，包括分词、去除停用词等，以便于处理。接下来，NER 系统会扫描每个词或短语，试图识别命名实体。在这个例子中，它会识别“Elon Musk”为人名（Person），“2021 年”为时间（Date），“SpaceX”为组织（Organization），“Starship”作为产品名（Product），“火星”为地点（Location）^[75]。每个识别出的实体会被分类到相应的类别。如“Elon Musk”分类为人名，“火星”分类为地点。

最终，系统会输出这句话中所有已识别和分类的实体：Elon Musk (Person), 2021 年 (Date), SpaceX (Organization), Starship (Product), 火星 (Location)。通过这些步骤，

NER 系统能够从文本中提取有意义的信息，为进一步的数据分析和处理奠定基础。

3.1.2 属性映射

在属性映射中，对于分类任务，首先进行的是 BERT 模型的预训练。BERT 是一种基于 Transformer 架构的深度双向编码器，通过遮蔽语言模型（Masked Language Model）和下一句预测（Next Sentence Prediction）等任务，在大规模语料上学习了文本中词语之间的语义关系和语境信息^[76]。这样预训练的 BERT 模型能够生成高质量的词向量表示，对文本的语义和语境有着丰富的理解。

在进行具体的分类任务时，首先将待分类的文本序列输入到预训练的 BERT 模型中。BERT 模型会将文本序列中的每个词转换为上下文相关的词向量表示，并将这些向量作为输入，经过一系列的 Transformer 层处理，最终得到整个文本序列的表示。这个表示包含了文本序列的丰富语义和语境信息，是基于上下文的语义表示。

然后，这个文本序列的表示会被传递给分类器模型，例如全连接神经网络。分类器模型会根据 BERT 模型生成的文本表示以及标注数据之间的关系进行学习，以实现文本的分类。在训练过程中，分类器模型通过调整其参数，不断优化模型，使其能够更好地对文本进行分类。在实际应用中，对于不同的分类任务，可以使用不同的分类器模型，如逻辑回归、支持向量机等。这些分类器模型会根据具体任务的特点和数据情况进行选择和调整，以达到最佳的分类效果^[77]。属性映射中的分类任务利用预训练的 BERT 模型将文本序列转换为上下文相关的词向量表示，然后通过分类器模型对文本进行具体分类。这种方法能够充分利用 BERT 模型学习到的丰富语义和语境信息，从而提高分类任务的准确性和效果。

属性映射可以用于将用户查询中的术语和系统数据库中的相关医学信息相对应。这对于提供准确、相关的答案至关重要。以下是一个具体例子：假设一个用户向医药问答系统提出了这样一个问题：“我最近被诊断出有高血压，可以吃阿司匹林吗？”

在这个问题中，关键的医学术语是“高血压”和“阿司匹林”。医药问答需要对这些术语进行属性映射，以便在其数据库中找到相关信息。这个过程可能如下：

系统首先识别出“高血压”是一种疾病，而“阿司匹林”是一种药物。接下来，系统在其数据库中查找与“高血压”相关的治疗建议和“阿司匹林”的药物信息。这可能涉及

到将用户问题中的术语映射到数据库中的特定字段，例如，“高血压”映射到疾病数据库中的“Hypertension”条目。“阿司匹林”映射到药物数据库中的“Aspirin”条目。最后根据从数据库中检索到的信息，分析阿司匹林是否适用于高血压患者。系统可能需要考虑阿司匹林的作用机制、副作用、与高血压治疗药物的相互作用等信息。

3.1.3 LSTM

本次采用的长短时记忆网络（LSTM）是一种专门为处理序列数据设计的神经网络架构，尤其擅长处理自然语言处理（NLP）中的长期依赖问题。在本研究中，LSTM 被用于编码问题和属性文本，从而捕获它们的深层语义信息，并通过计算这些编码的余弦相似度来评估问题与属性之间的语义关联度^[78]。LSTM 结构如图 3-4 所示。

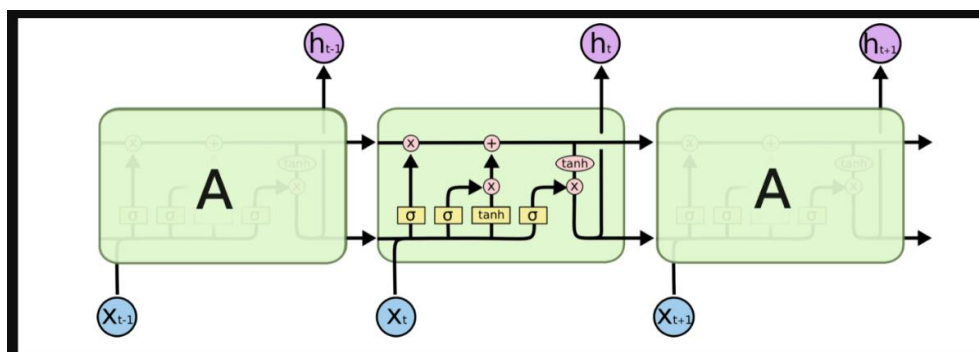


图 3-4 LSTM 循环结构

LSTM 的核心优势在于其能够记忆长期信息，这是通过其独特的门控机制实现的，包括遗忘门、输入门和输出门。遗忘门决定了保留多少旧信息，输入门控制了加入多少新信息，而输出门则决定了基于当前单元状态的输出量^[79]。这使得 LSTM 在处理诸如句子或段落这类具有长期依赖关系的数据时，能够保持较高的效率和准确性。

在本文的应用场景中，LSTM 用于将问句和属性文本转换为高维空间中的向量。这个过程始于文本的预处理，包括分词和词嵌入，将自然语言转换为机器可理解的格式。词嵌入，如 Word2Vec 或 GloVe，为每个词提供了一个密集的向量表示，捕获其语义特征。

这些词向量序列被馈送到 LSTM 网络。在序列的每个时间步，LSTM 更新其内部状态，考虑到新输入和先前的信息。最终，对于问题和属性文本，LSTM 输出它们

各自的向量表示^[80]。这些向量概括了文本的整体语义内容，是通过分析整个句子而非仅仅基于单个词来生成的。

最后研究使用余弦相似度来衡量这两个向量之间的相似性。余弦相似度是一种常用的相似度度量方法，它计算两个向量在空间中的夹角，以此作为它们相似性的指标。它的值范围从-1（完全不同）到 1（完全相同），0 表示无相关性。在本研究中，较高的余弦相似度表明问题和属性在语义上更为接近^[81]。

通过这种方式，LSTM 模型能够有效地捕捉问题和属性文本的深层语义关系。这对于自然语言理解系统至关重要，特别是在需要理解和处理复杂、多变的人类语言的情况下。这种方法能够显著提高系统的理解能力和回答的相关性，从而提供更准确、更人性化的用户体验。

3.1.4 基于单词语义相似度的注意力机制

基于单词语义相似度的注意力机制是一种在自然语言处理（NLP）中广泛使用的技术，尤其是在任务如机器翻译、文本摘要、问答系统等领域^[82]。这种注意力机制能够让模型更加聚焦于文本中的关键信息，提高处理复杂语言数据的能力。注意力机制的核心目标是使模型能够自动地聚焦于输入数据的关键部分，类似于人类在阅读或听觉处理时的注意力分配^[83]。

在基于单词语义相似度的注意力机制中，模型学习每个词与其他词之间的相对重要性，这通常是通过计算词之间的相似度或相关性来实现的。本文采用计算注意力矩阵，其中包含 N 个单词，每个单词通过词嵌入 Word2Vec 转换为一个 d 维的向量。目标是计算一个 $N \times N$ 的注意力矩阵，其中每个元素表示两个单词之间的相似度。

对于句子中的每对单词（假设为词 i 和词 j ），计算它们的语义相似度。这可以通过余弦相似度来完成，即计算它们向量表示的点积除以它们向量的模长。然后再构建注意力矩阵，对于句子中的每一对单词，按照上述方法计算相似度，并将这个值放入注意力矩阵的相应位置。这样，矩阵的 (i, j) 位置上的值就表示了第 i 个词和第 j 个词之间的相似度。最后进行归一化，为了使注意力分布更加合理，通常会对注意力矩阵的每一行进行归一化处理。这通常是通过应用 softmax 函数来实现的，它能够确保每一行的注意力分数总和为 1。

注意力矩阵的计算提供了一种有效的方式来量化和利用序列中的词之间的相互关系。这不仅提高了模型对关键信息的捕捉能力，也增强了模型处理复杂语言数据的准确性和效率。通过这种方式，模型能够在复杂的语言环境中更好地理解 and 生成文本，从而在 NLP 任务中实现更优的性能。

3.1.5 命名重排序

命名实体重排序是一种自然语言处理技术，专注于改善和优化命名实体识别（NER）的结果。在典型的 NER 系统中，实体识别通常依赖于模型对文本中的特定实体（如人名、地点、组织等）的识别和分类。然而，这个过程可能会产生多个候选实体，尤其是在存在歧义或多义词的情况下。为了解决这个问题，命名实体重排序应运而生。

为了对 NER 系统识别出的候选实体进行评估和排序，确保上下文中最相关的实体被优先考虑。这一过程通常涉及到对每个候选实体的重要性或相关性进行打分，这一打分可能基于各种因素，如上下文一致性、实体在文本中的出现频率、以及实体与周围词汇的语义关联等。

在命名实体重排序的过程中，即使某个命名实体在初始识别阶段未获得最高得分，其在后续属性映射步骤中的得分可能会更高，从而有助于重排序。以问题“张伟教授在哪个医院工作？”为例，假设在命名实体识别阶段，“张伟”和“伟教授”都被识别为候选实体。其中，“张伟”的识别得分为 2.70，而“伟教授”的得分为 3.10。尽管“伟教授”的得分更高，但在属性映射步骤中，“张伟”的相关属性“工作医院”得分为 0.85，明显高于“伟教授”的任何相关属性得分。因此，利用属性映射的结果，即使“张伟”在初始阶段得分较低，仍可对其进行重排序，提高“张伟”作为正确命名实体的可能性。这样的方法有助于提升问答系统在处理复杂问题时的准确性。

该过程通过将命名实体识别得分（NER_SCORE）与相关属性映射得分（PROP_SCORE）结合，根据特定公式重新计算得分，从而重新排列候选实体。在实际操作中，通过查询别名词典确定命名实体 NE 对应的实体集合 E，并检查每个实体 e 是否有相关属性 PROP。如果存在，相关属性值将被加入答案集 ANS。

对于识别出的命名实体，系统首先给予它一个识别得分，即 NER_SCORE。接

着，系统根据该实体对应的最高属性映射得分，即 PROP_SCORE ，进行加权处理。这里使用一个权重因子 α ，用来平衡命名实体识别得分和属性映射得分的影响。最终的得分计算方式是将命名实体识别得分乘以 α ，加上属性映射得分乘以 $(1-\alpha)$ 。这个加权后的分数用于对所有候选命名实体进行重新排序，以决定最终的实体识别结果。公式可表述为：得分 = $\alpha * \text{命名实体识别得分} + (1-\alpha) * \text{属性映射得分}$ 。

此次通过命名实体重排序能显著提高信息提取、问答和文本分析等应用的准确度和效率。通过准确地识别和排序最关键的实体，系统能更有效地处理复杂的语言结构和含糊不清的情境，从而提高整体的处理质量和用户体验。

3.1.6 答案选择

答案选择是智能问答的一个关键组成部分，它的核心任务是从一系列候选答案中选择最佳答案。这个过程通常涉及理解用户问题的深层含义和分析与问题相关的各种答案。在实际应用中，答案选择不仅要考虑文本的字面含义，还需理解复杂的语境、隐含意义和专业知识。在执行答案选择时，系统通常先通过搜索、信息检索或预先设定的答案库获取一组候选答案。随后，利用自然语言处理（NLP）技术，如语义分析、文本匹配和深度学习模型，评估每个候选答案与问题的相关性。这一过程可能包括对问题和答案进行语义编码、比较它们的语义相似度，或使用更复杂的方法如注意力机制和神经网络进行深入分析。最终，系统根据评估结果选择最相关、最准确的答案。答案选择的准确性直接影响问答系统的有效性，是提升用户体验和满意度的关键。随着深度学习和人工智能技术的进步，答案选择的准确度和复杂度都在不断提升。

此次在实现利用 BERT 计算输入问题与三元组关系属性的相似度并进行答案匹配的过程中，首先需要进行数据准备。这包括准备包含问题、三元组及其对应答案的数据集，并对这些文本数据进行预处理，确保它们符合 BERT 的输入格式。预处理阶段可能包括分词、标记化、转换为数字表示。通过 BERT 模型对问题和三元组的关系属性进行嵌入表示。对于问题，使用 BERT 的预训练模型，输入问题并获取其嵌入表示，该表示反映了问题的语境信息。同样，对于每个三元组的关系属性，也将其输入 BERT 模型并获得相应的嵌入表示。这一步骤是关键的，因为 BERT 的上下文理解能

力能够捕捉词汇的语义以及它们在句子中的关系。

随后，计算输入问题与每个三元组的关系属性之间的相似度。这通常通过余弦相似度等度量方法实现。余弦相似度度量两个向量之间的夹角，值越接近 1 表示相似度越高。对于每个三元组，通过计算问题嵌入与属性嵌入的余弦相似度，得到一个相似度分数。

然后选择相似度分数最高的三元组作为预测答案。这一步骤通过比较问题与所有三元组的相似度，从中选取最相关的三元组，进一步提高了答案的准确性。BERT 的嵌入表示能够在语义空间中捕获问题与三元组属性之间的语义关联，使得预测更加精准。

最后，得到预测答案后，将其与正确答案进行匹配。如果预测答案与正确答案一致，即匹配成功，将正确匹配的计数值加一。这一步骤验证了模型的性能，确保模型输出的答案与真实答案的一致性，进一步提高了模型的可信度。

在技术实现的过程中，对 BERT 模型的选择和配置也是至关重要的。可以选择预训练的 BERT 模型，也可以在特定任务上进行 fine-tuning 以适应特定领域的语境。模型的超参数、学习率等调优也是一个迭代的过程，通过在训练集上进行实验，选择最佳配置以提高模型的性能。

整个过程中，技术实现的关键在于充分利用 BERT 模型的上下文理解能力，将其嵌入表示用于计算问题与三元组属性之间的相似度。通过细致的数据预处理和模型调优，确保输入数据格式的兼容性和模型的稳健性。这种技术实现方法充分发挥了 BERT 模型在自然语言处理领域的优势，为实体关系问题提供了一种高效而准确的解决方案。

3.2 基于 BERT 的命名实体识别

3.2.1 BERT 模型输入

BERT 模型自从 2018 年由 Google 推出以来，已经在自然语言处理（NLP）领域引起了革命性的变革。这一模型的核心优势在于其深度双向表示的学习，这使得它在各种语言理解任务中表现出色。理解 BERT 模型的输入是把握其工作原理的关键。

BERT 模型的输入设计非常独特，以支持它的双向训练架构。模型接受的输入主要是一系列经过预处理的令牌（token），这些令牌通常来自于文本数据。在对文本进行预处理时，首先是将文本分割成令牌，这些令牌可能是单词、子词或字符。BERT 使用 WordPiece 嵌入来处理令牌，这允许它有效地处理未知词或罕见词。

为了处理不同类型的 NLP 任务，BERT 的输入包括一系列特殊的令牌。最重要的两个是[CLS]和[SEP]。[CLS]令牌是每个输入序列的第一个令牌，其最终隐藏状态被用作整个序列的聚合表示，特别适用于分类任务。[SEP]令牌用于分隔句子，这在处理涉及多个句子的任务。BERT 还使用位置嵌入来理解令牌在句子中的位置。与传统的序列模型不同，BERT 不依赖于递归神经网络（RNN）或卷积神经网络（CNN），因此它需要某种机制来理解序列中的词汇顺序。位置嵌入通过为序列中的每个令牌位置编码一个独特的向量来实现这一点，然后将这些向量加到对应令牌的嵌入上。

在处理特定任务时，BERT 能够接受单个句子或一对句子作为输入。当模型接收一对句子时，它们被[SEP]令牌分隔，并且整个序列以[CLS]令牌开头。这样的结构使得 BERT 能够处理各种复杂的双句子任务，如问答系统中的问题和文本段落的配对。

BERT 的输入还包括段落嵌入，这有助于区分不同句子或段落。这在处理对比两个或多个不同文本段落的任务时尤为重要。整体上，BERT 的输入机制通过结合令牌嵌入、位置嵌入和特殊令牌，为模型提供了丰富的信息，使其能够理解和处理各种复杂的语言任务。它的设计不仅支持强大的双向语境理解能力，而且为不同类型的 NLP 任务提供了灵活性和适应性。

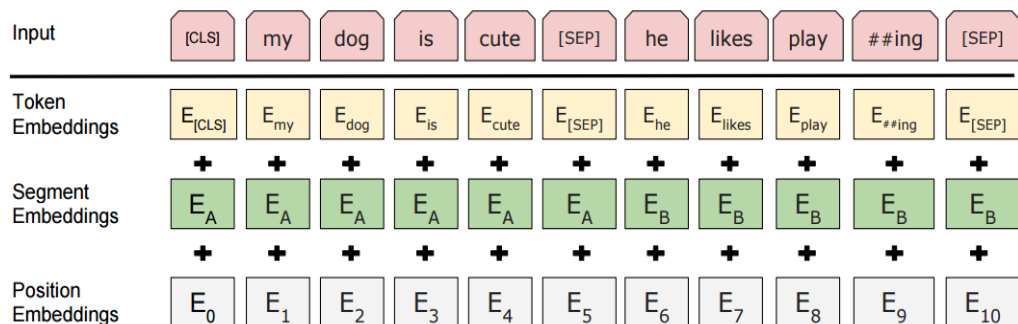


图 3-5 Embedding 可视化表示图

输入表示可以在一个词序列中表示单个文本句或一对文本(例如,[问题,答案])。对于给定的词，其输入表示是可以通过三部分 Embedding 求和组成。Embedding 的

可视化表示如图 3-5 所示。

token Embeddings 表示的是词向量，第一个单词是 CLS 标志，可以用于之后的分类任务，对于非分类任务，可以忽略词向量；Segment Embeddings 用来区别两种句子，因为预训练不只做语言模型还要做以两个句子为输入的分类任务；Position Embeddings 是通过模型学习得到的。

此次设计采用了 BERTCrf。BERTCrf 模型的出现是为了解决自然语言处理领域中命名实体识别（NER）任务中的一些挑战和问题。NER 任务旨在从文本中识别出具有特定意义的实体，如人名、地名、组织名等。这项任务对于信息提取、问答系统、机器翻译等应用具有重要意义。传统的基于规则或特征工程的方法在处理 NER 任务时往往受限于特征表达的能力和标签之间的依赖关系建模的能力。而 BERTCrf 模型则通过结合了 BERT 和 CRF 的优势，显著提高了 NER 任务的性能。BERTCrf 结构如图 3-6 所示。

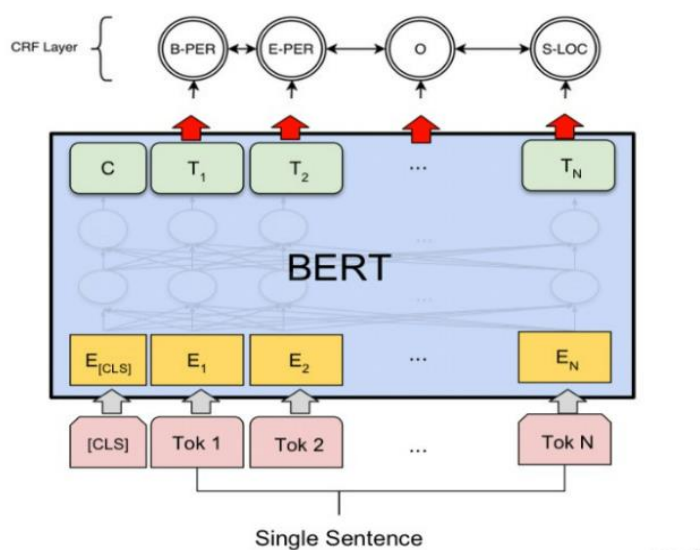


图 3-6 BERTCrf 结构

BERT 是一种基于 Transformer 结构的预训练语言模型，它通过大规模无监督学习从大量的文本数据中学习到了丰富的语言表示。BERT 模型能够利用上下文信息来生成每个词的向量表示，这使得它在处理自然语言理解任务时具有很强的表达能力。BERT 是一种革命性的自然语言处理模型，它通过深度双向表示，显著提升了文本理解的精度。其预训练过程包含两个主要任务：掩码语言模型（Masked Language Model,

MLM) 和下一句预测 (Next Sentence Prediction, NSP)。BERT 模型的基础是 Transformer 架构, 特别是其编码器部分。这种架构采用了自注意力机制, 使得模型在处理文本时能够同时考虑到每个词的上下文信息。这与传统的顺序处理模型 (如 RNN 和 LSTM) 形成对比, 后者在处理文本时受到先前词的限制, 不能有效地捕捉双向上下文。通过随机遮蔽输入序列中的一些词 (大约 15% 的 token), 然后训练模型来预测这些被遮蔽词的原始值。这个任务的设计是为了使模型能够有效地学习深度双向表示。在 MLM 中, 一些词被特殊的 "[MASK]" 标记替换, 模型需要根据上下文来预测这些被掩盖的词。

本文之所以采用这种方法的一个关键优点是能够充分利用双向上下文, 因为模型在预测每个被遮蔽词时, 都有机会考虑到它的左侧和右侧上下文。这与传统的单向语言模型不同, 后者在预测每个词时只能利用之前的词。MLM 使 BERT 能够生成更为丰富和准确的词嵌入, 从而提高在下游任务中的性能。

最后再进行下一句预测 NSP 任务旨在理解两个句子之间的关系。在这个任务中, 模型接受一对句子作为输入, 并需要判断这两个句子是否在原始文本中相邻。这一过程涉及将语料库中的句子随机配对, 其中一半的句子对是连续的, 而另一半则是不连续的。模型必须预测哪些句子对是连续的, 哪些不是。这个任务对于理解句子间的关系至关重要, 尤其是对于那些需要文本间关联理解的任务, 在此次智能问答系统和自然语言推理中通过 NSP, BERT 学习了如何从更广泛的角度理解文本, 这不仅仅限于单个句子或短语, 还包括了整个段落或对话。

BERT 的预训练过程是理解其在各种 NLP 任务中表现卓越的关键。通过在大规模语料库上进行预训练, BERT 能够学习到丰富的语言表示, 这些表示可以很好地迁移到各种特定的下游任务上。预训练过程中获得的深层次语言理解是 BERT 在各种任务上取得优异性能的基石。

在预训练过程中, BERT 使用了大量的无标签文本数据, 例如维基百科和书籍语料库。这使得模型能够在没有特定任务指导的情况下, 学习到词汇、语法和语言结构的丰富知识。通过这些广泛的数据上进行预训练, BERT 能够捕捉到语言的复杂性和细微差别。

尽管预训练是 BERT 成功的关键，但微调同样重要。此次在特定任务问答上进行微调时，BERT 模型会在其预训练得到的知识基础上，进一步学习特定任务的细节和特点。这个过程通常涉及到少量标记数据，用于指导模型在特定任务上的性能优化。

BERT 的预训练过程通过 MLM 和 NSP 两种任务，为模型提供了一个坚实的基础，使其能够更好地理解和处理自然语言。这种深度双向表示的学习对于实现复杂的语言理解任务至关重要，是 BERT 在各种 NLP 挑战中取得优异成绩的核心。通过在大规模无标签数据上进行预训练，BERT 能够把握语言的丰富性和多样性，进而在特定任务的微调阶段展现出显著的适应性和精准性。

在命名实体识别（NER）任务中，实体通常表现为连续的词序列，因此，准确建模标签之间的依赖关系对于精确界定实体边界至关重要。条件随机（Conditional Random Field，简称 CRF）作为一种概率图模型，在序列化数据的标注问题中扮演着关键角色，如自然语言处理中的命名实体识别和词性标注等任务。CRF 模型巧妙地结合了马尔可夫随机场和最大熵模型的优势，为序列标注任务提供了强大的性能保障。

CRF 模型的核心思想在于构建观测序列与标记序列之间的联合概率分布模型。通过对模型参数的精细学习，CRF 能够寻找出最佳的标记序列，从而实现对数据的精确标注。在 CRF 的框架下，观测序列代表输入的数据序列，而标记序列则对应着这些数据序列的标注结果。CRF 模型主要由两部分构成：特征表示和参数学习。特征表示是将观测序列和标记序列的特征转化为模型能够处理的形式。这些特征通常包括局部特征和全局特征，用以描述序列的固有属性，如当前词的词性、上下文词的信息等。这些特征的设计既可以是基于经验的手工构建，也可以通过机器学习方法自动学习获得。通过精细设计的特征表示和参数学习，CRF 模型能够充分利用输入序列的上下文信息，有效建模标签之间的依赖关系，从而在 NER 任务中取得优异的表现。

参数学习是指通过训练数据来估计 CRF 模型的参数，使得模型能够最大化训练数据的似然函数。常用的方法包括极大似然估计和正则化方法，如 L2 正则化，以防止过拟合。CRF 模型有两种常见的形式：线性链条件随机场和全局条件随机场。线

性链条件随机场适用于序列标注任务，如命名实体识别和词性标注。在该模型中，标记序列的条件概率由观测序列和标记序列的特征函数的线性组合得到，并通过对数线性模型进行建模。其条件概率可以通过前向-后向算法高效地计算得到。全局条件随机场适用于更复杂的结构化预测任务，如句法分析和语义角色标注。与线性链条件随机场不同，全局条件随机场允许特征函数直接依赖于整个观测序列和标记序列，因此能够捕捉更长距离的依赖关系。

CRF 模型在自然语言处理等领域取得了广泛的应用，其优点包括模型能够建模序列间的复杂依赖关系、具有良好的泛化能力和灵活性等。然而，CRF 模型也存在一些挑战，如需要手工设计特征、参数学习过程中的计算复杂度高等问题。

BERTCrf 模型将 BERT 模型和 CRF 模型相结合，形成了一个端到端的序列标注模型。在训练过程中，模型首先利用 BERT 模型对输入文本进行特征提取，将每个 token 转换为对应的向量表示。然后，将这些向量表示输入到 CRF 层中，CRF 层考虑了标签之间的依赖关系，并对整个序列进行标签推断，得到最可能的实体标签序列。在反向传播过程中，通过最小化 CRF 层的损失函数来调整模型参数，使得模型能够更准确地预测实体标签。

BERTCrf 模型的优势在于它能够充分利用 BERT 模型强大的语言表示能力和 CRF 模型建模标签依赖关系的能力。与传统的基于规则或特征工程的方法相比，BERTCrf 模型更加灵活和高效。它能够自动学习文本中的特征，并能够捕获到标签之间复杂的依赖关系，从而在 NER 任务中取得更好的性能。

此外，BERTCrf 模型还具有良好的泛化能力。由于 BERT 模型是通过大规模无监督学习得到的通用语言表示，因此 BERTCrf 模型在处理各种类型的文本数据时都能够取得良好的效果。这使得它在不同领域和不同语言的 NER 任务中都能够得到有效的应用。BERTCrf 模型通过结合了 BERT 和 CRF 的优势，提高了 NER 任务的性能，并在自然语言处理领域中取得了广泛的应用。随着 BERT 和 CRF 模型的不断发展和改进，相信 BERTCrf 模型在未来会有更广阔的应用前景。

3.2.2 BERTForSequenceClassification 模型

BERTForSequenceClassification 是一种预训练的深度学习模型，用于句子分类任

务。该模型基于 BERT 架构，能够对输入的文本序列进行编码，并输出相应的分类结果。通常情况下，BERT 模型可以用于多种文本分类任务，例如情感分析、文本匹配和问题分类等。在本文中，BERTForSequenceClassification 被设计用于问题与属性拼接，即判断一个给定的问题是否与某个属性相关。

BERT 模型是一种预训练的深度双向变换器（Transformer）模型，其核心思想是利用大规模未标记的文本数据进行预训练，然后在特定任务上进行微调。BERT 模型由多个 Transformer 编码器组成，每个编码器都包含多头自注意力机制和前馈神经网络。这使得 BERT 模型能够在处理文本序列时捕捉到上下文信息和句子之间的关系，从而在各种自然语言处理任务中表现出色。

在使用 BERTForSequenceClassification 进行句子分类任务时，通常需要将输入文本序列编码为 BERT 模型所需的输入格式。具体来说，需要对输入文本进行分词，并将其转换为 BERT 模型能够理解的词嵌入向量表示。BERT 模型接受的输入格式通常包括 token embeddings（标记嵌入）、segment embeddings（段嵌入）和 position embeddings（位置嵌入），其中 token embeddings 用于表示单词在词汇表中的索引，segment embeddings 用于区分不同句子，而 position embeddings 则用于表示单词在句子中的位置信息。对于问题与属性拼接的任务，首先需要将问题和属性进行合并，并添加一些特殊的标记来区分它们。例如，可以使用特殊的标记[CLS]表示序列的开始，然后将问题和属性连接起来，最后再添加一个特殊的标记[SEP]表示序列的结束。这样处理后的文本序列就可以输入到 BERT 模型中进行处理。

一旦输入文本序列被转换为 BERT 模型所需的输入格式，接下来就可以将其输入到 BERTForSequenceClassification 模型中进行分类。在模型的输出端，通常会得到一个关于每个类别的得分向量，然后通过 softmax 函数将这些得分转换为概率值，最终确定文本序列所属的类别。在训练 BERTForSequenceClassification 模型时，通常会使用带标签的训练数据集，并通过反向传播算法来调整模型参数，使得模型在训练数据上的损失函数最小化。

在训练过程中，还可以使用一些常见的优化器，如 Adam 优化器，来进一步优化模型参数。一旦模型训练完成，就可以在新的文本序列上进行预测或推断。对于问题

与属性拼接的任务，可以将待分类的问题与各个属性进行拼接，然后通过 BERTForSequenceClassification 模型来预测每个属性的相关性。根据模型输出的概率值，可以确定问题与哪些属性相关性更高，从而对问题进行分类或判断。

3.3 基于 BIOES 标注的 NER 模型训练

3.3.1 数据集创建

数据集的创建过程可以大致分为四步，具体过程如表 3-1 所示。

第一，初始化。首先创建一个 Build_Ner_data 类，该类在初始化过程中完成了以下任务：建立标签类型与索引之间的映射关系，确保了数据的一致性和可检索性；设定文本处理的最大长度，以适应不同长度的文本输入；定义一个常用标点符号列表，以便在后续处理中进行过滤；并且，为每种标签类型构建并加载 Aho-Corasick，这是一种高效的字符串匹配算法，能够加速命名实体的识别过程。

第二，定义文本分割函数 split_text(text)。在文本分割阶段，split_text 函数首先替换了文本中的换行符，以确保文本的连贯性。随后，利用正则表达式将文本分割为独立的句子，并在此过程中过滤掉标点符号，从而得到纯净的句子文本。为了适应后续处理的需要，还依据一定的规则将这些句子进一步分割成较短的文本片段。此外，为了提高数据的多样性，可能随机替换部分文本片段的末尾标点。

第三，定义实体标签生成函数 make_text_label(text)。在实体标签生成阶段，make_text_label 函数首先将所有字符的标签初始化为'O'，表示这些字符非命名实体。接着，利用之前加载的 Aho-Corasick 自动机来识别文本中的命名实体。识别到的实体按照长度进行降序排序，这是为了避免在处理过程中出现实体重叠的问题。随后，为这些识别到的实体分配 B-I (Begin-Inside) 标签，以标识实体的起始和内部字符，同时确保非实体部分被正确标记。

第四，定义数据写入文件函数 build_file(all_text, all_label)。在数据写入文件阶段，build_file 函数首先打开一个用于写入的文件。随后，遍历所有经过处理的文本及其对应的标签，将这些数据逐字符写入文件。这一过程确保了数据集以标准格式存储，便于后续机器学习或深度学习模型的训练和评估。

表 3-1 数据集创建

Algorithm
1. 类 Build_Ner_data: 2. 初始化: 3. 定义标签类型列表 idx2type 和对应的索引字典 type2idx 4. 设置最大文本长度 max_len 5. 定义常用标点符号列表 p 6. 为每种标签类型创建一个 ahocorasick 自动机，并加载对应标签的实体列表 7. 函数 split_text(text): 8. 替换文本中的换行符为逗号 9. 使用正则表达式分割文本为句子，并过滤掉标点符号，得到纯文本句子列表 sentences_text 和标点符号列表 sentences_Punctuation 10. 初始化空列表 split_text 用于存储分割后的短文本 11. 初始化空字符串 now_text 用于拼接当前处理的短文本 12. 遍历 sentences_text，根据一定规则将句子拼接到 now_text，或将其存入 split_text 并开始新的 now_text 13. 对 split_text 中的部分文本进行末尾标点的随机替换 14. 返回分割后的短文本列表 split_text 15. 函数 make_text_label(text): 16. """ 17. 通过 ahocorasick 自动机对文本进行实体识别，并生成 NER 标签 18. """ 19. 初始化标签列表 label，全部设为'O'（表示非实体） 20. 初始化标志位 flag 为 0 21. 初始化字典 mp 为空 22. 对于每种标签类型 type 在 idx2type 中: 23. 使用 ahocorasick 自动机识别文本中的实体 24. 如果识别到实体，则按实体长度降序排序 25. 对于每个识别到的实体 en: 26. 获取实体的结束位置和名称 27. 计算实体的起始位置 28. 如果起始或结束位置已在 mp 中，则跳过该实体 29. 为实体范围内的字符设置 B-I 标签 30. 增加 flag 计数 31. 将实体范围内的位置添加到 mp 中 32. 返回标签列表 label 和 flag 33. 函数 build_file(all_text, all_label): 34. """ 35. 将文本和对应的 NER 标签写入文件 36. """ 37. 打开文件 ner_data_aug.txt 进行写入 38. 对于每对文本和标签在 all_text 和 all_label 中: 39. 将文本和对应标签逐字符写入文件 40. 每个文本标签对之后写入一个换行符

3.3.2 数据集标注

本次采用 BIOES 数据集标注是一种用于命名实体识别任务的标注方法。该方法通过在文本中标注实体的起始位置和结束位置，以及实体的类型，为模型提供了训练所需的数据^[76]。训练文本与训练标签如图 3-7 所示。

用来干什么的，尿钠	O O O O O O O B-che E-che
可以查出啥疾病，尿 17-生酮类固醇	O O O O O O O O B-che I-che I-che I-che I-che I-che I-che E-che
用来干什么的，粪便脂肪定量	O O O O O O O B-che I-che I-che I-che I-che E-che
有什么用，脑脊液游离型髓鞘碱性蛋白	O O O O O B-che I-che I-che I-che I-che I-che I-che I-che I-che E-che
有啥用，单克隆丙种球蛋白（MMG）	O O O O B-che I-che I-che I-che I-che I-che I-che I-che I-che E-che
可以查出什么疾病，4字试验	O O O O O O O O B-che I-che I-che E-che
蛋白结合碘（PBI）可以查出哪些病	B-che I-che I-che I-che I-che I-che I-che I-che E-che O O O O O O
Ⅱ级产前超声检查有哪些用	B-che I-che I-che I-che I-che I-che I-che E-che O O O O
可以查出哪些病，足长身高预测法	O O O O O O O B-che I-che I-che I-che I-che E-che
用来干什么的，骨骼动诊	O O O O O O O B-che I-che I-che E-che
有啥用，导管吹张法	O O O O B-che I-che I-che I-che E-che
肺顺应性（C）能查出啥病	B-che I-che I-che I-che I-che I-che I-che E-che O O O O O
甲状腺吸碘率试验可以查出啥疾病	B-che I-che I-che I-che I-che I-che I-che E-che O O O O O O
可以查出什么疾病，硝基四唑氮蓝试验	O O O O O O O O B-che I-che I-che I-che I-che I-che E-che
有啥用，血清钠	O O O O B-che I-che E-che
能查出啥病，斑疹伤寒血清学检测	O O O O O O B-che I-che I-che I-che I-che I-che I-che E-che
可以查出哪些病，电子束CT	O O O O O O O O B-che I-che I-che I-che E-che
可以查出什么疾病，白细胞计数（WBC）	O O O O O O O O B-che I-che I-che I-che I-che I-che I-che E-che
狼疮细胞试验可以查出哪些疾病	B-che I-che I-che I-che I-che E-che O O O O O O O

图 3-7 训练文本与标签

标注过程包括以下三个关键步骤：

首先，对原始文本进行预处理，包括分词和词性标注，以便后续的标注工作。然后，识别出需要标注的命名实体，这可以通过人工标注、自动标注工具或已有的标注库来完成。定义标注规则，明确每个实体的起始位置和结束位置，并给出相应的标签。BIOES 标签包括 B（Begin）、I（Inside）、O（Outside）、E（End）和 S（Single）五种类型，分别表示实体的起始、内部、外部、结束和单独出现^[77]。

然后，根据标注规则，对识别出的每个实体进行标注。如果实体只有一个词，则标记为 S；如果实体是多个词的连续片段，并且是实体的起始部分，则第一个词标记为 B，其余词标记为 I；如果实体是多个词的连续片段，并且是实体的结束部分，则最后一个词标记为 E，其余词标记为 I；如果实体不是多个词的连续片段，则所有词都标记为 O。

最后，在标注完成后，对标注结果进行校对和修正，确保标注的准确性和一致性，然后将标注好的文本数据组织成适合模型训练的数据集格式，通常包括将文本分割成句子并添加标注信息。

BIOES 数据集标注具有较高的表达能力和灵活性，能够有效地捕捉文本中的命

名实体信息，为命名实体识别任务提供了重要的数据支持。

3.3.3 NER 训练

NER 训练过程大致可以分为三步：

第一步，初始化。定义一个继承自 `nn.Module` 的 `BERT_Model` 类，然后在构造函数中，加载预训练的 BERT 模型，并初始化一个 LSTM 网络和分类器。

第二步，前向传播。输入数据 `x` 先经过 BERT 处理，得到最后一层的隐藏状态；BERT 的输出传递给 LSTM 进行序列建模；LSTM 的输出再经过分类器，得到预测结果 `pre`。

第三步，损失计算与预测。如果有真实标签 `label`，则计算预测结果与标签之间的交叉熵损失，并返回损失值；如果没有标签，则返回预测结果中概率最大的标签索引，这通常用于测试或实际应用。

具体过程如表 3-2 所示。

表 3-2 基于 BERT 的 NER 训练

Algorithm
1. 类 <code>BERT_Model</code> 继承自 <code>nn.Module</code>; 2. 构造函数 <code>__init__(model_name, hidden_size, tag_num, bi)</code>; 3. 调用父类的构造函数 4. 初始化 BERT 模型，从预训练模型中加载 5. 初始化 LSTM 网络，输入大小为 768，隐藏层大小为 <code>hidden_size</code>，层数为 2，<code>batch_first</code> 为 <code>True</code>，是否双向由 <code>bi</code> 决定 6. 如果 LSTM 是双向的，则分类器的输入大小为 <code>hidden_size*2</code>，否则为 <code>hidden_size</code> 7. 分类器的输出大小为 <code>tag_num</code> 8. 初始化交叉熵损失函数，<code>ignore_index</code> 设置为 0 9. 前向传播函数 <code>forward(x, label=None)</code>; 10. 使用 BERT 模型处理输入 <code>x</code>，并返回最后一层的隐藏状态，<code>attention_mask</code> 设置为 <code>x > 0</code> 11. 使用 LSTM 网络处理 BERT 的输出 12. 使用分类器处理 LSTM 的输出得到预测结果 <code>pre</code> 13. 如果有标签 <code>label</code>: 14. 计算交叉熵损失，并将预测结果和标签都重塑为一维 15. 返回损失值 16. 否则: 17. 返回预测结果中概率最大的标签索引，并去除额外的维度

3.4 基于数据增强策略和 TF-IDF 算法的模型改进

3.4.1 基于数据增强策略的模型改进

在此次研究中,为了提高模型的泛化能力和性能,设计了三种数据增强策略:实体替换、实体掩码、以及实体拼接。这些策略可以有效地丰富训练数据的多样性,使得模型在面对各种实体和上下文时能够做出更为准确的预测。

(1) 实体替换:通过替换文本中的实体,让模型学会从变化的上下文中捕捉关键信息。这种方法增强了模型对不同实体的适应能力。

(2) 实体掩码:在某些训练样本中,故意掩盖部分实体信息,迫使模型学会从剩余的信息中推断出缺失的部分。这种策略提升了模型对信息不完整情况的处理能力。

(3) 实体拼接:通过将不同实体拼接在一起形成新的训练样本,模拟更复杂、更丰富的语境,从而提高模型处理复杂情况的能力。

通过应用这些策略,显著提升了两个先进模型——BERT-base-chinese 和 chinese-roBERTa-wwm-ext——在测试集上的性能。

表 3-3 测试结果

模型名称	未数据增强	数据增强
BERT-base-chinese	97.13%	97.42%
chinese-roBERTa-wwm-ext	96.77%	97.40%

由表 3-3 可知,在未进行数据增强时,BERT-base-chinese 的 F1 Score 为 97.13%,而 chinese-roBERTa-wwm-ext 的 F1 Score 为 96.77%。在实施了数据增强后,两个模型的 F1 Score 分别提升至 97.42%和 97.40%,这充分证明了数据增强策略的有效性。

3.4.2 基于 TF-IDF 算法的实体对齐

在处理自然语言任务时,确保模型的输出与现有知识图谱中的实体名一致至关重要。为了实现这一目标,采用了 TF-IDF(Term Frequency-Inverse Document Frequency, 词频-逆文档频率)算法来进行实体对齐。TF-IDF(词频-逆文档频率)是一种常用于

文本分析和信息检索的权重计算方法。它的主要思想是，如果某个词在一份文档中出现的频率很高，而在其他文档中出现的频率很低，则认为这个词具有很好的类别区分能力，适合用来分类。

TF-IDF 实际上是 TF（词频）与 IDF（逆文档频率）的乘积。具体来说：

（1）词频（Term Frequency, TF）：指的是某一个给定的词语在该文档中出现的频率。对于文档 D 中的词语 t ，其词频可以定义为： $F(t) = \frac{\text{词语 } t \text{ 在文档 } D \text{ 中出现的次数}}{\text{文档 } D \text{ 中的总词数}}$ 。这个公式用于计算词语在单个文档中的重要性。

（2）逆文档频率（Inverse Document Frequency, IDF）：是一个词语普遍重要性的度量。对于语料库中的词语 t ，其逆文档频率可以定义为： $DF(t) = \log_e \left(\frac{\text{语料库中的文档总数}}{\text{包含词语 } t \text{ 的文档数} + 1} \right)$ 加 1 是为了避免分母为 0 的情况。IDF 反映了词语在整个语料库中的稀缺程度，如果一个词在多个文档中都出现，那么它的 IDF 值会较低，反之则会较高。

（3）TF-IDF 值：就是 TF 和 IDF 的乘积，即： $F_{\text{TF-IDF}}(t) = F(t) \times DF(t)$ 这样计算得到的 TF-IDF 值既考虑了词语在单个文档中的重要性（由 TF 反映），也考虑了词语在整个语料库中的稀缺性（由 IDF 反映）。因此，TF-IDF 方法常被用于文本挖掘和信息检索中，以评估一个词在特定文档中的重要性。

TF-IDF 算法通过分析文本中词语的频率和在整个语料库中的罕见程度，为每个词语分配一个权重。在本次研究场景中，这个算法帮识别出文本中最重要实体，并将它们与知识图谱中的标准实体名进行匹配。通过这种方法，不仅能够确保模型的输出与知识图谱保持一致，还能提高实体识别的准确率和效率。这对于后续的信息检索、数据分析等任务至关重要，因为它确保了数据的一致性和准确性。具体的 TF-IDF 算法步骤如表 3-4 所示。

表 3-4 TF-IDF 算法

Algorithm
1. 定义文档集合 $D = \{d_1, d_2, \dots, d_n\}$
2. 定义词汇表 $V = \{t_1, t_2, \dots, t_n\}$
3. // 计算每个词的 TF(t, d)

Algorithm

4. 对于每个文档 $d \in D$:
5. 对于每个词 $t \in V$:
6. $tf(t, d) = \text{词 } t \text{ 在文档 } d \text{ 中出现的次数} / \text{文档 } d \text{ 中的总词数}$
7. // 计算每个词的 $IDF(t)$
8. 对于每个词 $t \in V$:
9. 包含词 t 的文档数 $= 0$
10. 对于每个文档 $d \in D$:
11. 如果词 t 在文档 d 中出现:
12. 包含词 t 的文档数 $+= 1$
13. $idf(t) = \log_e(D \text{ 的总文档数} / \text{包含词 } t \text{ 的文档数})$
14. // 计算每个词的 TF-IDF 值
15. 对于每个文档 $d \in D$:
16. 对于每个词 $t \in V$:
17. $tfidf(t, d) = tf(t, d) * idf(t)$

需要注意的是，这里的对数是以自然常数 e 为底的。在实际应用中，可以根据需要选择其他的底数。此外，为了避免除数为零的情况，在计算 IDF 时，通常会在“包含词 t 的文档数”后加上一个小的平滑值（如 1），尽管在上面的算法中未明确显示这一点。表 3-4 提供了一个 TF-IDF 计算的基本框架。在实际应用中，可能还需要进行额外的优化和调整，比如考虑文本的预处理（如去除停用词、词形还原等），以及使用更高效的数据结构来存储和计算词频和文档频率。

总的来说，通过结合数据增强策略和 TF-IDF 实体对齐技术，显著提升了模型在实体识别任务上的性能，并确保模型输出与知识图谱的高度一致性。

3.5 基于 Adam 优化器的 BERT 算法优化

3.5.1 BERT 算法改进

在 Adam 优化器的初期阶段，由于一阶动量（ m ）和二阶动量（ v ）的值较小，可能导致最初的优化方向偏离理想方向。这是因为在训练初始阶段，梯度较小，使得一阶动量和二阶动量的计算结果也较小。这会影响到模型的优化效果，使得模型的收敛速度较慢，甚至可能在训练初期陷入局部最优解。

为了解决这个问题，一种常见的方法是对一阶动量进行修正。具体来说，可以使用指数加权平均的方式对一阶动量进行调整，以增大其值，使得优化方向更为准确。

这种修正方式可以使得在训练初期梯度较小的情况下，仍然能够保持较大的一阶动量值，从而更好地指导参数更新方向，提高模型的优化效果。

另外，关于 Adam 优化器中的权重衰减问题，BERT 源码中采用了一种特殊的权重衰减方式，以达到等效于 L2 正则化的效果。通常情况下，L2 正则化会通过添加在损失函数中添加正则项的方式来限制模型参数的大小，以防止过拟合。

然而，在 Adam 优化器中直接添加权重的平方项并不是一种正确的 L2 正则化方式，因为这样会影响一阶动量和二阶动量的计算，从而影响参数的更新方向。BERT 中采用了一种特殊的权重衰减方式，来达到与 L2 正则化类似的效果，同时不影响一阶动量和二阶动量的计算。

具体而言，可能涉及到对梯度进行调整，使得在参数更新时能够考虑到权重的大小，但不直接影响到优化器的一阶动量和二阶动量的计算。这样可以保持优化器的性能，并且实现了对模型参数的正则化效果，提高了模型的泛化能力。在 Adam 上直接添加权重的平方不是一种正确的 L2 正则/权重衰减的方式，为了不影响 m 和 v ，本文使用了另一种的权重衰减方式，来等效 L2 正则。

首先，L2 Norm 的算法如式(3.1)：

$$f_t^{reg}(\theta) = f_t(\theta) + \frac{\lambda'}{2} \|\theta\|_2^2 \quad (3.1)$$

weight decay 如式(3.2)示：

$$\theta_{t+1} = (1 - \lambda)\theta_t - \alpha \nabla f_t(\theta_t) \quad (3.2)$$

在标准 SGD 下，进行参数修正，如式(3.3)所示：

$$\lambda' = \frac{\lambda}{\alpha} \quad (3.3)$$

由于两者是等价的，所以在实现时可以直接用 weight decay 来实现。而当在 Adam 中使用 L2 Norm 后，如式(3.4)所示：

$$\theta_L \leftarrow \theta_{t-1} - \alpha \frac{\beta_1 m_{t-1} + (1 - \beta_1)(\nabla f_t + \lambda \theta_{t-1})}{\sqrt{\hat{v}_t} + \varepsilon} \text{diag} \quad (3.4)$$

update 项中与权重 θ 有关的项为式(3.5)：

$$\frac{\lambda \theta_{t-1}}{\sqrt{\hat{v}_t}} \quad (3.5)$$

在优化过程中，二阶动量（通常指的是梯度平方的累积值，用 v_t 表示）扮演着关键的角色。当在模型训练过程中观察到梯度在某个方向上变化迅速时，这通常意味着该方向上的参数对损失函数的影响较大。相应地，这个方向上的二阶动量 v_t 的数值也会显著增大，反映了这一方向上梯度的波动性和重要性。

然而，当探讨 Adam 优化器在 BERT 等模型中的应用时，会发现它在处理权重衰减这一正则化策略时存在一些问题。理想情况下，权重衰减应当独立于梯度下降的方向，即无论在哪个方向上，模型参数都应当受到相同程度的正则化。这种一致性对于防止模型过拟合至关重要，因为它可以确保模型在训练过程中不会因特定方向上的过度拟合而损失泛化能力。在原始的 Adam 优化器中，权重衰减的实施与梯度下降的方向产生了意料之外的关联。具体来说，权重衰减的数值会受到梯度大小和方向的影响，导致在梯度变化较快的方向上，权重受到的正则化程度相对较低。这种情况显然不符合期望的一致性正则化效果，也可能因此增加了模型过拟合的风险。

为了克服这一限制，本文提出一种有效的改进方法，即将权重衰减与梯度下降过程进行解耦。这种方法的核心思想是在梯度下降步骤完成之后，再对模型参数实施独立的权重衰减操作。通过这种方式，可以模拟 L2 正则化的效果，即向损失函数添加一个与参数平方和成正比的项，以限制模型参数的绝对值大小。

这种解耦的权重衰减策略具有显著的优势。首先，它确保了梯度下降方向的纯粹性，使得模型能够按照损失函数的梯度信息在参数空间中寻找最优；其次，权重衰减能够在各个方向上对参数施加一致的正则化作用，有效避免了因方向性差异导致的过拟合问题；最后，这种策略有助于降低模型在未见过的数据上的错误率，提高模型的泛化能力和鲁棒性。因此，在实际操作中，推荐在梯度下降步骤完成后，再进行权重衰减操作，如表 3-5 所示。

表 3-5 优化过程

Algorithm	Adam with L_2 regularization and Adam with decoupled weight decay (AdamW)
1:	given $\alpha = 0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon = 10^{-8}$, $\lambda \in \mathbb{R}$
2:	initialize time step $t \leftarrow 0$, parameter vector $\theta_{t=0} \in \mathbb{R}^n$, first moment vector $m_t=0 \leftarrow 0$, second moment

Algorithm Adam with L₂ regularization and Adam with decoupled weight decay (AdamW)

```

vector  $v_{t=0} \leftarrow 0$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$  ▷select batch and return the corresponding gradient
6:    $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ 
7:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$  ▷here and below all operations are element-wise
8:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
9:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  ▷ $\beta_1$  is taken to the power of t
10:   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  ▷ $\beta_2$  is taken to the power of t
11:   $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$  ▷can be fixed, decay, or also be used for warm restarts
12:   $\theta_t \leftarrow \theta_{t-1} - \eta_t (\alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda \theta_{t-1})$ 
13: until stopping criterion is met
14: return optimized parameters  $\theta_t$ 

```

算法 Adam 和 AdamW 都使用了梯度下降法来优化参数，但加入了不同的正则化项。Adam 算法中，L2 正则化项与梯度一同被用于更新参数，而 AdamW 算法则将权重衰减项与参数更新步骤解耦。在 Adam 和 AdamW 算法中，都需要初始化时间步、参数向量、一阶矩向量、二阶矩向量和调度乘数。

在每一轮迭代中，算法会从训练集中选取一个批次，并计算相应的梯度。然后，利用梯度、一阶矩向量、二阶矩向量和调度乘数来更新参数。当满足停止条件时，算法返回优化后的参数。Adam 和 AdamW 算法的不同之处在于正则化项的处理方式，Adam 算法在更新参数时考虑了梯度的一阶和二阶矩，而 AdamW 算法则在参数更新中加入了权重衰减项，即 L2 正则化，这有助于减少过拟合现象，提高模型的泛化能力。尽管两者在正则化处理上有所不同，但它们的核心思想是相似的，都旨在通过自适应学习率的策略来提高优化过程的效率和效果，都采用了自适应学习率的方法来提高优化效果。

3.5.2 算法分析

(1) 时间复杂度

该算法的时间复杂度主要取决于梯度计算、动量更新、偏差修正、参数更新、学习率调度等五个操作。

梯度计算 (SelectBatch 和 $\nabla f_t(\theta_{t-1})$): 通常与模型的复杂度和所选批量的大小成正比。假设模型有 n 个参数, 批量大小为 b , 那么这部分的时间复杂度可以粗略估计为 $O(bn)$ (取决于具体的模型架构和计算框架)。动量更新 (m_t 和 v_t): 这两个操作都是元素级的, 时间复杂度与参数数量成正比, 即 $O(n)$ 。偏差修正 ($\hat{m}_t$ 和 $\hat{v}_t$): 这些操作也是元素级的, 时间复杂度为 $O(n)$ 。参数更新 (θ_t): 同样是元素级的操作, 时间复杂度为 $O(n)$ 。学习率调度 ($\text{SetScheduleMultiplier}(t)$): 取决于具体的调度策略, 但通常是一个常数时间操作或依赖于 t 的低阶操作, 可以视为 $O(1)$ 。

综合以上各项, 总的时间复杂度是这些操作的组合, 但主要由梯度计算和参数更新决定, 因为它们涉及更复杂的数学运算。因此, 整体时间复杂度可以近似为 $O(bn)$, 其中 b 是批量大小, n 是参数数量。

(2) 空间复杂度

该算法的空间复杂度取决于其在内存中存储的数据量, 具体包括参数 (θ_t)、梯度 (g_t)、动量 (m_t 和 v_t)、学习率调度器 (η_t) 和其他辅助变量 (如 $\hat{m}_t$ 和 $\hat{v}_t$)。

参数 (θ_t): 需要存储模型的参数, 空间复杂度为 $O(n)$ 。梯度 (g_t): 需要存储当前批量的梯度, 空间复杂度为 $O(n)$ 。动量 (m_t 和 v_t): 需要存储一阶和二阶动量, 空间复杂度为 $O(n)$ 。学习率调度器 (η_t): 这可能是一个简单的变量或更复杂的结构, 但通常其空间复杂度可以忽略不计, 视为 $O(1)$ 。其他辅助变量 (如 $\hat{m}_t$ 和 $\hat{v}_t$): 这些是临时变量, 可以在原地更新, 不需要额外的空间。

综合以上各项, 整体空间复杂度主要由参数、梯度和动量决定, 因此空间复杂度为 $O(n)$, 其中 n 是参数数量。

3.5.3 算法优化结果

表 3-6 BERT 算法优化结果

	H=128	H=256	H=512	H=768
L=2	2/128(BERT-Tiny)	2/256	2/512	2/768
L=4	4/128	4/256 (BERT-Mini)	4/512 (BERT-Small)	4/768
L=6	6/128	6/256	6/512	6/768
L=8	8/128	8/256	8/512 (BERT-Medium)	8/768
L=10	10/128	10/256	10/512	10/768
L=12	12/128	12/256	12/512	12/768(BERT-Base)

BERT 模型的不同配置选项，主要涉及到隐藏层的大小（H）和模型层数（L）。隐藏层大小决定了模型内部表示的能力，而层数则影响着模型能够捕获的语言结构和关系的复杂程度。

H（隐藏层大小）是 BERT 模型中每一层 Transformer 的嵌入维度。数值越大，模型能够表示的信息就越丰富，但同时也需要更多的计算资源。通常，较大的隐藏层大小意味着模型具有更强的表示能力，但也可能导致更高的计算成本和参数数量。L（模型层数）是 BERT 模型中 Transformer 层的数量。层数越多，模型就越能够捕获输入数据中的深层和复杂结构。然而，增加层数也可能导致模型更加复杂，从而增加过拟合的风险和计算成本。

算法优化结果如表 3-6 所示。表格中的每一行表示具有特定隐藏层大小和层数的 BERT 模型配置。例如，“2/128(BERT-Tiny)”表示一个具有 2 层 Transformer 和隐藏层大小为 128 的 BERT 模型，通常被视为一个非常小的变体，适合在资源受限的环境中使用。而“12/768(BERT-Base)”则表示一个标准的 BERT 基础模型，具有 12 层 Transformer 和隐藏层大小为 768，这是在许多自然语言处理任务中常用的配置。通过调整这些参数，可以根据具体需求选择合适的 BERT 模型大小。大型模型（如 BERT-Base 或具有更多层和更大隐藏层大小的变体）通常具有更高的性能，但也需要更多的计算资源。相反，小型模型（如 BERT-Tiny 或 BERT-Mini）可能在性能上稍逊一筹，但更适合在资源有限的场景中使用。

表 3-7 不同测试集对应的 GLUE 分数表

Model	CoLA	SST-2	MRPC	STS-B	QQP	MNLI-m	MNLI-mm	QNLI (v2)	RTE	WNLI
BERT-Tiny	0.0	83.2	81.1/ 71.1	74.3/ 73.6	62.2/ 83.4	70.2	70.3	81.5	57.2	62.3
BERT-Mini	0.0	85.9	81.1/ 71.8	75.4/ 73.3	66.4/ 86.2	74.8	74.3	84.1	57.9	62.3
BERT-Small	27.8	89.7	83.4/ 76.2	78.8/ 77.0	68.1/ 87.0	77.6	77.0	86.4	61.8	62.3
BERT-Medium	38.0	89.6	86.6/ 81.6	80.4/ 78.4	69.6/ 87.9	80.0	79.1	87.7	62.2	62.3

BERT 模型在不同测试集对应的 GLUE 分数如表 3-7 所示。该表格展示了不同大小的 BERT 模型（BERT-Tiny、BERT-Mini、BERT-Small、BERT-Medium）在多个自然语言处理任务上的性能评分。每个 BERT 模型都在一系列标准数据集上进行了评估，这些数据集涵盖了语法正确性（CoLA）、情感分析（SST-2）、语义等价性（MRPC）、语义相似性（STS-B）、问题对匹配（QQP）、自然语言推理（MNLI-m、MNLI-mm、QNLI、RTE）以及词序预测（WNLI）等任务。

- **Model:** 模型的名称，指示了 BERT 模型的大小。
- **CoLA:** Corpus of Linguistic Acceptability，一个用于评估语法正确性的数据集。
- **SST-2:** Stanford Sentiment Treebank 2，一个用于情感分析的数据集。
- **MRPC:** Microsoft Research Paraphrase Corpus，一个用于评估语义等价性的数据集，即判断两个句子是否表达相同的意思。
- **STS-B:** Semantic Textual Similarity Benchmark，一个用于评估两个句子语义相似性的数据集。
- **QQP:** Quora Question Pairs，一个用于问题对匹配的数据集，即判断两个问题是否相似或相同。
- **MNLI-m 和 MNLI-mm:** Multi-Genre Natural Language Inference，一个用于自然语言推理的数据集。MNLI-m 是匹配的开发集，而 MNLI-mm 是不匹配的开发集，用于评估模型在不同领域数据上的泛化能力。

- **QNLI(v2)**: Question-answering NLI, 一个从 SQuAD 问答数据集转换而来的自然语言推理任务。
- **RTE**: Recognition of Textual Entailment, 另一个用于自然语言推理的数据集。
- **WNLI**: Winograd NLI, 一个用于评估模型在解决特定类型指代消解问题上的能力。

每个单元格中的分数代表了对应模型在特定任务上的性能。对于 MRPC 和 QQP 任务,通常会有两个分数,第一个是在开发集上的 F1 分数,第二个是在测试集上的精度。

根据表 3-7,可以观察到随着模型大小的增加(从 BERT-Tiny 到 BERT-Medium),性能普遍提高,这表明更大的模型具有更强的表示能力和更好的性能。然而,也需要注意到模型大小的增加通常伴随着计算成本和存储需求的增加。因此,在选择模型时需要根据实际需求和资源限制进行权衡。

下面链接了采用全词掩蔽策略的预训练模型。在数据利用和模型训练的过程中,确保了所有其他条件保持一致,这些模型不仅结构上与原始模型相吻合,在词汇层面也保持了高度的一致性。特别地,本研究聚焦于 BERT 大型模型,并选取了 BERT Large 的全词掩蔽变体进行深入探讨。通过实验,能够更加细致地挖掘和分析全词掩蔽在预训练模型中的实际应用效果。以下是结果如表 3-8 所示:

表 3-8 不同 BERT 大型模型对应的结果

Model	SQUAD 1.1 F1/EM	Multi NLI Accuracy
BERT-Large, Uncased (Original)	91.0/84.3	86.05
BERT-Large, Uncased (Whole Word Masking)	92.8/86.7	87.07
BERT-Large, Cased (Original)	91.5/84.8	86.09
BERT-Large, Cased (Whole Word Masking)	92.9/86.7	86.46

表 3-8 展示了不同配置的 BERT-Large 模型在两项自然语言处理任务上的性能。BERT-Large 是 BERT 模型的一个较大版本,通常具有更好的性能,但也需要更多的

计算资源。表格中的模型配置主要涉及到是否区分大小写（Cased vs. Uncased）以及是否使用全词掩蔽（Whole Word Masking）。

Model: 模型的名称和配置。Cased 表示模型区分大小写，即它会考虑输入文本中的大小写差异；Uncased 则表示模型不区分大小写，所有输入文本都会被转换为小写。Whole Word Masking 是一种 BERT 训练时的技术，它会确保在遮盖（即替换为 [MASK] 标记）时保持完整的单词，这有助于模型更好地学习词汇的边界。SQUAD 1.1 F1/EM: SQuAD (Stanford Question Answering Dataset) 是一个用于问答系统的数据集。F1 和 EM (Exact Match) 是评估问答系统性能的常见指标。F1 分数结合了精确率和召回率，提供了一个平衡的评估。EM 分数则是直接衡量模型回答是否完全匹配参考答案。在这个表格中，F1 和 EM 分数都是越高越好，表明模型在问答任务上的性能越好。Multi NLI Accuracy: Multi-Genre Natural Language Inference (MultiNLI) 是一个用于自然语言推理任务的数据集。Accuracy (准确率) 是评估模型在这个任务上性能的主要指标，即模型正确判断文本对之间关系的比例。准确率越高，说明模型在自然语言推理任务上的表现越好。

从表格中的数据可以看出：

(1) 使用全词掩蔽 (Whole Word Masking) 的模型在 SQUAD 1.1 和 MultiNLI 任务上的性能普遍优于不使用全词掩蔽的模型。这可能是因为全词掩蔽有助于模型更好地学习词汇边界和语义信息。

(2) 区分大小写的模型 (Cased) 在 SQUAD 1.1 任务上的性能略优于不区分大小写的模型 (Uncased)，但在 MultiNLI 任务上的性能差异不大。这可能表明大小写信息对于某些任务（如问答）来说是有用的，但对于其他任务（如自然语言推理）来说可能不那么重要。

3.6 实验结果及分析

本文采用精确率 P 、召回率 R 和 $F1$ 值来评判意图识别和槽位填充两个模型的性能。

Precision (P)：精确度是指模型预测为正例的样本中真正为正例的比例。它衡量了模型预测为正例的可靠性。**Recall (R)**：召回率是指实际为正例的样本中被模型预测为正例的比例。它衡量了模型找出所有正例的能力。**F1 Score (F1)**：F1 分数是精确度和召回率的调和平均值，用于综合评估模型的性能。它同时考虑了模型在精确度和召回率上的表现，提供了一个平衡的评估指标。

各评价指标的计算方法如式(3.6)所示：

$$\begin{cases} P = \frac{\text{正确识别的标签个数}}{\text{识别出的所有标签个数}} \times 100\% \\ R = \frac{\text{正确识别的标签个数}}{\text{所有标签个数}} \times 100\% \\ F1 = \frac{2 \times P \times R}{P + R} \times 100\% \end{cases} \quad (3.6)$$

在本实验中，选择了 BERT Base 预训练模型，这是一个经过广泛验证的强大模型，它具有 12 层编码器、768 个隐藏单元和 12 个注意头，总共拥有 110M 个参数。这个选择基于 BERT 在自然语言处理任务上的卓越性能和其在各个领域的广泛应用。为了搭建提出的模型，采用了 PyTorch 作为深度学习框架，因为 PyTorch 具有灵活的设计和易用的接口，能够帮助快速实现和调试模型。

针对意图分类模型，设置了序列长度为 45，这是经过实验和调研后确定的最优长度。这个长度可以在保持有效信息的同时，减少计算负担和训练时间。而对于槽位填充模型，将序列长度设置为 128，这是因为在这个任务中，需要更长的序列来捕捉更多的上下文信息，以便准确地填充槽位。在模型的隐藏层方面，对于槽位填充模型，选择了 128 个隐藏单元。这个大小的隐藏层经过实验验证能够在保持模型效率的同时，提供足够的表示能力来处理槽位填充任务。而学习率方面，将其设置为 1E-5，这是在训练过程中经验性地调整得到的最优值。这个学习率可以使模型在训练过程中更好地收敛，并且有助于避免梯度爆炸或梯度消失的问题。

最后，将训练次数设置为 100，这是为了充分训练模型并确保其收敛到最佳性能。通过这些设置，能够构建出高性能的意图分类和槽位填充模型，在各自的任务上取得了令人满意的结果。

为了评估提出的模型的性能，进行了与几个基线模型的比较。对于意图分类模型，考虑了几种经典的文本分类模型，包括 Fasttext、LSTM 和 Text-CNN。而针对槽位填充模型，将目的模型与 LSTM、LSTM-CRF、BiLSTM 和 BiLSTM-CRF 模型进行了对比。选择这些基线模型是因为它们在自然语言处理领域中已经被广泛研究和应用。

Fasttext 是一种快速文本分类模型，它利用词袋模型和简单的神经网络进行分类，通常用于文本分类任务。LSTM（长短期记忆网络）是一种循环神经网络，适用于处理序列数据，如自然语言文本。Text-CNN 是基于卷积神经网络的文本分类模型，通过卷积操作来提取文本特征，适用于捕获局部特征和上下文信息。针对槽位填充模型，选择了几种具有代表性的模型。LSTM-CRF 模型结合了 LSTM 和条件随机场（CRF），在序列标注任务中表现出色。BiLSTM 模型是双向 LSTM，能够同时考虑输入序列的过去和未来信息，适用于捕获上下文相关性。BiLSTM-CRF 模型结合了双向 LSTM 和 CRF，充分利用了序列数据的双向信息和标签之间的依赖关系。

为了进一步扩展的实验设计，采取了严格的评估方法，确保结果的可靠性和可复现性。首先，对数据集进行了预处理，包括文本清洗、分词、标记化等，以确保输入数据的一致性和标准化。接着，将数据集划分为训练集、验证集和测试集，通常采用 70/15/15 的比例进行划分，以便在训练模型时进行验证和调优，并在独立的测试集上进行最终评估。

针对模型的训练过程，采用了适当的超参数调优策略，如学习率调整、批量大小选择、正则化等，以确保模型能够在训练集上充分学习，并在验证集上泛化良好。还利用交叉验证技术来评估模型的稳定性和一般化能力，通过多次训练和验证，减少因随机初始化或数据划分而引入的偏差。

在评估指标方面，考虑了多个常见的文本分类和序列标注指标，如准确率、召回率、F1 分数等，以全面评估模型在不同方面的表现。同时，也关注了模型的训练和推理速度，特别是在大规模数据集上的效率，以便在实际应用中能够满足实时性和效率的要求。

表 3-9 QIC 数据集实验结果 (%)

模型	P	R	F1
Fasttext	70.84	62.15	66.21
LSTM	72.72	68.63	70.61
Text-CNN	73.40	70.23	71.78
BERT	82.35	80.96	81.65

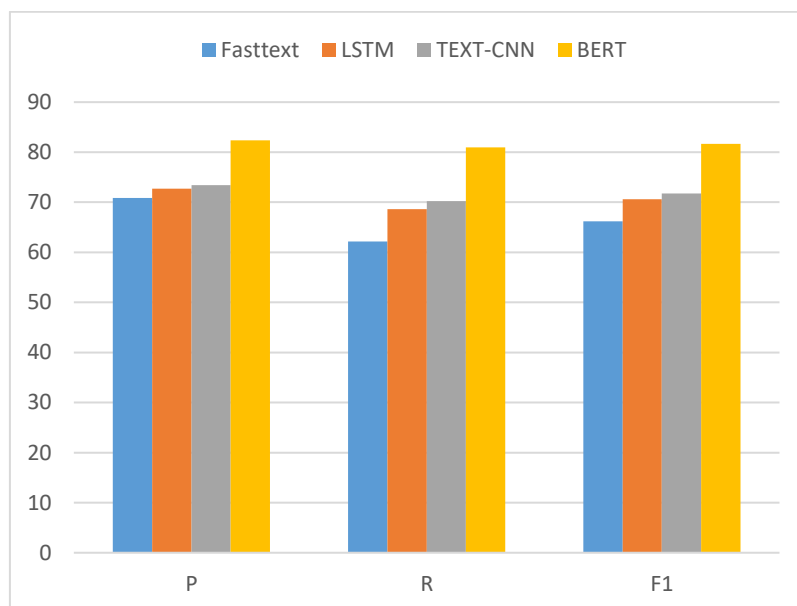


图 3-8 QIC 数据集实验结果 (%)

表 3-9 和图 3-8 QIC 数据集实验结果 (%) 展示了四种不同的文本分类模型 (Fasttext、LSTM、Text-CNN、BERT) 在某一文本分类任务上的性能评估结果。每个模型在精确度 (Precision, P)、召回率 (Recall, R) 和 F1 分数 (F1 Score) 三个指标上进行了评估。

从表格中可以看出, BERT 模型在精确度、召回率和 F1 分数三个指标上都表现最好, 超过了其他三种模型十个百分点左右。这说明 BERT 模型在文本分类任务上具有更好的性能。其他模型如 Text-CNN、LSTM 和 Fasttext 则在性能上稍逊于 BERT, 但各自也有其特点和适用场景。

表 3-10 CCKS2017 数据集实验结果 (%)

模型	P	R	F1
LSTM	75.59	78.29	76.92
LSTM-CRF	83.59	85.28	84.42
BiLSTM	82.42	84.67	83.53
BiLSTM-CRF	84.69	87.31	85.98
BERT-BiLSTM-CRF	86.51	90.48	88.45

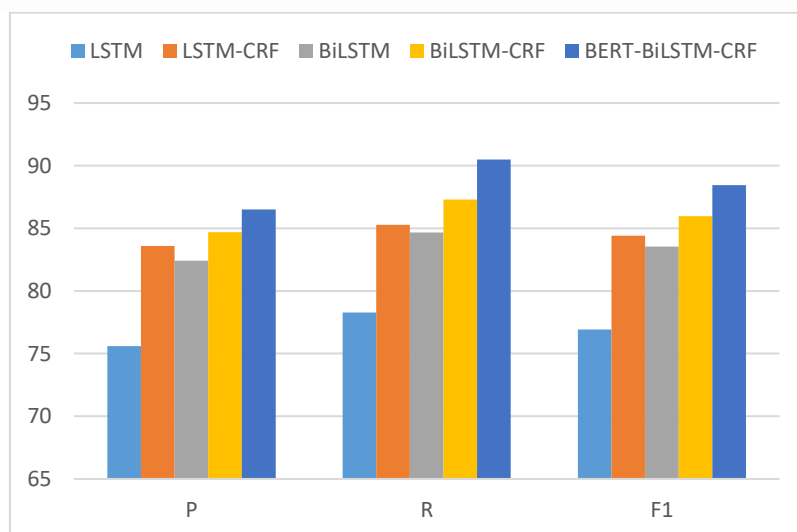


图 3-9 CCKS2017 数据集实验结果 (%)

表 3-10 和图 3-9 展示了五种不同的模型(LSTM、LSTM-CRF、BiLSTM、BiLSTM-CRF 和 BERT-BiLSTM-CRF)在文本分类或命名实体识别(NER)任务上的性能评估结果。

LSTM: 作为基准模型, LSTM(长短时记忆网络)在三个指标上的表现相对较低。**LSTM-CRF:** 将条件随机场(CRF)层添加到 LSTM 模型后,性能得到了显著提升,尤其是在召回率和 F1 分数上。CRF 层常用于序列标注任务,能够考虑标签之间的依赖关系,从而提高性能。**BiLSTM:** 双向 LSTM(BiLSTM)通过同时考虑输入序列的前后信息,相比单向 LSTM 在性能上有所提升。**BiLSTM-CRF:** 结合 BiLSTM 和 CRF 层的模型在三个指标上都取得了更好的性能,特别是在 F1 分数上。**BERT-**

BiLSTM-CRF: 将 BERT 与 BiLSTM 和 CRF 层结合起来的模型在所有指标上都达到了最佳性能。BERT 模型能够提供强大的上下文表示,与 BiLSTM 和 CRF 层相结合,进一步提升了模型的性能。

本次研究在 QIC 和 CCKS2017 数据集上进行了实验,以评估各模型的性能。通过比较提出的模型与基线模型在这些数据集上的表现,能够评估的模型在意图分类和槽位填充任务上的效果。这些实验结果对于了解模型的优势和不足,并指导在未来的改进和优化方向具有重要意义。

3.7 本章小结

本章深入探讨了基于深度学习的智能问答模型,特别聚焦于命名实体识别(NER)和实体匹配两大核心任务。通过结合 BERT、LSTM 和 CRF 模型,构建了一个高效的命名实体识别系统,显著提高了实体识别的准确性和鲁棒性。此外,还探讨了数据增强策略和 TF-IDF 实体对齐技术,有效缓解了数据稀疏和实体不一致性问题。

4 基于 BERT 预训练模型的中文文本分类

本章主要进行基于 BERT 预训练模型的中文文本分类。包括数据预处理、加载预训练模型、文本分类等过程。

4.1 基于 padding 与 mask 的数据预处理

4.1.1 数据输入

在构建数据集之前，首先需要知道的是模型到底应该接收什么样的输入，然后才能构建出正确的数据形式。在上面说到，基于 BERT 的文本分类模型的输入就等价于 BERT 模型的输入。

由于对于文本分类这个场景来说其输入只有一个序列，所以在构建数据集的时候并不需要构造 Segment Embedding 的输入，直接默认使用全为 0 即可。同时，对于 Position Embedding 来说在任何场景下都不需要对其指定输入，因为在代码实现时已经做了相应默认时的处理。

因此，对于文本分类这个场景来说，只需要构造原始文本对应的 Token 序列，并在首位分别再加上一个[CLS]符和[SEP]符作为输入即可。

4.1.2 数据语料

在这里，使用到的数据集是今日头条开放的一个新闻分类数据集[3]，一共包含有 382688 条数据，15 个类别。

同时已经将其进行了格式化处理，以 7:2:1 的比例划分成了训练集、验证集和测试集三个部分。如下所示便是部分示例数据：

- 1 千万不要乱申请网贷，否则后果很严重_!_4
- 2 10 年前的今天，纪念 5.12 汶川大地震 10 周年_!_11
- 3 怎么看待杨毅在一 NBA 直播比赛中说詹姆斯的球场统治力已经超过乔丹、伯德和科比? _!_3
- 4 戴安娜王妃的车祸有什么谜团? _!_2

其中_!_左边为新闻标题，也就是后面需要用到的分类文本，右边为类别标签。

4.1.3 处理流程

现在有两个样本构成了一个 batch，那么其整个数据的处理过程则大致分为四个步骤：

第 1 步需要将原始的数据样本进行分字 (tokenize) 处理；第 2 步再根据 tokenize 后的结果构造一个字典，不过在使用 BERT 预训练时并不需要自己来构造这个字典，直接载入谷歌开源的 vocab.txt 文件构造字典即可，因为只有 vocab.txt 中每个字的索引顺序才与开源模型中每个字的 Embedding 向量一一对应的。第 3 步则是根据字典将 tokenize 后的文本序列转换为 Token 序列，同时在 Token 序列的首尾分别加上[CLS]和[SEP]符号，并进行 Padding。第 4 步则是根据第 3 步处理后的结果生成对应的 Padding Mask 向量。

需要注意的是，在模型训练时只需要将第 3 步和第 4 步处理后的结果一起喂给模型即可。

4.1.4 数据集构建

数据集的构建大致分为五步：定义 tokenize、建立词表、转换为 Token 序列、padding 处理与 mask 以及构造 DataLoader 与使用示例。

第 1 步：定义 tokenize

第 1 步需要完成的就是将输入进来的文本序列 tokenize 到字符级别。对于中文语料来说就是将每个字和标点符号都给切分开。在这里，使用 transformers 包中的 BERTTokenizer 方法来完成，如表 4-1 所示：

表 4-1 定义 tokenize 过程

Algorithm
<pre>1. 创建一个名为 ModelConfig 的实例，并将其配置赋值给 model_config 变量 2. model_config = ModelConfig(pretrained_model_name_or_path='BERT-base-chinese') # 假设 ModelConfig 类已定义，并接受预训练模型路径 3. 使用 model_config 中指定的预训练模型目录来加载 BERTTokenizer，并将 tokenize 方法赋值给 tokenizer 变量 4. from transformers import BERTTokenizer 5. tokenizer = BERTTokenizer.from_pretrained(model_config.pretrained_model_name_or_path) # 假设 ModelConfig 类有 pretrained_model_name_or_path 属性</pre>

Algorithm

6. 使用 `tokenizer` 方法将字符串 "青山不改，绿水长流，我们月来客栈见！" 进行分词，并打印分词结果
7. `tokens_1 = tokenizer.tokenize("青山不改，绿水长流，我们月来客栈见！")`
8. `print("对于第一个字符串:")`
9. `print(tokens_1)`
10. 使用 `tokenizer` 方法将字符串 "10 年前的今天，纪念 5.12 汶川大地震 10 周年" 进行分词，并打印分词结果
11. `tokens_2 = tokenizer.tokenize("10 年前的今天，纪念 5.12 汶川大地震 10 周年")`
12. `print("对于第二个字符串:")`
13. `print(tokens_2)`

第 2 步：建立词表

由于 BERT 预训练模型中已经有了一个给定的词表 (`vocab.txt`)，因此并不需要根据自己的语料来建立一个词表。当然，也不能够根据自己的语料来建立词表，因为相同的字在自己构建的词表中 `vocab.txt` 中的索引顺序肯定会不一样，而这就会导致后面根据 `token id` 取出来的向量是错误的。将 `vocab.txt` 中的内容读取进来形成一个词表，接着定义一个 `build_vocab()` 方法来实例化一个词表。

在经过上述处理后，通过 `vocab.itos` 得到一个列表，返回词表中的每一个词；通过 `vocab.itos[2]` 返回得到词表中对应索引位置上的词；通过 `vocab.stoi` 得到一个字典，返回词表中每个词的索引；通过 `vocab.stoi['月']` 返回得到词表中对应词的索引；通过 `len(vocab)` 来返回词表的长度。如下便是建立后的词表：

```
1 {'[PAD]': 0, '[unused1]': 1, '[unused2]': 2, '[unused3]': 3,
2 '[unused4]': 4, '[unused5]': 5, '[unused6]': 6, '[unused7]': 7,
3 '[unused8]': 8, '[unused9]': 9, '[unused10]': 10, '[unused11]': 11,
4 '[unused12]': 12, '[unused13]': 13, '[unused14]': 14, '[unused15]': 15,
5 '[unused16]': 16, '[unused17]': 17, '[unused18]': 18, '[unused19]': 19,
6 '[unused20]': 20, '[unused21]': 21, '[unused22]': 22, '[unused23]': 23,
7 '[unused24]': 24, '[unused25]': 25, '[unused26]': 26, '[unused27]': 27,
8 '[unused28]': 28, '[unused29]': 29, '[unused30]': 30, '[unused31]': 31,
9 '[unused32]': 32, '[unused33]': 33, '[unused34]': 34, '[unused35]': 35,
10 '[unused36]': 36, '[unused37]': 37, '[unused38]': 38, '[unused39]': 39,
```

此时，定义一个 `LoadSingleSentenceClassificationDataset` 类，并在类的初始化过程中根据训练语料完成字典的构建等工作，如表 4-2 所示：

表 4-2 字典构建等过程

Algorithm
1. 定义一个类 <code>LoadSingleSentenceClassificationDataset</code>： 2. 初始化方法 <code>__init__</code> 接收以下参数： 3. <code>vocab_path</code>（默认 <code>'./vocab.txt'</code>）：词汇表文件路径 4. <code>tokenizer</code>（默认 <code>None</code>）：分词器对象 5. <code>batch_size</code>（默认 <code>32</code>）：批处理大小 6. <code>max_sen_len</code>（默认 <code>None</code>）：最大句子长度 7. <code>split_sep</code>（默认 <code>'\n'</code>）：样本分割符 8. <code>max_position_embeddings</code>（默认 <code>512</code>）：最大位置嵌入数量 9. <code>pad_index</code>（默认 <code>0</code>）：填充索引 10. <code>is_sample_shuffle</code>（默认 <code>True</code>）：是否打乱样本顺序 11. 在初始化方法中执行以下操作： 12. 设置 <code>self.tokenizer</code> 为传入的 <code>tokenizer</code> 13. 调用 <code>build_vocab</code> 函数，传入 <code>vocab_path</code>，并将返回值设置为 <code>self.vocab</code> 14. 设置 <code>self.PAD_IDX</code> 为 <code>pad_index</code> 15. 尝试从 <code>self.vocab</code> 中获取 <code>'[SEP]'</code> 的索引，并设置为 <code>self.SEP_IDX</code> 16. 尝试从 <code>self.vocab</code> 中获取 <code>'[CLS]'</code> 的索引，并设置为 <code>self.CLS_IDX</code> 17. 设置 <code>self.batch_size</code> 为 <code>batch_size</code> 18. 设置 <code>self.split_sep</code> 为 <code>split_sep</code> 19. 设置 <code>self.max_position_embeddings</code> 为 <code>max_position_embeddings</code> 20. 如果 <code>max_sen_len</code> 是整数且大于 <code>max_position_embeddings</code>，则设置 <code>max_sen_len</code> 为 <code>max_position_embeddings</code> 21. 设置 <code>self.max_sen_len</code> 为 <code>max_sen_len</code> 22. 11. 设置 <code>self.is_sample_shuffle</code> 为 <code>is_sample_shuffle</code>

在上述算法中，`vocab_path` 表示本地词表的路径。`max_sen_len` 表示最大样本长度，当 `max_sen_len = None` 时，即以每个 batch 中最长样本长度为标准，对其它进行 padding；当 `max_sen_len = 'same'` 时，以整个数据集中最长样本为标准，对其它进行 padding；当 `max_sen_len = 50`，表示以某个固定长度符样本进行 padding，多余的截掉。`split_sep` 表示样本与标签之间的分隔符。`is_sample_shuffle` 表示是否打乱数据集。第 14-15 行为建立词表并取对应特殊字符的索引；第 18 行中 `max_position_embeddings` 为最大样本长度，最大为 512。第 19-21 行则是用来判断传入的最大样本长度。

第 3 步：转换为 Token 序列

在得到构建的字典后，通过如表 4-3 的算法来将训练集、验证集和测试集转换成 Token 序列：

表 4-3 转换为 Token 序列

Algorithm
1. 定义一个方法 <code>data_process</code> 接收参数 <code>filepath</code>： 2. 打开文件 <code>filepath</code> 并以 <code>utf8</code> 编码读取内容，将内容逐行读入到 <code>raw_iter</code> 迭代器中 3. 初始化一个空列表 <code>data</code> 用于存储处理后的数据 4. 初始化变量 <code>max_len</code> 为 0，用于记录最长句子的长度 5. 遍历 <code>raw_iter</code> 中的每一行 <code>raw</code>（显示进度条，每列显示 80 个字符） 6. 移除行尾的换行符，并使用 <code>split_sep</code> 分隔符将行分割成列表 <code>line</code> 7. 从 <code>line</code> 中提取句子 <code>s</code> 和标签 <code>l</code> 8. 初始化一个临时列表 <code>tmp</code>，将分类标记 <code>CLS_IDX</code> 添加到 <code>tmp</code> 的开头 9. 使用分词器 <code>tokenizer</code> 对句子 <code>s</code> 进行分词，并查找每个 <code>token</code> 在词汇表 <code>vocab</code> 中的索引，将索引添加到 <code>tmp</code> 中 10. 如果 <code>tmp</code> 的长度大于 <code>max_position_embeddings - 1</code>，则截断 <code>tmp</code> 到该长度（BERT 预训练模型只取前 512 个字符） 11. 在 <code>tmp</code> 的末尾添加分隔标记 <code>SEP_IDX</code> 12. 将 <code>tmp</code> 转换为 PyTorch 的长整型张量 <code>tensor_</code> 13. 将标签 <code>l</code> 转换为 PyTorch 的长整型张量 14. 更新 <code>max_len</code> 为 <code>tensor_</code> 的长度和 <code>max_len</code> 的较大值 15. 将 <code>(tensor_, l)</code> 添加到 <code>data</code> 列表中 16. 返回 <code>data</code> 列表和 <code>max_len</code>

在上述算法中，第 6-7 行便是用来取得文本和标签；第 8 行则是首先对序列进行 `tokenize`，然后转换成 Token 序列并在最前面加上分类标志位[CLS]。第 9-11 行则是用来对 Token 序列进行截取，最长为 `max_position_embeddings` 个字符即 512，并同时在末尾加上[SEP]符号。不过不加也不会有影响，因为这本来是单个序列的分类。第 14 行则是用来保存最长序列的长度。在处理完成后，4.1.2 节中的 4 个样本将会被转换成如下形式：

```
1 tensor([[ 101, 1283,  674,  679, 6206,  744, 4509, 6435, 5381, 6587, 8024, 1415,
2 1156, 1400, 3362, 2523,  698, 7028,  102,    0,    0,    0,    0,    0,
3 0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0,
4 0,    0,    0],
5 [ 101, 8108, 2399, 1184, 4638,  791, 2399, 8024, 5279, 2573, 126, 119,
6 8110, 3746, 2335, 1920, 1765, 7448, 8108, 1453, 2399, 102,    0,    0,
```

```
7 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
8 0, 0, 0],
9 [ 101, 2582, 720, 4692, 2521, 3342, 3675, 1762, 671, 8391, 4684, 3064,
10 3683, 6612, 704, 6432, 6285, 1990, 3172, 4638, 4413, 1767, 5320, 3780,
11 1213, 2347, 5307, 6631, 6814, 730, 710, 510, 843, 2548, 1469, 4906,
12 683, 8043, 102],
13 [ 101, 2785, 2128, 2025, 4374, 1964, 4638, 6756, 4877, 3300, 784, 720,
14 6466, 1730, 8043, 102, 0, 0, 0, 0, 0, 0, 0, 0,
15 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
16 0, 0, 0]]
17 torch.Size([39, 4])
```

从上面的输出结果可以看出,101 就是[CLS]在词表中的索引位置,102 则是[SEP]在词表中的索引;其它非 0 值就是 tokenize 后的文本序列转换成的 Token 序列。同时可以看出,这里的结果是以第 3 个样本的长度 39 对其它样本进行 padding 的,并且 padding 的 Token ID 为 0。

第 4 步: padding 处理与 mask

从第 3 步的输出结果看出,在对原始文本序列 tokenize 转换为 Token ID 后还需要对其进行 padding 处理。对于这一处理通过如表 4-4 的过程来完成:

表 4-4 padding 处理

Algorithm

1. 定义一个函数 pad_sequence,接收以下参数:
2. sequences: 待填充的张量列表
3. batch_first: 是否将批处理维度作为第一个维度(默认为 False)
4. max_len: 最大序列长度(默认为 None,即自动计算)
5. padding_value: 填充值(默认为 0)
6. 如果 max_len 为 None:
7. 计算 sequences 中所有张量的最大长度,并赋值给 max_len
8. 始化一个空列表 out_tensors 用于存储填充后的张量
9. 遍历 sequences 中的每个张量 tensor:
10. 如果 tensor 的长度小于 max_len:
11. 创建一个新的张量,其元素全部为 padding_value,长度为 max_len - tensor.length
12. 将新张量拼接 to tensor 后面,使得总长度为 max_len
13. 否则:

Algorithm

14. 截断 `tensor` 使其长度为 `max_len`
15. 将处理后的 `tensor` 添加到 `out_tensors` 列表中
16. 将 `out_tensors` 列表中的张量堆叠为一个新的张量，堆叠维度为 1（即列方向）
17. 如果 `batch_first` 为 `True`:
18. 将堆叠后的张量的第一和第二个维度进行转置，使得批处理维度成为第一个维度
19. 返回填充并堆叠后的张量

在上述算法中，`sequences` 为待 `padding` 的序列所构成的列表，其中的每一个元素为一个样本的 Token 序列；`batch_first` 表示是否将 `batch_size` 这个维度放在第 1 个；`max_len` 表示指定最大序列长度，当 `max_len = 50` 时，表示以某个固定长度对样本进行 `padding` 多余的截掉，当 `max_len=None` 时表示以当前 `batch` 中最长样本的长度对其它进行 `padding`。第 2-3 行用来获取 `padding` 的长度；第 5-11 行则是遍历每一个 Token 序列，根据 `max_len` 来进行 `padding`。

进一步，定义了 `generate_batch()` 函数来对每个 `batch` 的 Token 序列进行 `padding` 处理，如表 4-5 所示：

表 4-5 批处理 `padding` 过程

Algorithm

1. `def generate_batch(self, data_batch):`
2. `# 初始化批次句子和标签列表`
3. `batch_sentence = []`
4. `batch_label = []`
5. `# 遍历数据批次中的每个样本`
6. `对于 data_batch 中的每个 (句子, 标签):`
7. `# 将句子和标签添加到对应的列表中`
8. `将句子添加到 batch_sentence`
9. `将标签添加到 batch_label`
10. `# 使用 padding_sequence 对句子列表进行填充`
11. `# 填充到指定长度 self.max_sen_len, 并使用 self.PAD_IDX 作为填充值`
12. `# 填充后的句子列表转换为 [max_len, batch_size] 的形式`
13. `batch_sentence = 填充句子列表(batch_sentence, 填充值=self.PAD_IDX,`
14. `是否批次优先=否, 最大长度=self.max_sen_len)`
15. `# 将标签列表转换为 PyTorch 张量, 数据类型为 long`
16. `batch_label = 转换为 PyTorch 张量(batch_label, 数据类型=torch.long)`
17. `# 返回填充后的句子张量和标签张量`
18. `返回 batch_sentence, batch_label`

上述算法的作用就是对每个 batch 的 Token 序列进行 padding 处理。最后，对于每一序列的 attention_mask 向量，只需要判断其是否等于 padding_value 便可以得到这一结果。

第 5 步：构造 DataLoader 与使用示例

经过前面 4 步的操作，整个数据集的构建已经基本完成，再构造一个 DataLoader 迭代器，算法如表 4-6 所示：

表 4-6 迭代器构建过程

Algorithm
1. 定义一个方法 load_train_val_test_data，接收以下参数： 2. train_file_path: 训练数据文件路径（默认为 None） 3. val_file_path: 验证数据文件路径（默认为 None） 4. test_file_path: 测试数据文件路径（默认为 None） 5. only_test: 是否仅加载测试数据（默认为 False） 6. 执行以下步骤： 7. 调用 self.data_process 方法处理测试数据文件，得到 test_data 和其他不需要的变量 8. 构造一个 DataLoader 对象 test_iter，用于加载测试数据，设置 batch_size、shuffle=False 和自定义的 collate_fn=self.generate_batch 9. 如果 only_test 为 True，则只返回 test_iter 10. 如果不是仅加载测试数据，则继续以下步骤： 11. 调用 self.data_process 方法处理训练数据文件，得到 train_data 和最大句子长度 max_sen_len 12. 如果 self.max_sen_len 的值为 'same'，则将 max_sen_len 赋值给 self.max_sen_len 13. 调用 self.data_process 方法处理验证数据文件，得到 val_data 14. 构造一个 DataLoader 对象 train_iter，用于加载训练数据，设置 batch_size、shuffle=self.is_sample_shuffle 和自定义的 collate_fn=self.generate_batch 15. 构造一个 DataLoader 对象 val_iter，用于加载验证数据，设置 batch_size、shuffle=False 和自定义的 collate_fn=self.generate_batch 16. 返回 train_iter、test_iter 和 val_iter

在上述算法中，第 5-7 行用来得到预处理后的数据并构造对应的 DataLoader，其中 generate_batch 将作为一个参数传入来对每个 batch 的样本进行处理；第 8-9 行则判断是否只返回测试集；同理，第 10-18 行则是用来构造相应的训练集和验证集。

表 4-7 数据集构建与使用

Algorithm
1. # 实例化 ModelConfig，获取模型配置信息 2. 实例化 model_config 为 ModelConfig 的一个对象

Algorithm

```
3. # 使用 ModelConfig 初始化 LoadSingleSentenceClassificationDataset, 并设置数据集参数
4. 实例化 load_dataset 为 LoadSingleSentenceClassificationDataset 的一个对象, 参数包括:
5. vocab_path = model_config.vocab_path
6. tokenizer = 使用预训练模型从 model_config.pretrained_model_dir 初始化 BERTTokenizer
7. batch_size = model_config.batch_size
8. max_seq_len = model_config.max_seq_len
9. split_sep = model_config.split_sep
10. max_position_embeddings = model_config.max_position_embeddings
11. pad_index = model_config.pad_token_id
12. is_sample_shuffle = model_config.is_sample_shuffle
13. # 加载训练、验证和测试数据集
14. train_iter, test_iter, val_iter = load_dataset.load_train_val_test_data(
15. model_config.train_file_path,
16. model_config.val_file_path,
17. model_config.test_file_path )
18. # 遍历训练数据集中的一批样本
19. 对于 train_iter 中的每个样本 sample 和标签 label:
20. # 打印样本的形状 (预期为 [seq_len, batch_size])
21. 打印 sample.shape
22. # 打印样本转置后的结果 (期望是 [batch_size, seq_len])
23. 打印 sample.transpose(0, 1)
24. # 生成 padding mask (标记哪些位置是填充的)
25. padding_mask = (sample 等于 load_dataset.PAD_IDX).transpose(0, 1)
26. # 打印 padding mask
27. 打印 padding_mask
28. # 打印标签
29. 打印 label
30. # 仅打印一批数据后退出循环
31. 跳出循环
```

在完成类 LoadSingleSentenceClassificationDataset 所有的编码过程后, 通过如表 4-7 所示的过程进行使用。执行完上述算法后便可以得到如下所示的结果:

```
1 torch.Size([39, 4])
2 tensor([[ 101, 1283,  674,  679, 6206,  744, 4509, 6435, 5381, 6587, 8024,
3 1156, 1400, 3362, 2523,  698, 7028,  102,    0,    0,    0,    0,
4 0,    0,    0,    0,    0,    0,    0,    0,    0,    0,    0,
5 0,    0,    0],
6 ...
7 ])
```

```
8 tensor([[False, False, False, False, False, False, False, False, False, False,
9 False, False, False, False, False, False, False, False, False, True,
10 True, True, True, True, True, True, True, True, True, True,
11 True, True, True, True, True, True, True, True, True, True],
12 ...
13 ])
14 tensor([ 4,...])
```

到此，整个数据集构建结束，接下来对加载预训练模型进行微调。

4.2 加载 BERT 预训练模型

在进行模型微调之前，需要知道拿到一个模型参数后怎么读取以及分析。下面是以 PyTorch 训练的 BERT-base-chinese 模型参数进行的读取和分析的过程。

4.2.1 查看模型参数

对于一个通过 PyTorch 框架保存的模型参数，通过如表 4-8 的方式来进行载入：

表 4-8 查看参数模型

Algorithm
<pre>1. # 导入 PyTorch 库 2. 导入 PyTorch 为 torch 3. # 从指定路径加载模型参数 4. 加载模型参数为 loaded_paras 从 './pytorch_model.bin' 5. # 打印加载的模型参数的类型 6. 打印 loaded_paras 的类型 7. # 如果 loaded_paras 是一个字典（通常 PyTorch 保存的模型参数是字典） 8. # 打印字典中键的数量 9. 如果 loaded_paras 是一个字典: 10. 打印 loaded_paras 字典中键的数量 11. # 打印 loaded_paras 字典中所有的键 12. 打印 loaded_paras 字典中的键</pre>

执行完上述算法后，便可以得到如下输出结果：

```
<class 'collections.OrderedDict'>
207
['BERT.embeddings.word_embeddings.weight',
```



```
'BERT.embeddings.position_embeddings.weight',
'BERT.embeddings.token_type_embeddings.weight',
.....
'BERT.encoder.layer.11.output.dense.bias',
'BERT.encoder.layer.11.output.LayerNorm.gamma',
'BERT.encoder.layer.11.output.LayerNorm.beta',
.....]
```

从上面的输出结果可以看到，参数 `pytorch_model.bin` 被载入后变成了一个有序的字典 `OrderedDict`，并且其中一共有 207 个参数，其名字分别就是列表中的各个元素。进一步，还可以将各个参数的形状打印出来，如表 4-9 所示：

表 4-9 打印参数形状

Algorithm

1. // 假设 `loaded_paras` 是一个字典，包含了所有加载的参数
2. // 以下是根据提供的输出转换成的伪代码表示
3. // 定义 BERT 嵌入层的词嵌入权重
4. `WORD_EMBEDDINGS_WEIGHT = Matrix(21128, 768)` // 形状: [21128, 768]
5. # 这是 BERT 的词嵌入层权重，包含 21128 个词汇的 768 维向量
6. // 定义 BERT 嵌入层的位置嵌入权重
7. `POSITION_EMBEDDINGS_WEIGHT = Matrix(512, 768)` // 形状: [512, 768]
8. # 这是 BERT 的位置嵌入层权重，用于表示句子中每个词的位置信息
9. // 定义 BERT 嵌入层的类型嵌入权重
10. `TOKEN_TYPE_EMBEDDINGS_WEIGHT = Matrix(2, 768)` // 形状: [2, 768]
11. # 这是 BERT 的类型嵌入层权重，用于区分句子中的不同部分（如问题和答案）
12. // 定义 BERT 嵌入层的 LayerNorm 层的 `gamma` 参数
13. `LAYER_NORM_GAMMA = Vector(768)` // 形状: [768]
14. # 这是 BERT 嵌入层 LayerNorm 的 `gamma` 参数，用于调整归一化过程中的缩放比例
15. // ...（这里省略了其他参数的伪代码，因为它们的转换方式类似）
16. // 伪代码中的 `Matrix` 和 `Vector` 是假设的数据类型，用于表示二维矩阵和一维向量

同样，还可以直接打印出某个参数具体的值。不过想要将它迁移到自己所搭建的模型上还要进一步的来分析自己所搭建的模型。

4.2.2 载入模型配置

在进行下一步之前，需要载入本地的 BERT 配置参数 `config.json` 这个文件。先定义一个配置类，然后将这些参数从 `config.json` 载入后来实例化这个配置类，过程如表 4-10 所示：

表 4-10 载入模型配置

Algorithm
1. 定义一个类 <code>BERTConfig</code>: 2. <code>"""BERT 模型的配置类"""</code> 3. 构造函数 <code>BERTConfig(__init__)</code>: 4. 初始化参数: 5. <code>vocab_size = 21128</code> 6. <code>hidden_size = 768</code> 7. <code>num_hidden_layers = 12</code> 8. ... (其他参数省略) 9. 将传入的参数值分别赋值给类的属性 10. 类方法 <code>from_dict(cls, json_object)</code>: 11. <code>"""从 Python 字典中构造 BERTConfig 对象"""</code> 12. 创建一个 <code>BERTConfig</code> 的实例 <code>config</code>，并只传入 <code>vocab_size=None</code> (作为占位符) 13. 遍历 <code>json_object</code> 中的每个键值对(<code>key, value</code>) 14. 将键值对中的值赋给 <code>config</code> 对象的对应属性 15. 返回 <code>config</code> 对象 16. 类方法 <code>from_json_file(cls, json_file)</code>: 17. <code>"""从 JSON 文件中构造 BERTConfig 对象"""</code> 18. 打开 <code>json_file</code> 文件并读取内容 19. 打印日志，表示成功导入 BERT 配置文件 20. 使用 <code>json.loads()</code>将读取的内容转换为 Python 字典 21. 调用 <code>from_dict</code> 方法，将字典转换为 <code>BERTConfig</code> 对象并返回

在上述算法中，第 5-29 行是通过指定参数值来实例化 `BERTConfig` 这个类；第 40 行的 `from_json_file()`则是通过指定的路径来载入本地的 `config.json` 配置文件（如表 4-11 所示），其中的 `cls` 表示未经实例化的 `BERTConfig` 类，是 Python 语法中 `@classmethod` 的用法。

表 4-11 指定路径

Algorithm
1. <code>attention_probs_dropout_prob</code> 设置为 0.1 2. <code>directionality</code> 设置为 "bidi" 3. <code>hidden_act</code> 设置为 "gelu"

Algorithm

4. hidden_dropout_prob 设置为 0.1
5. hidden_size 设置为 768
6. initializer_range 设置为 0.02
7. intermediate_size 设置为 3072
8. ... (其他字段和值, 根据实际需要添加)

定义完成后通过如表 4-12 的方式来载入 config.json 配置文件。

表 4-12 载入配置文件

Algorithm

1. 设置 json_file 为字符串 './BERT_base_chinese/config.json', 表示配置文件的路径
2. 调用一个名为 "加载 JSON 配置文件" 的函数, 传入 json_file 作为参数
3. 将该函数返回的配置对象赋值给变量 config

接下来便可以以访问类成员的方式 config.hidden_size 来使用这些参数。

4.2.3 载入并初始化

前面已经实现了构建 BERT 模型, 接下来需要载入已有参数来初始化网络中的参数。在将本地参数迁移到一个新的模型之前, 除了像 4.1.1 节那样分析本地参数之外, 还需要将网络的参数信息也打印出来, 以便将两者一一对应上, 具体过程如表 4-13 所示:

表 4-13 打印网络信息参数

Algorithm

1. # 定义 BERT 配置文件的路径
2. 设定配置文件路径为 json_file, 其值为 './BERT_base_chinese/config.json'
3. # 从 JSON 文件中加载 BERT 的配置
4. 使用 BERTConfig 的 from_json_file 方法加载 json_file 中的配置, 并赋值给 config
5. # 使用加载的配置初始化 BERT 模型
6. 根据 config 创建一个新的 BERTModel 实例, 并赋值给 BERT_model
7. # 打印 BERT 模型参数的相关信息
8. 打印消息 "\n ===== BERTModel 参数: ====="
9. # 获取 BERT 模型的状态字典 (包含所有权重和偏置) 的长度
10. 计算 BERT_model 状态字典的长度, 并打印
11. # 遍历 BERT 模型的状态字典中的每一个参数张量
12. 对于 BERT_model 状态字典中的每一个参数张量 param_tensor:
13. # 打印参数张量的名称和大小
14. 打印 param_tensor

Algorithm

15. 打印 制表符 "\t"
16. 打印 BERT_model 状态字典中 param_tensor 对应张量的大小

在执行完上述算法后，便可以得到如下输出结果：

```

1  ===== BERTMolde 参数: =====
2 200
3 ### BERT_embeddings.position_ids                torch.Size([1, 512])
4 ### BERT_embeddings.word_embeddings.embedding.weight  torch.Size([21128, 768])
5 ### BERT_embeddings.position_embeddings.embedding.weight  torch.Size([512, 768])
6 ### BERT_embeddings.token_type_embeddings.embedding.weight  torch.Size([2, 768])
7 .....
8 ### BERT_encoder.BERT_layers.11.BERT_output.dense.weight  torch.Size([768, 3072])
9 ### BERT_encoder.BERT_layers.11.BERT_output.dense.bias    torch.Size([768])
10 ### BERT_encoder.BERT_layers.11.BERT_output.LayerNorm.weight  torch.Size([768])
11 ### BERT_encoder.BERT_layers.11.BERT_output.LayerNorm.bias    torch.Size([768])
12 ### BERT_pooler.dense.weight                            torch.Size([768, 768])
13 ### BERT_pooler.dense.bias                              torch.Size([768])

```

从上面的输出结果可以发现，BERTMolde 一共有 200 个参数，而 BERT-base-chinese 一共有 207 个参数。这里需要注意的是 BERTMolde 模型中的 position_ids 这个参数并不是模型中需要训练的参数，只是一个默认的初始值。最后，经分析（两者一一进行对比）后发现 BERT-base-chinese 中除了最后的 8 个参数以外，其余的 199 个参数和 BERTMolde 模型中的 199 个参数一样且顺序也一样。

最后通过在 BERTMolde 类（BERT.py 文件中）中再加入一个如表 4-14 所示的函数来用 BERT-base-chinese 中的参数初始化 BERTMolde 中的参数**[2]**：

表 4-14 初始化 BERTModle 参数

Algorithm

1. 类方法 from_pretrained(cls, config, pretrained_model_dir=None):
2. """从预训练模型中加载参数并初始化模型"""
3. # 使用给定的配置初始化模型
4. model = 使用配置 cls(config)初始化模型
5. # 构建预训练模型参数文件的路径

Algorithm

```
6. pretrained_model_path = 在 pretrained_model_dir 下找到"pytorch_model.bin"文件
7. # 加载预训练模型的参数
8. loaded_paras = 使用 torch.load 从 pretrained_model_path 加载参数
9. # 复制当前模型的参数状态字典
10. state_dict = 复制(model.state_dict())
11. # 获取加载的参数名列表，忽略最后 8 个（可能是特殊层或不匹配的）
12. loaded_paras_names = 加载参数键名列表[:-8]
13. # 获取模型参数名列表，忽略第一个（可能是特殊键，如'module.'前缀）
14. model_paras_names = 模型参数键名列表[1:]
15. # 遍历并更新模型的参数状态字典
16. 对于 loaded_paras_names 中的每个参数名:
17. state_dict 中对应的模型参数名 = loaded_paras 中对应的参数值
18. 记录日志：成功将参数 loaded_paras_names[i]赋值给 model_paras_names[i]
19. # 将更新后的状态字典加载到模型中
20. model.加载状态字典(state_dict)
21. # 返回加载了预训练参数的模型
22. 返回 model
```

在上述算法中，第 4-5 行用来载入本地的 BERT-base-chinese 参数；第 6 行用来拷贝一份 BERTModel 中的网络参数，这是因为无法直接修改里面的值；第 7-10 行则是根据上面的分析，将 BERT-base-chinese 中的参数赋值到 state_dict 中；第 12 行是用 state_dict 中的参数来初始化 BERTModel 中的参数。

最后，通过如下方式便可以返回一个通过 BERT-base-chinese 初始化的 BERT 模型：BERT = BERTModel.from_pretrained(config, BERT_pretrained_model_dir)。当然，如果需要冻结其中某些层的参数不参与模型训练，那么可以通过类似如表 4-15 所示的代码来进行设置：

表 4-15 设置冻结参数

Algorithm

```
1. # 遍历 BERT 模型中的所有参数
2. for para in BERT_model.parameters():
3. # 假设条件：检查参数名称是否包含特定的字符串，比如'embedding'
4. if 'embedding' in para.name: # 这里'xxxx'被替换为具体的条件
5. # 如果条件满足，将该参数的 requires_grad 设置为 False
6. # 这意味着在反向传播时，这个参数将不会被更新
7. para.requires_grad = False
```

到此，整个预训练模型的加载过程也告一段落，接下来正式进入到基于 BERT 预训练模型的文本分类场景中。

4.3 基于前向传播的文本分类

4.3.1 项目结构

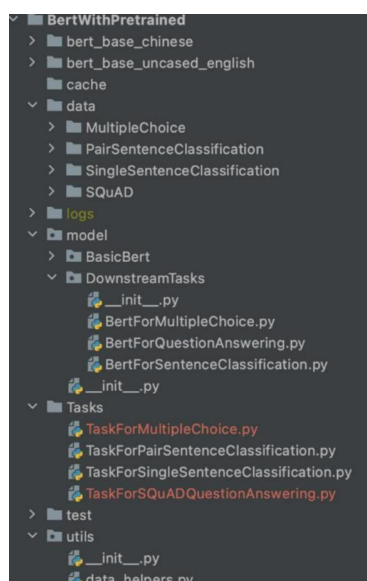


图 4-1 项目目录结构图

图 4-1 完整的展示了此次的项目结构。BERT_base_chinese 目录里面是原始中文 BERT 的配置文件和模型参数；cache 目录用来保存微调后保存的模型；data 里面存放了各类下游任务需要用到的数据集；logs 目录用来保存日志；model 目录里面是实现的模型代码：BasicBERT 目录里面是实现整个 BERT 模型，包括加载预训练参数所用到的代码；DownstreamTasks 目录里面实现的是各类基于 BERT 结构所实现的模型代码，例如 BERTForSentenceClassification.py 中实现的便是基于 BERT 的文本分类模型，后续还会在同级目录中来新建其它场景下基于 BERT 的模型代码；Tasks 目录里面实现的则是 model 目录中对应下游场景任务的训练代码，例如 TaskForSingleSentenceClassification.py 实现的则是基于单文本进行分类的训练代码，后续也会添加其它场景下的训练代码；test 目录中是各个类的测试文件；utils 目录中是一些工具类文件，例如数据集载入和日志打印工具等。

4.3.2 文本分类前向传播

在将 BERT-base-chinese 中的参数赋值到 BERT 模型中之后，接下来先要做的就是实现文本分类的前向传播过程。在图 4-1 中的 BERTForSentenceClassification.py 文件中，通过定义如表 4-16 表 4-16 的一个类来完成整个前向传播的过程：

表 4-16 前向传播初始化

Algorithm
1. 定义类 BERTForSentenceClassification 继承自 nn.Module: 2. 初始化函数 <code>__init__(self, config, BERT_pretrained_model_dir=None)</code>: 3. 调用父类的初始化函数 4. 设置分类标签数量: <code>self.num_labels = config.num_labels</code> 5. 如果预训练模型目录存在: 6. 使用预训练目录加载 BERT 模型: <code>self.BERT = BERTModel.from_pretrained(config, BERT_pretrained_model_dir)</code> 7. 否则: 8. 使用配置初始化 BERT 模型: <code>self.BERT = BERTModel(config)</code> 9. 设置 dropout 层: <code>self.dropout = nn.Dropout(config.hidden_dropout_prob)</code> 10. 设置分类器层: <code>self.classifier = nn.Linear(config.hidden_size, self.num_labels)</code>

在上述算法中，第 4 行分别就是用来指定模型配置、分类的标签数量以及预训练模型的路径；第 7-10 行则是用来定义一个 BERT 模型，可以看到如果预训练模型的路径存在则会返回一个由 BERT-base-chinese 参数初始化后的 BERT 模型，否则则会返回一个随机初始化参数的 BERT 模型；第 12 行则是定义最后的分类层。最后，整个前向传播的实现如表 4-17 所示：

表 4-17 文本分类前向传播过程

Algorithm
1. 在 BERTForSentenceClassification 类中定义前向传播函数 <code>forward</code>: 2. 输入: 3. <code>input_ids</code>: 输入的词元 ID, 形状为 <code>[src_len, batch_size]</code> 4. <code>attention_mask</code>: 注意力掩码, 形状为 <code>[batch_size, src_len]</code>, 可选 5. <code>token_type_ids</code>: 分段类型 ID, 形状为 <code>[src_len, batch_size]</code>, 单句分类时为 None, 可选 6. <code>position_ids</code>: 位置 ID, 形状为 <code>[1, src_len]</code>, 可选 7. <code>labels</code>: 真实标签, 形状为 <code>[batch_size,]</code>, 可选 (用于计算损失) 8. 输出: 9. 如果提供了 <code>labels</code>, 则输出损失和 logits 10. 否则, 仅输出 logits 11. 过程:

Algorithm

12. 调用 BERT 模型的前向传播, 获取 `pooled_output` (池化后的输出) 和其他输出 (这里不关心其他输出)
13. 将 `pooled_output` 通过 dropout 层
14. 将 dropout 后的 `pooled_output` 通过分类器层, 得到 logits
15. 如果提供了 labels:
16. 初始化交叉熵损失函数
17. 计算损失
18. 返回损失和 logits
19. 如果没有提供 labels:
20. 仅返回 logits

在上述算法中, 第 6-9 行返回的就是原始 BERT 网络的输出, 其中 `pooled_output` 为 BERT 第一个位置的向量经过一个全连接层后的结果, 第二个参数是 BERT 中所有位置的向量; 第 10-11 行便是用来进行文本分类的分类层; 第 12-17 行则是用来判断返回损失值还是返回 logits 值。

4.3.3 模型训练

在图4-1中, 在Task目录下新建一个名为TaskForSingleSentenceClassification的模块来完成分类模型的微调训练任务。

首先, 需要定义一个ModelConfig类来对分类模型中的超参数进行管理, 过程如表4-18所示:

表 4-18 模型训练初始化

Algorithm

- 1 定义一个类 ModelConfig:
- 2 初始化方法 `__init__(self)`:
- 3 设置项目目录 `self.project_dir` 为当前文件的上级目录的上级目录
- 4 设置数据集目录 `self.dataset_dir` 为项目目录下的 'data/SingleSentenceClassification'
- 5 设置预训练模型目录 `self.pretrained_model_dir` 为项目目录下的 'BERT_base_chinese'
- 6 设置词汇表路径 `self.vocab_path` 为预训练模型目录下的 'vocab.txt'
- 7 设置设备 `self.device` 为可用的 CUDA 设备 (如果有) 或 CPU
- 8 设置训练文件路径 `self.train_file_path` 为数据集目录下的 'toutiao_train.txt'
- 9 设置验证文件路径 `self.val_file_path` 为数据集目录下的 'toutiao_val.txt'
- 10 设置测试文件路径 `self.test_file_path` 为数据集目录下的 'toutiao_test.txt'
- 11 设置模型保存目录 `self.model_save_dir` 为项目目录下的 'cache'
- 12 设置日志保存目录 `self.logs_save_dir` 为项目目录下的 'logs'

Algorithm

```
13  设置分隔符 self.split_sep 为 '!'_'  
14  设置是否打乱样本顺序 self.is_sample_shuffle 为 True  
15  设置批处理大小 self.batch_size 为 64  
16  设置最大句子长度 self.max_sen_len 为 None  
17  设置标签数量 self.num_labels 为 15  
18  设置训练轮数 self.epochs 为 10  
19  设置每轮验证次数 self.model_val_per_epoch 为 2  
20  调用 logger_init 函数初始化日志, 设置日志文件名、日志级别和日志目录  
21  检查模型保存目录 self.model_save_dir 是否存在, 如果不存在则创建  
22  获取 BERT 配置文件的路径 self.BERT_config_path 为预训练模型目录下的  
    'config.json'  
23  从 BERT 配置文件加载配置 self.BERT_config  
24  遍历 BERT 配置的所有键值对, 并复制到当前 ModelConfig 对象的属性中  
25  打印一条日志消息, 表示开始打印当前配置  
26  遍历当前 ModelConfig 对象的所有键值对, 并打印到日志中
```

在上述算法中, 第 2-16 行则是分别用来定义模型中的一些数据集目录、超参数和初始化日志打印类等; 第 18-22 行则是将原始 BERT_base_chinese 配置文件, 即 config.json 中的参数也导入到类 ModelConfig 中; 第 24-26 行则是将所有的超参数配置情况一同打印到日志文件中方便后续分析。

表 4-19 模型训练具体过程

Algorithm

```
1.  def train(config):  
2.  # 初始化 BERT 句子分类模型  
3.  model = 初始化 BERT 句子分类模型(config, 预训练模型目录)  
4.  # 设置优化器  
5.  优化器 = 设置 Adam 优化器(model 的参数, 学习率=5e-5)  
6.  # 将模型设置为训练模式  
7.  model.train()  
8.  # 加载 BERT 分词器  
9.  分词器 = 加载 BERT 分词器(预训练模型目录)  
10. # 初始化数据集加载器  
11. 数据加载器 = 初始化单句分类数据集加载器(词汇表路径, 分词器, 批量大小, 最大句子  
    长度, 分隔符, 最大位置嵌入, 填充索引)  
12. # 加载训练、验证和测试数据迭代器  
13. 训练迭代器, 测试迭代器, 验证迭代器 = 数据加载器.加载训练验证测试数据(训练文件  
    路径, 验证文件路径, 测试文件路径)  
14. # 初始化最大准确率  
15. 最大准确率 = 0
```

Algorithm

```
16. # 开始训练
17. 对于每个训练周期从 1 到总周期数:
18. 损失总和 = 0
19. # 开始遍历训练数据
20. 对于每个训练样本和标签在训练迭代器中:
21. # 将数据移到设备 (如 GPU)
22. 样本 = 样本.移到设备(config 指定的设备)
23. 标签 = 标签.移到设备(config 指定的设备)
24. # 计算 padding mask
25. padding_mask = 计算 padding mask(样本, 填充索引)
26. # 前向传播, 计算损失和预测
27. 损失, 预测 = model(
28. 输入 ID=样本,
29. 注意力掩码=padding_mask,
30. 其他参数=None
31. )
32. # 计算准确率
33. 当前准确率 = 计算准确率(预测, 标签)
34. # 省略反向传播和优化步骤
35. # 每隔一定周期评估模型
36. 如果当前周期是保存周期的倍数:
37. 验证准确率 = 在验证集上评估模型(验证迭代器, model, 设备)
38. 记录日志(f'验证集准确率: {验证准确率:.3f}')
39. # 省略保存模型等步骤
```

模型训练具体过程如表 4-19 所示。在上述算法中，第 2-3 行用来初始化一个基于 BERT 的文本分类模型；第 9-19 行则是载入相应的数据集；第 20-39 行则是整个模型的训练过程。如下是网络的训练结果：

```
1 -- INFO: Epoch: 0, Batch[0/4186], Train loss :2.862, Train acc: 0.125
2 -- INFO: Epoch: 0, Batch[10/4186], Train loss :2.084, Train acc: 0.562
3 -- INFO: Epoch: 0, Batch[20/4186], Train loss :1.136, Train acc: 0.812
4 -- INFO: Epoch: 0, Batch[30/4186], Train loss :1.000, Train acc: 0.734
5 ...
6 -- INFO: Epoch: 0, Batch[4180/4186], Train loss :0.418, Train acc: 0.875
7 -- INFO: Epoch: 0, Train loss: 0.481, Epoch time = 1123.244s
8 ...
9 -- INFO: Epoch: 9, Batch[4180/4186], Train loss :0.102, Train acc: 0.984
```

```
10 -- INFO: Epoch: 9, Train loss: 0.100, Epoch time = 1130.071s
```

```
11 -- INFO: Accurcay on val 0.884
```

```
12 -- INFO: Accurcay on test 0.888
```

4.4 本章小结

在本章中，首先从总体上介绍了建立基于 BERT 预训练模型的文本分类模型的大致思路；然后介绍了模型的输入部分，以及如何从零开始来构建数据集；接着详细介绍了如何分析模型的参数，并将其载入到相应的模型中；最后介绍了如何在 BERT 网络模型的基础之上添加一个分类层，来完成最后的文本分类任务。

5 基于 RAG 和大模型技术的医疗智能问答系统

本章设计一个基于 RAG 与大模型技术的医疗问答系统，利用 MdeicalKG 数据集与 Neo4j 构建知识图谱，结合 BERT 的命名实体识别和 34b 大模型的意图识别，通过精确的知识检索和问答生成，提升系统在医疗咨询中的性能，解决大模型在医疗领域应用的可靠性问题。

5.1 系统架构与工作流程

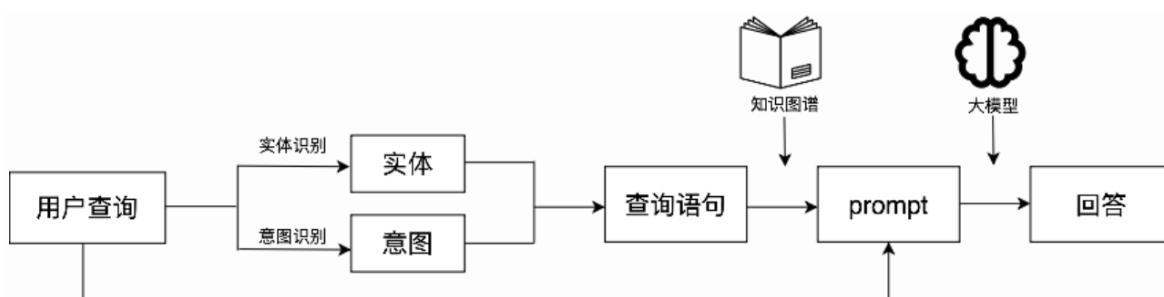


图 5-1 整体流程

图 5-1 展示了一个医疗智能问答系统的核心工作流程，从用户提出查询到最终生成回答，每个步骤都扮演着重要的角色。

(1) 用户查询：这是整个流程的起始点，用户通过界面输入他们的查询需求。查询的准确性和清晰度对后续步骤的成功至关重要，因为它们直接影响了系统能否准确理解和满足用户的需求。

(2) 实体识别：利用自然语言处理技术从查询文本中识别出具体实体，如人名、地名等。这有助于系统理解查询的上下文，并为后续步骤提供关键信息。

(3) 意图识别：旨在理解用户查询的真正目的，如寻求答案或购买渠道。准确识别用户意图能确保系统提供更符合用户需求的回答。

(4) 查询语句生成：基于实体识别和意图识别的结果，生成更精确、具体的查询语句。优化和重构的查询语句能更有效地进行查询，提高搜索效率。

(5) 知识图谱查询：使用生成的查询语句在知识图谱中搜索相关信息和数据。知识图谱提供丰富的背景信息和数据支持，有助于找到与用户查询相关的各种信息。

(6) 大模型处理：利用大模型的自然语言处理能力和深度学习算法深入理解和分析用户查询。生成更准确的提示，反映模型对查询的理解，并可能包含对查询结果的初步预测或建议。

(7) 回答生成：综合知识图谱查询结果和大模型生成的提示，生成最终回答。回答包含用户所需的关键信息，并经过组织和优化，易于理解和接受。

5.2 知识数据获取

5.2.1 数据来源

项目从国内可靠的医学数据网站——寻医问药网(<http://www.xywy.com/>)上爬取了大量的医学数据用于构建本系统的知识图谱。图 5-2 展示了部分爬取的典型数据。

```
{ "id": { "$oid": "5bb578b6831b973a137e3ee6" }, "name": "肺泡蛋白质沉积症", "desc": "肺泡蛋白质沉积症(简称PAP), 又称Rosen-Castle-man-Liebow综合征, 是一种罕见疾病。该病以肺泡和细支气管腔内充满PAS染色阳性, 来自肺的富磷脂蛋白质物质为其特征, 好发于青中年, 男性发病约3倍于女性。", "category": [ "疾病百科", "内科", "呼吸内科" ], "prevent": "1、避免感染支杆菌病, 卡氏肺囊肺炎, 巨细胞病毒等。\\n2、注意锻炼身体, 提高免疫力。", "cause": "病因未明, 推测与几方面因素有关: 如大量粉尘吸入(铝, 二氧化硅等), 机体免疫功能下降(尤其婴幼儿), 遗传因素, 酗酒, 微生物感染等, 而对于感染, 有时很难确认是原发致病因素还是继发于肺泡蛋白沉着症, 例如巨细胞病毒, 卡氏肺孢子虫, 组织胞浆菌感染等均发现有肺泡内高蛋白沉着。\\n虽然启动因素尚不明确, 但基本上同意发病过程为脂质代谢障碍所致, 即由于机体内, 外因素作用引起肺泡表面活性物质的代谢异常, 到目前为止, 研究较多的有肺泡巨噬细胞活力, 动物实验证明巨噬细胞吞噬粉尘后其活力明显下降, 而病员灌洗液中的巨噬细胞内颗粒可使正常细胞活力下降, 经支气管肺泡灌洗治疗后, 其肺泡巨噬细胞活力可上升, 而研究未发现II型细胞生成蛋白增加, 全身脂代谢也无异常, 因此目前一般认为本病与清除能力下降有关。", "symptom": [ "紫绀", "胸痛", "呼吸困难", "乏力", "臃肿" ], "yibao status": "否", "get prob": "0.00002%", "get way": "无传染性", "acompany": [ "多重肺部感染" ], "cure department": [ "内科", "呼吸内科" ], "cure way": [ "支气管肺泡灌洗" ], "cure lasttime": "约3个月", "cured prob": "约40%", "cost money": "根据不同医院, 收费标准不一致, 省市三甲医院约( 8000——15000 元)", "check": [ "胸部CT检查", "肺活检", "支气管镜检查" ], "recommand drug": [], "drug detail": [] }
```

图 5-2 医学领域数据集

5.2.2 网页数据抓取及处理

网页数据抓取及处理大致有以下六个方面的内容：

第一，规划数据需求与合规性检查。在进行寻医问药网的网页数据抓取之前，首要任务是详细规划数据需求，明确抓取目标，这包括药品的名称、成分、价格、使用方法、副作用等信息，医生的专业领域、咨询记录、用户对医生的评价，以及用户对于药品或医学服务的反馈和评分等。明确目标后，需检查寻医问药网的 robots.txt 文件，以确认网站是否允许爬虫抓取，并仔细阅读服务条款，确保抓取行为不违反网站使用规定或侵犯版权。

第二，选择合适的技术栈。Python 作为一种广泛用于网络爬虫的编程语言，拥有丰富的库支持，如 requests 库可用于发送 HTTP 请求并获取网页内容，而 BeautifulSoup 和 lxml 库则便于解析 HTML 和 XML 文档，以提取所需数据。值得注意的是，若网站数据为动态加载，可能还需使用 Selenium 库模拟浏览器行为，以获取 JavaScript 生成的内容。

第三，编写抓取脚本。技术栈选定后，即可开始编写抓取脚本。这包括设置请求头信息，如 User-Agent，以模拟浏览器访问并降低被封禁风险。编写解析函数时，需根据网页结构定位数据所在的 HTML 元素，并提取相应的文本或属性值。

第四，异常处理与多页数据抓取。脚本中应加入异常处理机制，以应对网络请求失败、解析错误等问题，确保程序的健壮性。考虑到网站可能包含大量分布在不同页面上的数据，脚本还需实现多页数据的连续抓取，这可能涉及处理分页链接或滚动加载的情况。

第五，数据保存与格式选择。数据抓取完成后，根据研究需求，选择将数据保存为 JSON 格式，以便于后续的数据分析和处理。

第六，遵守法律法规与道德标准。在整个抓取过程中，必须时刻注意遵守法律法规，尊重数据隐私和版权，避免进行高频率访问，以免对寻医问药网的正常运行造成影响。同时，需根据网站的反爬虫机制调整抓取策略，确保抓取行为不会对网站造成过大的访问压力，并符合网站使用规定和道德标准。

最终，成功获得了 7 类实体和 5 类关系，如表 5-1 所示。以疾病实体为例，它包含了 14 种属性，使其成为整个知识图谱中关系最为复杂和丰富的中心实体。具体而言，构建的知识图谱包含了 30402 个节点和 206873 对关系。

表 5-1 实体、属性、关系信息表

类型	包含内容
实体	疾病、药物、症状、菜谱、检查项目、科室、药物厂商
属性	名描述、病因、预防措施、诊断科室、治疗方式、治愈时间、治愈率、费用、易感人群、发病率、推荐饮食、饮食禁忌、药物功能、制作方式、使用方法、单价
关系	并发疾病、相关症状、检查项目、推荐食谱、推荐药物

在表 5-1 中，实体部分列举了本次医学智能问答系统的实体类型，包括疾病、药物、症状、菜谱、检查项目、科室和药物厂商等。这些实体是医学和健康信息中的基础单位，例如“感冒”是一种疾病，“阿司匹林”是一种药物，“头痛”是一种症状等。属性部分描述了与上述实体相关的各种属性。这些属性提供了关于实体的详细信息，例如“名描述”可能描述了实体的名称和简短介绍，“病因”解释了疾病发生的原因，“预防措施”提供了避免疾病的方法，“诊断科室”指出了进行诊断的科室，“治疗方式”描述了治疗疾病的方法等。关系部分定义了实体之间的关联。这些关系描述了不同实体之间的相互作用或联系，例如“并发疾病”表示一种疾病可能与其他疾病同时发生，“相关症状”描述了疾病与症状之间的关联，“检查项目”列出了用于诊断疾病的检查方法，“推荐食谱”和“推荐药物”则分别提供了与疾病或症状相关的饮食和药物建议。各实体类型的中文含义、实体数量如表 5-2 所示：

表 5-2 实体类型数据信息

实体类型	中文含义	实体数量	举例
Check	检查名称	3400+	血清胆汁酸测定（TBA）、蔗糖水溶血试验、
Department	科室名称	60+	血液科、中医综合、精神科、妇产科、外科
Disease	疾病名称	7900+	肺泡蛋白质沉积症(简称 PAP)、百日咳
Drug	医药名称	4100+	除疤膏、复方乌鳢口服液、阿莫西林胶囊
Food	食物名称	5100+	烧鹅饭、黄焖鸡米饭、锅盔、东北饺子
Producer	厂商名称	21000+	长春迪瑞维生素 C 咀嚼片
Symptom	症状名称	约 7000	产褥期躁狂状态
Total	总计数量	45000+	约 4.5 万实体量级

5.3 基于 Neo4j 的知识图谱可视化

知识图谱是一种结构化的语义知识库，它以图的形式表示实体及其相互关系。Neo4j 作为一种流行的图数据库管理系统，因其高效的图处理能力和灵活的查询语言（Cypher）而广泛用于知识图谱的存储和可视化。以下是利用 Neo4j 进行知识图谱存储和可视化的详细描述。

知识图谱通常包含大量的实体（如人、地点、组织）和它们之间的关系。在 Neo4j 中，这些实体和关系分别被存储为节点（Node）和边（Relationship）每个节点代表一个实体，可以包含多个属性（如名称、描述）。例如，一个代表“苹果公司”的节点可以有属性如 `name="Apple Inc."`和 `founded="1976"`。

关系连接两个节点，表示它们之间的某种联系，如“位于”、“拥有”或“创立”。每个关系都有一个类型和可能的属性。例如，苹果公司和史蒂夫·乔布斯之间可以有一个“FOUNDED_BY”（由...创立）的关系。为了提高查询效率，可以在节点的关键属性上创建索引，比如在“name”或“id”字段上。

Neo4j 提供了内置的浏览器工具，允许用户直观地查看图形数据。这个可视化工具非常有用，特别是在探索复杂关系和模式时。通过 Neo4j 浏览器，用户可以执行 Cypher 查询，并以图形方式查看结果。节点和关系会以不同的颜色和大小显示，便于区分不同类型的实体和关系。用户可以点击节点和关系，查看其属性。这对于深入了解图形结构中的特定部分非常有用。可以根据需要调整节点和关系的颜色、大小和形状，以更好地展示知识图谱的特定方面。例如，可以将不同类型的实体用不同的颜色表示。搭建成功后的医学领域的图数据库效果如图 5-3 所示。

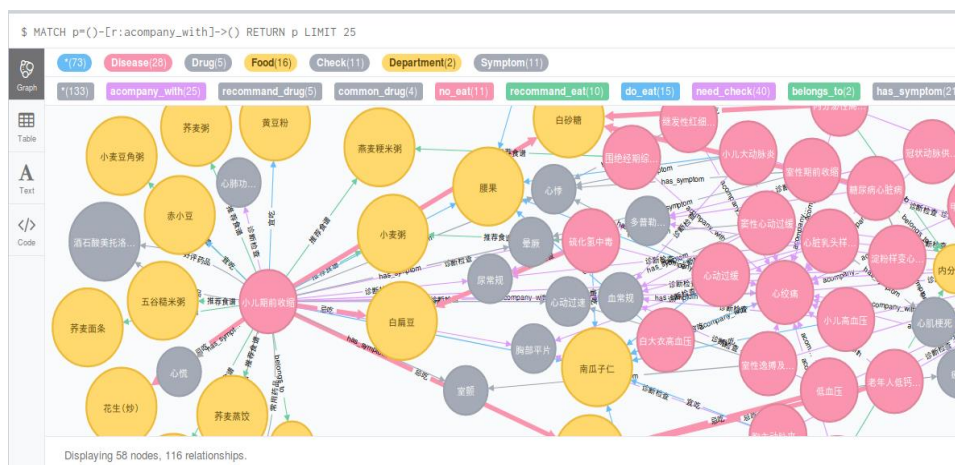


图 5-3 Neo4j 中部分医疗数据节点

5.4 基于 34b 大模型的意图识别

关于命名实体识别模型的研究，已在第三章详尽阐述，此处不再重复。本节将重点介绍本研究的另一核心内容——基于大模型的意图识别。在智能问答系统中，运用

意图识别模型进行用户意图的准确识别和问句的有效分类，是至关重要的环节，如图 5-4 所示。

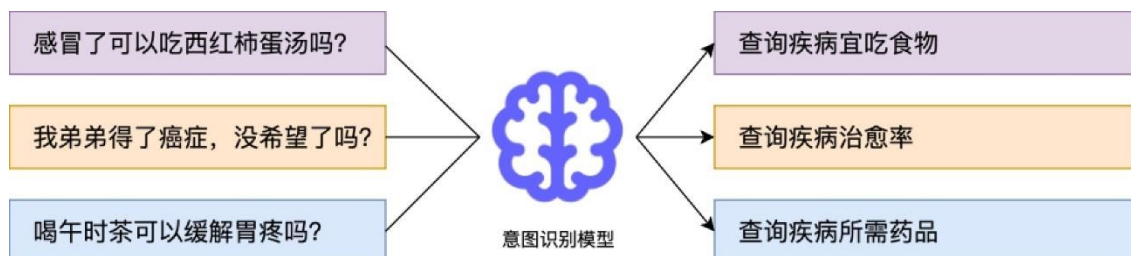


图 5-4 意图识别

5.4.1 基于 34B 的意图识别

基于 34B（340 亿参数）大模型的意图识别是指利用这种大型的自然语言处理模型来识别和理解用户输入的意图。这种大模型通常具有强大的文本生成和理解能力，因此非常适合用于复杂的意图识别任务。以下是基于 34B 大模型进行意图识别的一般步骤：

（1）数据准备：收集并整理包含各种用户意图的历史对话数据或查询数据；对数据进行标注，明确每个数据点的意图类别。

（2）模型微调（Fine-tuning）：使用标注好的数据集对 34B 大模型进行微调，使其能够适应特定的意图识别任务。微调过程中，模型会学习到如何从用户输入中提取关键信息，并将其映射到相应的意图类别上。

（3）输入处理：当新用户输入到来时，首先对其进行必要的预处理，如分词、去除无关字符等；将处理后的输入文本转换为模型可以理解的格式。

（4）意图推断：将处理后的输入文本输入到微调后的 34B 大模型中。模型会基于其学习到的知识和模式，对用户输入的意图进行分析和推断。

（5）输出解析：模型会输出一个或多个意图类别，以及可能的相关参数或实体信息。根据模型的输出，解析出用户的具体意图，以便后续的操作或响应。

（6）持续优化：收集用户对意图识别结果的反馈，用于不断优化和改进模型。可以通过定期更新模型、增加新的训练数据或调整模型参数等方式来提升意图识别的准确性和效率。

基于 34B 大模型的意图识别具有以下几个优势：第一，强大的理解能力：34B 大模型具有出色的文本理解和生成能力，能够更准确地捕捉用户输入的语义和上下文信息；第二，高度的灵活性：大模型可以处理更加复杂和多样的用户输入，包括不同领域、不同风格的语言表达；可扩展性：随着数据的不断积累和模型的持续更新，意图识别的性能可以不断提升和改进。

然而，使用 34B 大模型进行意图识别也面临一些挑战，如计算资源需求高、训练时间长等。因此，在实际应用中需要综合考虑性能、成本和效率等因素。

5.4.2 意图识别策略

表 5-3 三种意图识别策略

策略	准确性	多意图识别	人工成本	推理速度	资源消耗
规则匹配	低	x	低	快	低
训练模型	高	x	高(数据标注)	中等	中等
提示工程	高	✓	低	慢	高

此次研究对比了三种意图识别策略，如表 5-3 所示。

（1）规则匹配

准确性：规则匹配方法的准确性相对较低，因为它依赖于预设的规则，而这些规则可能无法覆盖所有情况，尤其是在面对复杂或模糊的查询时；多意图识别：此策略不支持多意图识别，这在处理用户查询时可能是一个限制因素；人工成本：虽然规则匹配方法的初次设置成本可能较低，但随着规则和例外的增加，维护成本可能会上升；推理速度：由于规则匹配通常是基于硬编码的逻辑，因此推理速度相对较快；资源消耗：资源消耗较低，因为不需要大量的计算资源来进行复杂的模型推理。

（2）训练模型

准确性：通过训练数据驱动模型通常能够达到较高的准确性，尤其是当训练数据量足够大且多样时；多意图识别：和规则匹配一样，传统的训练模型可能也不支持多意图识别，除非特别设计来处理这种情况；人工成本：数据标注是一个劳动密集且成本高昂的过程，尤其是在需要大规模和高质量数据集的情况下；推理速度：模型推

理速度通常比规则匹配稍慢，但仍在可接受的范围内；资源消耗：训练和使用机器学习模型需要更多的计算资源。

（3）提示工程

准确性：通过精心设计的提示（Prompts），可以引导大模型准确地理解用户查询的意图，从而实现高准确性；多意图识别：提示工程方法可以更灵活地处理多意图识别问题，因为可以通过设计复杂的提示来捕捉和解析多个意图；人工成本：虽然设计有效的提示需要专业知识和经验，但相比数据标注来说，这一过程的成本可能更低；推理速度：由于依赖于大型语言模型，提示工程的推理速度可能相对较慢；资源消耗：运行大型语言模型需要显著的计算资源，因此资源消耗较高。

综合考虑以上因素，最终采用了提示工程的手段，将意图分为 16 种，根据 16 类意图设计 Prompt，让大模型对用户的查询进行意图分析。选择理由主要是基于以下五个方面：第一，高准确性：提示工程能够提供与训练模型相当的准确性，且在某些情况下可能更优；第二，多意图识别能力：这是规则匹配和传统训练模型可能缺乏的关键功能，而提示工程可以较好地处理这一问题；第三，相对较低的人工成本：与数据标注相比，设计提示可能更为高效和经济；第四，可接受的推理速度：尽管推理速度不是最快的，但在许多应用场景中仍然是可接受的；第五，资源投入的长期效益：虽然资源消耗较高，但考虑到大型语言模型在多个任务中的通用性和灵活性，这一投入可能是合理的。

提示工程算法可以分为以下七步：

第一，定义意图集合：首先，系统定义了一个包含 16 个不同意图的集合，这些意图代表了用户可能想要执行的操作或询问的信息类型。

第二，定义提示模板集合：接着，系统定义了一个包含 N 个提示模板的集合。这些模板是预先设计好的文本模式，用于引导用户以特定方式表达他们的意图，或者用于构建与用户查询相结合的完整输入。

第三，生成提示：对于意图集合中的每个意图，系统选择最适合的提示模板，并根据该意图和模板填充具体的信息，生成具体的提示。这些生成的提示被添加到提示集合中。

第四，处理用户查询：当系统接收到用户的查询时，它会遍历提示集合，将用户查询与每个提示相结合，形成完整的输入文本。

第五，输入大模型并获取输出：接着，系统将每个完整的输入文本输入到一个大模型中（如深度学习模型），并获取模型的输出文本。

第六，解析输出并识别意图：系统解析模型的输出文本，判断其是否符合当前提示对应的意图。如果符合，系统返回识别的意图和相关参数（如果有）。

第七，处理无法识别的意图：如果系统无法找到与用户查询匹配的意图，它可能会执行一个默认的处理操作，或者向用户反馈无法识别其意图。

提示工程的具体算法步骤如表 5-4 所示。

表 5-4 提示工程的算法步骤

Algorithm
<pre>1. 定义意图集合 intents = {intent1, intent2, ..., intent16} 2. 定义提示模板集合 prompt_templates = {template1, template2, ..., templateN} 3. // 对于每个意图，生成对应的提示 4. 对于每个 intent 在 intents 中: 5. 选择最适合的 prompt_template 在 prompt_templates 中: 6. 根据 intent 和 prompt_template 填充具体的信息，生成具体的 prompt 7. 将生成的 prompt 添加到 prompts 集合中 8. // 当接收到用户查询时 9. 定义用户查询 user_query 10. 对于每个 prompt 在 prompts 集合中: 11. 将 user_query 与 prompt 结合，形成完整的输入 input_text 12. 将 input_text 输入到大模型中 13. 获取大模型的输出 output_text 14. // 解析输出，判断是否符合当前意图 15. 如果 output_text 符合当前 prompt 对应的 intent: 16. 返回识别的 intent 和相关参数（如果有） 17. // 如果没有匹配的意图，可能需要一个默认的处理或者反馈给用户无法识别其意图 18. 如果没有匹配的 intent: 19. 返回 "无法识别您的意图，请重新表达或提供更多信息。"</pre>

5.5 医学领域的智能问答

5.5.1 问答架构设计与实现

要实现医学领域的智能问答，需要从问句识别、问句类型处理、模型准备、参数初始化、优化和测试等五个方面做好工作：

（1）用户问句识别与处理

首先第一步也是最重要的一步，就是要对用户提出的问句进行识别，主要有两个方面，一是提取实体关键词，例如疾病名称，这涉及到后面在 Neo4j 图数据库中查找什么节点，其次就是识别问句类型，例如 disease—cureway 类型，也就是‘疾病—治疗方式’类型，这涉及到后面在 Neo4j 图数据库中查找节点的什么属性或关系。另一方面，对于前面识别出来的不同的问句类型，可能在后面需要不同的提问方式，因为不同的问句类型可能需要更多的不同的参数才能进行下一步查询。

（2）不同问句类型的处理策略

例如当问句类型是 disease—cureway 类型即‘疾病—治疗方式’类型时，仅仅凭前面的信息还不足以到 Neo4j 图数据库中进行查询，原因在于不同的人群对于某一种疾病的治疗方式可能并不相同，比如成年人和未成年人的治疗方式就会有区别。再例如当用户的问句类型是 symptom—disease 类型时，也即由症状问疾病，也是常见的问句类型。但是这个问句类型其实并不能在数据库中直接找到答案。原因很简单，就是某一个症状可能存在于多个疾病的症状列表中，不能仅仅凭借用户输入的某一个症状就判断出该症状对应什么疾病，这是不符合逻辑的。所以在这种情况下，需要诱导用户继续输入还有什么症状，直到用户不再输入为止。然后对之前输入过的症状对应的疾病集合取交集，返回给用户。

（3）问句处理模型的准备与实现

这一步的实现是需要大量的准备工作的。例如对于用户输入的千奇百怪的问句，系统应该如何对问句进行切割；切割完的句子各部分如何标记，即告诉系统各部分对应于什么标签；然后就是将标签正确的对应到问句的切词上等等。这些都是需要提前构造好模型并进行测试和实现的。

（4）模型参数初始化与训练

具体而言，首先需要对一些参数初始化，例如学习率，加载训练数据的批次，输入数据的维数，结果标签的数量等。然后用类似于网页请求的原理从 session 中读入训练数据，然后双向传播并进行结果拼接，然后输入结果标签，然后定义损失函数与优化函数。然后是训练数据，可对数据进行总共 12 轮迭代。

(5) 训练过程中的优化与测试

在训练过程中，如果 `run()` 函数返回的参数中包含了优化器 (optimizer) 的信息，那么它将继续进行训练过程中的优化步骤，以不断提升模型的性能。相反，如果未返回优化器相关的参数，则 `run()` 函数将自动切换到测试或操作模式，对已经训练好的模型进行验证、评估或其他相关操作。

5.5.2 知识图谱查询与回答

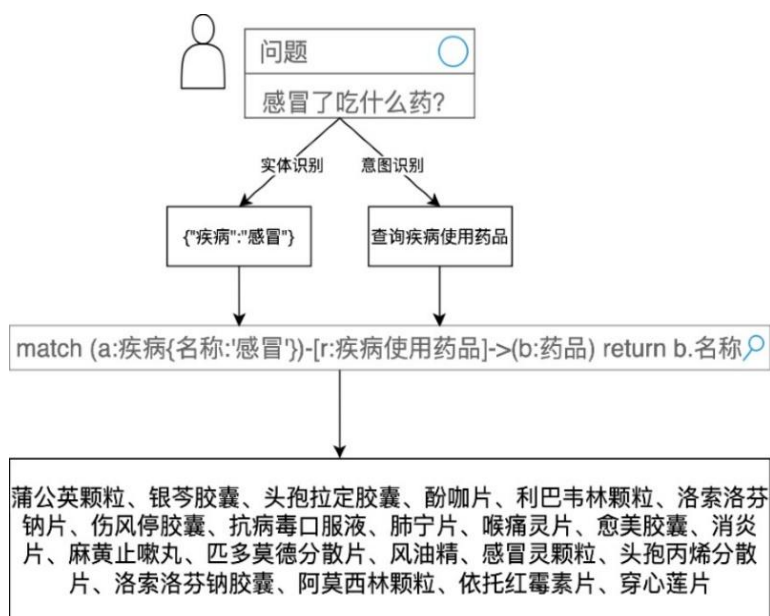


图 5-5 Cypher 查询语句

基于先前获取的问句类型和实体关键词，可以构建相应的 Cypher 查询语句，以便后续在 Neo4j 图数据库中进行查询。这一步骤相对简单，因为已经获得了问句类型和实体关键词，只需遵循 Cypher 语言的语法规则进行构造即可，如图 5-5 所示。

然后就是连接 Neo4j 图数据库，并利用前面已经构造好的 Cypher 语句进行查询。为了提升效果，此时查询得到的结果要按照事先设置好的格式进行输出，也就是将僵硬的机器语言转化为人们平常交流时通常使用的说话方式，使得整个系统更加智能和拟人化。

例如若问题类型为 `disease-lasttime`，即‘疾病-持续时间’类型时，回答模板如表 5-5 疾病-持续时间’类型回答模板算法所示：

表 5-5 疾病-持续时间'类型回答模板算法

Algorithm
<pre>1. 如果问题类型是 'disease_lasttime': 2. # 初始化一个列表来存储所有可能的最后治疗时间 3. 治疗周期列表 = [] 4. # 遍历所有答案 5. 对于 answers 中的每一个元素 i: 6. # 尝试从 i 中获取'n.cure_lasttime'的值（注意：原代码中有语法错误，已修正） 7. 如果 i 包含 'n.cure_lasttime': 8. 治疗周期列表.添加(i['n.cure_lasttime']) 9. # 获取第一个答案中的主体名称（如疾病名称） 10. 主体名称 = answers[0]['m.name'] 11. # 去除治疗周期列表中的重复项，并取前 self.num.limit 个（如果有限制） 12. 去重并限制的治疗周期列表 = 列表(集合(治疗周期列表))[:self.num.limit] 13. # 格式化最终答案 14. # 注意：原代码中的字符串格式化也有错误，已修正 15. 最终答案 = 主体名称 + "治疗可能持续的周期为:" + 字符串连接(去重并限制的治疗周 期列表, 分隔符=', ') 16. # 输出或返回最终答案 17. 输出(最终答案)</pre>

具体的问答效果如图 5-6 所示。

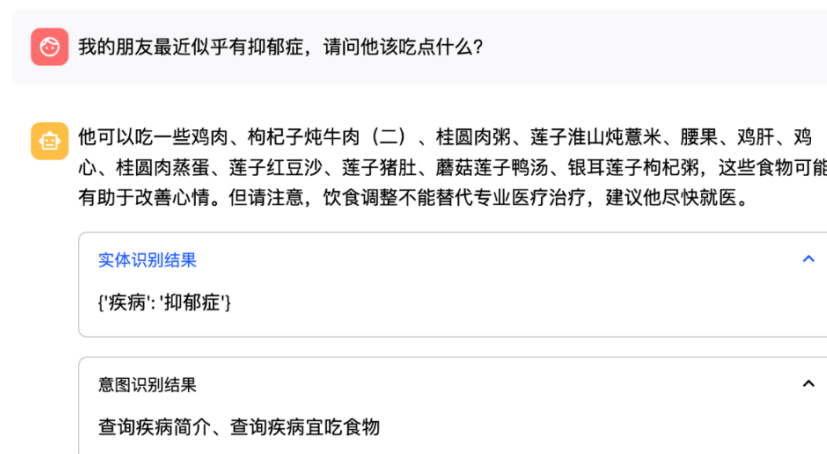


图 5-6 医学领域的智能问答示例

5.6 本章小结

本章主要探讨了基于 RAG 与大模型技术的医疗智能问答系统的设计与实现。该章首先概述了系统的整体运作流程，涉及用户查询、实体识别、意图识别、查询语句

构建、知识图谱查询、大模型处理及回答生成等核心环节，这些环节协同工作，使用户的医疗咨询需求得到有效回应。在数据获取方面，章节详细介绍了如何从权威医学机构抓取数据，并构建了涵盖疾病、药物、症状等医学实体的全面知识图谱，利用 Neo4j 数据库实现高效查询与可视化。此外，通过结合 BERT 与 34B 大模型，系统在实体识别和意图识别方面的性能得到显著提升，确保了用户输入的精准解析。在某市卫检委健康教育科普馆项目的实际应用中，该系统展现出较好的问题解答能力，不仅突破了传统问答系统的限制，还为医疗智能问答提供了新的解决方案。

6 总结及展望

6.1 总结

在如今智能技术不断兴起的时代，传统的问答检索方式已难以满足日益增长的复杂查询需求。传统的问答系统大多依赖于固定的模板或关键词匹配，尽管在处理简单直接的查询时表现出色，但在面对复杂或模糊的问题时却往往力不从心。这种方式的交互往往较为机械和有限，缺乏足够的个性化和上下文理解能力。而智能问答系统的出现，正好弥补了这些不足，以其卓越的处理复杂性、数据处理能力、用户体验和应用范围，展现出显著的优势。对于本文的总结如下：

本文围绕数据预训练、实体识别、意图识别以及医疗问答系统四个方面进行了深入研究与创新。以下是具体总结：

第一，数据预训练阶段的模型创新。在数据预训练阶段，针对句子级处理的准确性问题，提出了一种融合 BERT、LSTM 和 CRF 的先进模型框架。通过整合 BERT 的强大语义表示能力，该框架能够更深入地捕捉句子中的语义信息，进而利用 BERT 计算问句与属性之间的相似度，实现了更为精准的属性选择。

第二，实体识别任务的性能提升。在实体识别方面，通过采用数据增强策略和 TF-IDF 实体对齐技术，显著提高了模型在实体识别任务上的性能。这些技术的应用不仅增强了模型的识别能力，还确保了模型输出与知识图谱之间的高度一致性，为后续的实体匹配和知识推理提供了坚实的基础。针对实体匹配精度不足的问题，提出了采用相似度匹配优化的方法，通过计算输入语句的词组与实体特征词之间的字重叠率、词向量余弦相似度以及 DP 编辑距离，将这三个指标的平均值作为相似度评分，并选择相似度最高且超过设定阈值的实体特征词作为匹配结果。

第三，意图识别的准确性与精细化处理。为了提高意图识别的准确性，引入了 34B 大模型，并通过对比分析不同意图识别方案的优劣，最终选定了提示工程作为首选策略。此外，为了进一步提升识别的精细度，研究还结合了模板匹配方法，通过定义一系列与特定领域相关的模板，对识别出的意图进行了进一步的校验和调整。

第四，医疗问答系统的设计与性能提升。在某市卫检委健康教育科普馆项目中，集成和应用了医疗智能问答功能，设计了一个基于 RAG 与大模型技术的医疗问答系统。该系统利用 DiseaseKG 数据集与 Neo4j 构建了丰富的知识图谱，并结合了 BERT 命名实体识别和 34B 大模型意图识别的功能。通过精确的知识检索和高效的问答生成机制，该系统在医疗咨询领域展现出了卓越的性能，有效解决了大模型在医疗应用中的可靠性问题。

综上所述，本文进行基于 BERT 的医学智能问答关键算法的研究，通过引入 BERT 预训练模型、构建医学领域知识图谱以及优化实体匹配过程，有效地解决了传统问答系统在处理复杂或模糊问题时的局限性。这一算法不仅提高了问答系统的准确性和实用性，还为其在医学领域的应用提供了有力的支持。

6.2 展望

基于 BERT 的医学智能问答系统是当今人工智能领域的热点研究之一。本次设计聚焦于 BERT 算法和知识图谱构建，旨在提高问答系统的准确性和语义理解能力。BERT 算法作为一种预训练语言模型，通过双向注意力机制实现了对文本语境的深层理解，使得问答系统能够更好地理解问题和回答。通过对大规模文本语料进行预训练，BERT 可以捕捉到丰富的语义信息，从而提升了问答系统的回答准确度。

在知识图谱构建方面，通过整合结构化和半结构化数据，建立了一个包含实体、关系和属性的知识图谱。知识图谱的构建使得系统能够从海量数据中准确地抽取和理解相关信息，为智能问答提供了更加丰富和准确的语义支持。结合 BERT 算法和知识图谱，系统能够更好地理解用户提出的问题，并基于丰富的语义信息进行精准的回答。

未来的发展方向包括但不限于以下几个方面：首先，可以进一步优化 BERT 算法，针对特定领域或任务进行微调，提高问答系统在特定领域的准确性和效率。其次，可以探索将多模态数据（如文本、图像、视频等）引入问答系统，提供更加全面和多样化的信息支持。此外，还可以结合强化学习等技术，实现问答系统的自主学习和优化，使得系统能够不断适应新的场景和需求。

随着自然语言处理和知识图谱技术的不断发展，可以进一步完善知识图谱的构建和更新机制，使得系统能够及时获取最新的知识和信息。同时，还可以探索将个性化推荐技术应用于问答系统中，根据用户的偏好和历史信息，提供个性化的问答服务。

基于深度学习的智能问答系统在 BERT 算法和知识图谱构建的基础上，具有广阔的发展前景和应用潜力，将为人们的信息获取和交流带来更加便利和高效的体验。

参考文献

- [1] Schöller E F ,Plenge-Feidenhans'l K M ,Stets D J , et al.Assessing Deep-learning Methods for Object Detection at Sea from LWIR Images[J].IFAC PapersOn Line,2019,52(21):64-71.
- [2] 吴雅娟. 基于美丽乡村知识图谱的智能问答研究 [D]. 南京邮电大学,2023.DOI:10.27251/d.cnki.gnjdc.2023.000714.
- [3] 彭雨依. 基于知识图谱的玉米种植管理系统研发 [D]. 山东农业大学,2023.DOI:10.27277/d.cnki.gsdnu.2023.001341.
- [4] 徐平. 面向企业员工的智能客服系统的设计与实现 [D]. 北京交通大学,2023.DOI:10.26944/d.cnki.gbifu.2023.000469.
- [5] 卢汉康. 南水北调中线工程风险防控智能问答及应急方案智能生成[D].华北水利水电大学,2023.DOI:10.27144/d.cnki.ghbsc.2023.000465.
- [6] 冯柏杨. 炭疽病生物医学知识图谱及智能问答系统的研究与构建[D].内蒙古农业大学,2023.DOI:10.27229/d.cnki.gnmnu.2023.000770.
- [7] 张崇宇. 基于知识图谱的自动问答系统的应用研究与实现[D].北京邮电大学,2019.68(04):124-136.DOI:10.13266/j.issn.0252-3116.2024.04.010.
- [8] Valenstein L M ,Lalgudi V P ,Gu X , et al.Rag-Ragulator is the central organizer of the physical architecture of the mTORC1 nutrient-sensing pathway.[J].Proceedings of the National Academy of Sciences of the United States of America,2024,121(35):e2322755121.
- [9] Thomo A .PubMed Retrieval with RAG Techniques.[J].Studies in health technology and informatics,2024,316652-653.
- [10] Wang Y ,Leutner S ,Ingrisch M , et al.Optimizing Data Extraction: Harnessing RAG and LLMs for German Medical Documents.[J].Studies in health technology and informatics,2024,316949-950.
- [11] Kreimeyer K ,Canzoniero V J ,Fatteh M , et al.Using Retrieval-Augmented Generation to Capture Molecularly-Driven Treatment Relationships for Precision Oncology.[J].Studies in health technology and informatics,2024,316983-987.

- [12] Kim S ,Yoon J .VAIV bio-discovery service using transformer model and retrieval augmented generation[J].BMC Bioinformatics,2024,25(1):273-273.
- [13] Karadirek E I ,Rizvanoglu N A ,Okumus B , et al.Fate and transport of chlorine dioxide: Modeling chlorine dioxide in water distribution systems.[J].Water environment research : a research publication of the Water Environment Federation,2024,96(8):e11094.
- [14] Soong D ,Sridhar S ,Si H , et al.Improving accuracy of GPT-3/4 results on biomedical data using a retrieval-augmented language model.[J].PLOS digital health,2024,3(8):e0000568.
- [15] Krishnan A .S A.Morphometric analysis in the sub basins of the Kali River using Geographic Information System, Karnataka, India[J].Geomatica,2024,76(2):100013-100013.
- [16] Ye C .Exploring a learning-to-rank approach to enhance the Retrieval Augmented Generation (RAG)-based electronic medical records search engines[J].Informatics and Health,2024,1(2):93-99.
- [17] Wang Q ,Yang H W .An adaptive regularized proximal Newton-type methods for composite optimization over the Stiefel manifold[J].Computational Optimization and Applications,2024,(prepublish):1-39.
- [18] 梁水芬,郭志勇.深度学习在肾结石影像诊断的应用进展[J/OL].电子科技,1-7[2024-03-24].<https://doi.org/10.16180/j.cnki.issn1007-7820.2024.12.010>.
- [19] 张文耀.基于深度学习的医疗知识图谱问答系统研究与实现[D].山西大学,2023.DOI:10.27284/d.cnki.gsxiu.2023.001900.
- [20] Gao M ,Li M ,Ji T , et al.Key Technologies of Intelligent Question-Answering System for Power System Rules and Regulations Based on Improved BERTserial Algorithm[J].Processes,2023,12(1):
- [21] Developing an Intelligent Quiz System based on Genetic Algorithm[J].International Journal of Innovative Technology and Exploring Engineering,2019,8(10):70-76.
- [22] 赵佳博.基于农业病虫害知识图谱的智能问答系统的设计与实现[D].东北农业大学,2022.DOI:10.27010/d.cnki.gdbnu.2022.000254.

- [23] 刘亚东.农业机械化智能问答系统构建[D].华中农业大学,2022.DOI:10.27158/d.cnki.ghznu.2022.001759.
- [24] 刘明昊.面向开源数据 SRE 知识图谱构建与应用[D].内蒙古大学,2023.DOI:10.27224/d.cnki.gnmdu.2023.000977.
- [25] 乔温静.基于数字乡村模型的农村管理服务系统的设计与开发[D].河北北方学院,2023.DOI:10.27767/d.cnki.ghbbf.2023.000130.
- [26] 杨雪茹.融合多特征的保险行业智能问答中的文本检索研究[D].大连海事大学,2023.DOI:10.26989/d.cnki.gdlhu.2023.001045.
- [27] 方雨桐.基于知识图谱的中文智能问答方法研究[D].桂林理工大学,2023.DOI:10.27050/d.cnki.gglgc.2023.000433.
- [28] 马超.电力安规智能问答系统设计与实现[D].河北工程大学,2023.DOI:10.27104/d.cnki.ghbjy.2023.000430.
- [29] 李紫洲.基于机器学习的玉米病虫害智能问答系统设计与实现[D].华中师范大学,2022.DOI:10.27159/d.cnki.ghzsu.2022.003225.
- [30] Zhang F ,Li X .Knowledge-enhanced online doctor recommendation framework based on knowledge graph and joint learning[J].Information Sciences,2024,6621-20268-.
- [31] Wang C ,Shi W ,Lv H .Construction of Remote Sensing Indices Knowledge Graph (RSIKG) Based on Semantic Hierarchical Graph[J].Remote Sensing,2023,16(1):
- [32] 徐尚,蔡瑾如,胡玉龙.运动性疲劳相关研究热点及嬗变:基于文献计量及知识图谱分析[J].南京体育学院学报,2023,22(12):40-47.DOI:10.15877/j.cnki.nsin.2023.12.012.
- [33] 江永亨,任艳频,唐潇风.高校实验能力图谱基础架构及关键问题探讨[J].2023,40(12):187-191.DOI:10.16791/j.cnki.sjg.2023.12.027.
- [34] 王乾龙,王丽颖.基于知识图谱的数控机床故障问答系统研究[J].内蒙古科技大学学报,2023,42(04):372-376+393.DOI:10.16559/j.cnki.2095-2295.2023.04.013.
- [35] Cai Y ,Zhu C ,Lu S , et al.Arsenic sulfide enhances radiosensitivity in rhabdo

- myosarcoma via activating NFATc3-RAG1 mediated DNA double strand break (DSB)[J].Chemico-Biological Interactions,2024,399111149-111149.
- [36]Boulos K N M ,Dellavalle R .NVIDIA's "Chat with RTX" Custom Large Language Model and Personalized AI Chatbot Augments the Value of Electronic Dermatology Reference Material.[J].JMIR dermatology,2024,7e58396.
- [37]Hristova E ,Stoyanov R S ,McFarlane R , et al.Investigation of Unresolved Interface "Rag Layer" in Athabasca Oil Sand Bitumen In Situ Recovery.[J].ACS omega,2024,9(29):32201-32209.
- [38]Erratum for the Research Article "The lysosomal Rag-Ragulator complex licenses RIPK1- and caspase-8-mediated pyroptosis by Yersinia" by Z. Zheng et al. [J].Science (New York, N.Y.),2024,385(6706):eadr5480.
- [39]Zhang S ,Zhu F ,Deveci M , et al.Consensus-oriented linguistic multi-criteria group sorting method incorporating dynamic trust management[J].Information Fusion,2024,112102539-102539.
- [40]Irene S ,Marco F ,Andrea B .Ragopathies and the rising influence of RagGTPases on human diseases[J].Nature Communications,2024,15(1):5812.
- [41]Moron B J ,Klink V P G ,Gruter M J G .From rags to riches: Converting cellulose containing waste to 5-(chloromethyl)furfural (CMF)[J].Waste Management Bulletin,2024,2(3):58-68.
- [42]Jackson J K .RAGging on recombination signal sequence strength for diffusion-mediated recombination.[J].Immunology and cell biology,2024,
- [43]Suresh K ,Kackar N ,Schleck L , et al.Towards a RAG-based summarization for the Electron Ion Collider[J].Journal of Instrumentation,2024,19(07):
- [44]Zwickel M .Risikotragung für fehlerhafte oder überhöhte Rechnungen im Haftungsrecht des Straßenverkehrs[J].Monatsschrift für Deutsches Recht,2024,78(13):809-815.
- [45]钟来民,陆卫忠,傅启明,马洁明,崔志明,吴宏杰.基于 Transformer-BiLSTM 特征融合的 DNA 结合蛋白预测方法[J].微电子学与计算机,2023,(12):1-9.
- [46]吕潇涵.基于改进 Vision Transformer 的光伏电池缺陷识别研究[J].计算技术与自动化,2023,42(04):33-40.

- [47] 刘亚东.农业机械化智能问答系统构建[D].华中农业大学,2022.
- [48] 李紫洲.基于机器学习的玉米病虫害智能问答系统设计与实现[D].华中师范大学, 2022.
- [49] Yongqiang S . Prevention and detection of DDOS attack in virtual cloud computing environment using Naive Bayes algorithm of machine learning [J]. Measurement: Sensors, 2024, 31.
- [50] Nakhipova V ,Kerimbekov Y ,Umarova Z , et al. Use of the Naive Bayes Classifier Algorithm in Machine Learning for Student Performance Prediction [J]. International Journal of Information and Education Technology, 2024, 14 (1):
- [51] Yurong G. Exploring the application of teaching evaluation models incorporating association rules and weighted naive Bayesian algorithms [J]. Intelligent Systems with Applications, 2023, 20
- [52] 李政.基于嵌套命名实体识别算法的知识抽取系统设计与实现[D].华中科技大学, 2022.
- [53] 刘巨升, 杨惠宁, 孙哲涛, 杨鹤, 邵立铭, 于红, 张思佳, 叶仕根.面向知识图谱构建的水产动物疾病诊治命名实体识别[J].农业工程学报,2022,38(07):210-217.
- [54] 劳雅迪.基于强化学习的命名实体识别算法研究[D]. 北京邮电大学, 2019.
- [55] 徐梓豪.基于统计模型的中文命名实体识别方法研究及应用[D]. 北京化工大学, 2017.
- [56] Zhang T ,Huang Z ,Lu M , et al.Interactive spatiotemporal LSTM approach for enhanced industrial fault diagnosis[J].Industrial Lubrication and Tribology,2024, 76(2):149-159.
- [57] Wesley J B ,Babu S G ,Kumar S P .Design and control of LSTM-ANN controllers for an efficient energy management system in a smart grid based on hybrid renewable energy sources[J].Engineering Research Express,2024,6(1):
- [58] 金柯君, 于洪涛, 李邵梅, 张建朋. 基于注意力机制的图卷积网络防御方法[J].信息工程大学学报,2023,24(06):718-724.
- [59] 贾康, 李晓楠, 李冠宇. 一种基于自适应结构感知池化图匹配的图相似度计算模型[J]. 计算机工程与科学,2023,45(11):1999-2007.

- [60] 陈江萍.面向行人重识别的注意力机制及特征多样化提取技术研究[D].南京邮电大学, 2023.
- [61] 徐淑强.基于时空注意力机制的位置推荐模型研究[D]. 南京邮电大学, 2023.
- [62] Qiao S, Liu C, Shen W, et al. Few-shot image recognition by predicting parameters from activations[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 7229-7238.
- [63] Sung F, Yang Y, Zhang L, et al. Learning to compare: Relation network for few-shot learning[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 1199-1208.
- [64] Li W, Wang L, Xu J, et al. Revisiting local descriptor based image-to-class measure for few-shot learning[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2019: 7260-7268.
- [65] Li H, Eigen D, Dodge S, et al. Finding task-relevant features for few-shot learning by category traversal[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2019: 1-10.
- [66] 张元鸣,姬琦,徐雪松,等.基于知识图谱关系路径的多跳智能问答模型研究[J].电子学报,2023,51(11):3092-3099.
- [67] 赵伟航. 基于知识图谱的糖尿病智能问答系统关键技术研究及应用[D].西京学院,2023.DOI:10.27831/d.cnki.gxjxy.2023.000075.
- [68] 张羽立. 面向高校学科数据知识图谱构建中知识抽取和智能问答技术的研究与实现[D].南京邮电大学,2023.DOI:10.27251/d.cnki.gnjdc.2023.000718.
- [69] 丁仕涛. 基于茶叶病虫害知识图谱的智能问答系统设计与实现[D].四川农业大学,2023.DOI:10.27345/d.cnki.gsnyu.2023.000832.
- [70] 庞文婷. 基于知识图谱的玉米育种智能问答系统研发[D].西北农林科技大学,2023.DOI:10.27409/d.cnki.gxbnu.2023.000457.
- [71] 翟铁英. 面向应用系统的智能问答系统框架研究 [D]. 燕山大学,2023.DOI:10.27440/d.cnki.gysdu.2023.001718.
- [72] 蒋涛涛. 面向政务领域的智能问答研究与应用 [D]. 贵州大

- 学,2023.DOI:10.27047/d.cnki.ggudu.2023.001721.
- [73] 李园博. 学习质量体验驱动下的虚拟仿真教学平台设计与实现[D]. 北京交通大学,2023.DOI:10.26944/d.cnki.gbfju.2023.002375.
- [74] 王党炜. 基于领域知识图谱的智能问答系统研究[D]. 西安工业大学,2024.DOI:10.27391/d.cnki.gxagu.2023.000595.
- [75] 吕锡志. 知识管理视域下基于知识图谱的智能问答研究[D]. 安徽财经大学,2024.DOI:10.26916/d.cnki.gahcc.2023.000864.
- [76] 赵智博. 基于多源异构数据的煤层气知识图谱构建与应用研究[D]. 中国矿业大学,2024.DOI:10.27623/d.cnki.gzkyu.2023.000302.
- [77] 张博凯,李想.基于知识图谱的 Android 端农技智能问答系统研究[J].农业机械学报,2021,52(S1):164-171.
- [78] 聂同攀,曾继炎,程玉杰,等.面向飞机电源系统故障诊断的知识图谱构建技术及应用[J].航空学报,2022,43(08):46-62.
- [79] 刘壮,刘畅,Wayne Lin,等.用于金融文本挖掘的多任务学习预训练金融语言模型[J].计算机研究与发展,2021,58(08):1761-1772.
- [80] 蒲天骄,谈元鹏,彭国政,等.电力领域知识图谱的构建与应用[J].电网技术,2021,45(06):2080-2091.DOI:10.13335/j.1000-3673.pst.2020.2145.
- [81] 朱建楠,梁玉琦,顾复,等.基于深度学习的机械智能制造知识问答系统设计[J].计算机集成制造系统,2019,25(05):1161-1168.DOI:10.13196/j.cims.2019.05.014.
- [82] 赵明,董翠翠,董乔雪,等.基于 BIGRU 的番茄病虫害问答系统问句分类研究[J].农业机械学报,2018,49(05):271-276.
- [83] 龙新征,郑建宁,欧阳荣彬,等.基于多层策略的校园智能问答系统[J].华中科技大学学报(自然科学版),2016,44(11):117-122.